

本节内容

二叉树

先/中/后序
遍历

知识总览

二叉树的遍历

先序遍历

中序遍历

后序遍历

遍历算法的应用举例

什么是遍历

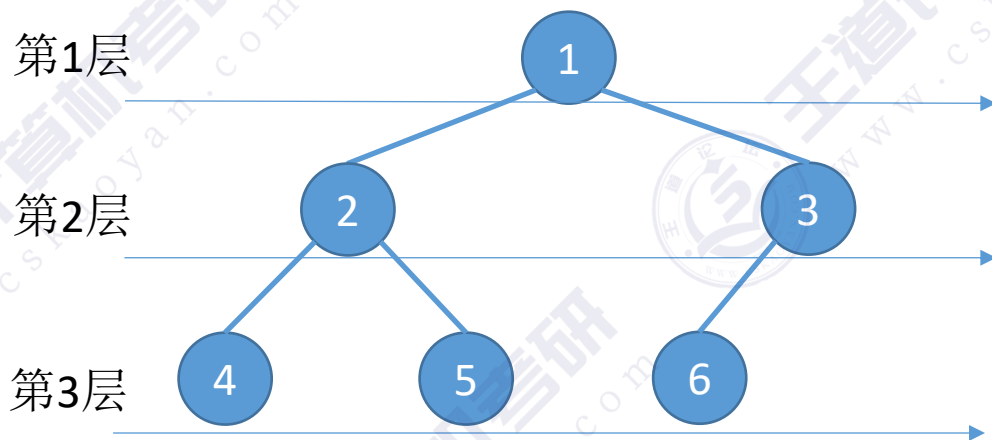
遍历：按照某种次序把所有结点都访问一遍



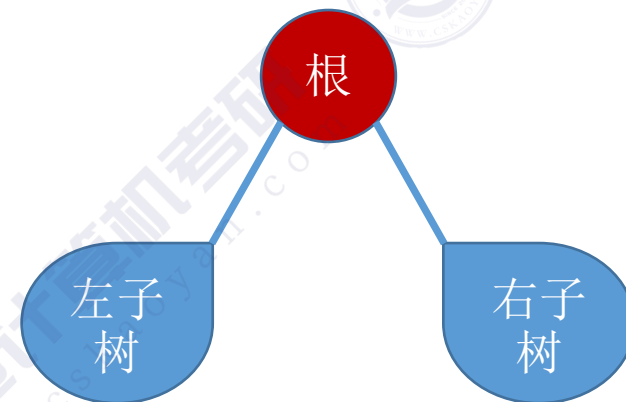
线性结构



简单来说



层次遍历：基于树的层次特性确定的次序规则



先/中/后序遍历：基于树的递归特性确定的次序规则

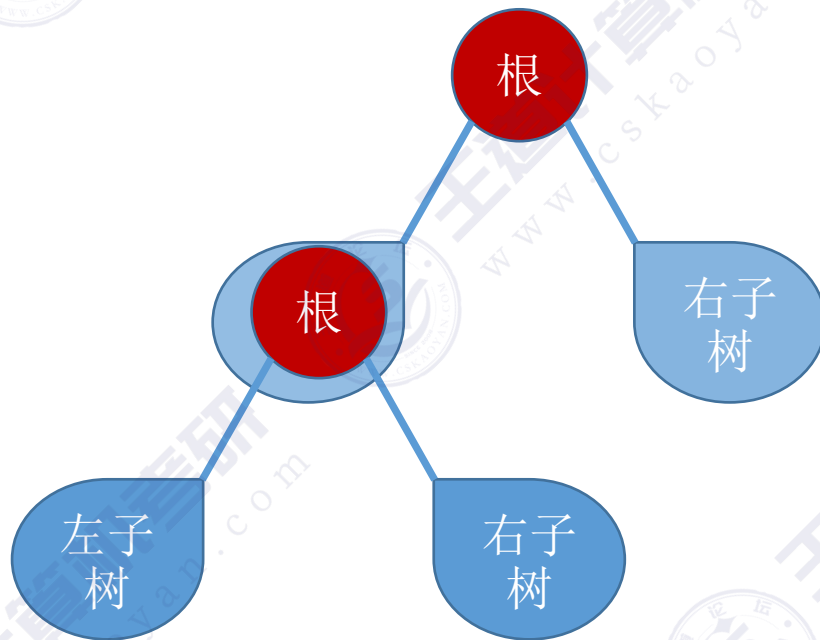
二叉树的遍历

二叉树的递归特性:

- ① 要么是个空二叉树
- ② 要么就是由“根节点+左子树+右子树”组成的二叉树



空二叉树



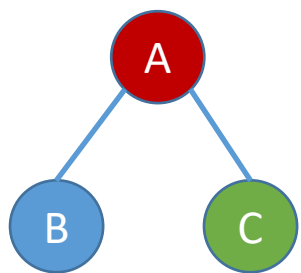
非空二叉树

先序遍历: 根左右 (NLR)

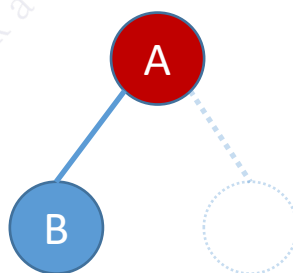
中序遍历: 左根右 (LNR)

后序遍历: 左右根 (LRN)

二叉树的遍历

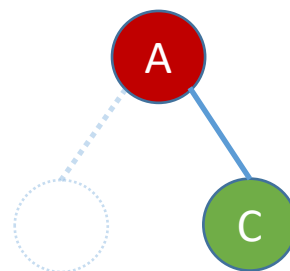


先序遍历: **A**BC
中序遍历: B**A**C
后序遍历: BC**A**



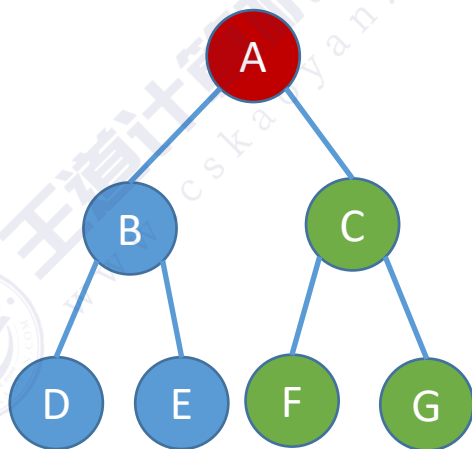
先序遍历: **A**B
中序遍历: B**A**
后序遍历: B**A**

右子树
为空树



先序遍历: **A**C
中序遍历: **A**C
后序遍历: C**A**

左子树
为空树



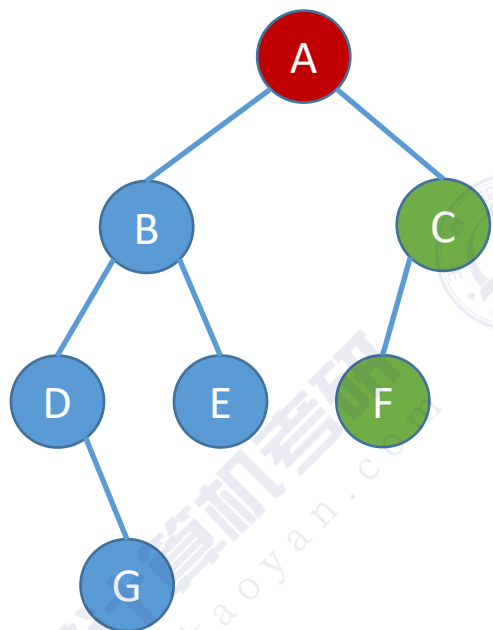
先序遍历: **A** B D E C F G
中序遍历: D B E **A** F C G
后序遍历: D E B F G C **A**

先序遍历: 根左右 (N**L**R)

中序遍历: 左根右 (L**N**R)

后序遍历: 左右根 (LR**N**)

二叉树的遍历（手算练习）



先序遍历：根 左 右
 根 (根 左 右) (根 左)
 根 (根 (根 右) 右) (根 左)

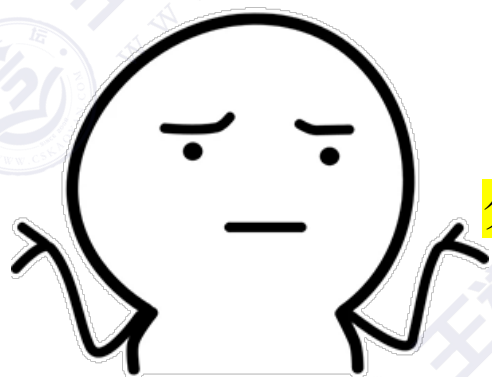
A B C
 A B D E C F
 A B D G E C F

中序遍历：左 根 右
 (左 根 右) 根 (左 根)
 ((根 右) 根 右) 根 (左 根)

B A C
 D B E A F C
 D G B E A F C

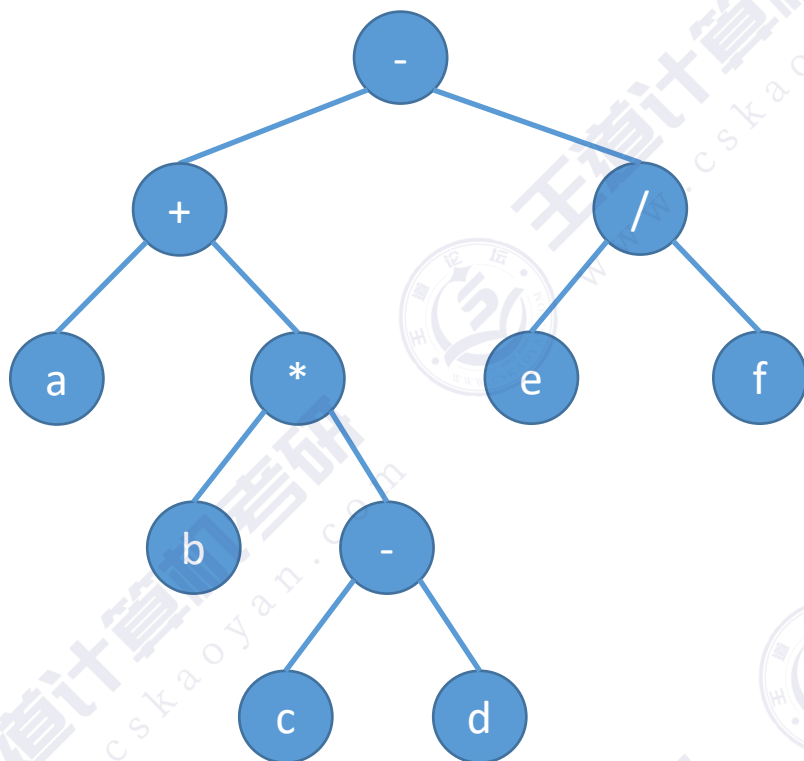
后序遍历：左 右 根
 (左 右 根) (左 右 根) 根
 ((右 根) 右 根) (左 根) 根

B C A
 D E B F C A
 G D E B F C A



分支结点逐层展开法...

二叉树的遍历（手算练习）



先序遍历: $-+a*b-cd/ef$

中序遍历: $a+b*c-d-e/f$

后序遍历: $abcd-*+ef/-$

先序遍历 → 前缀表达式

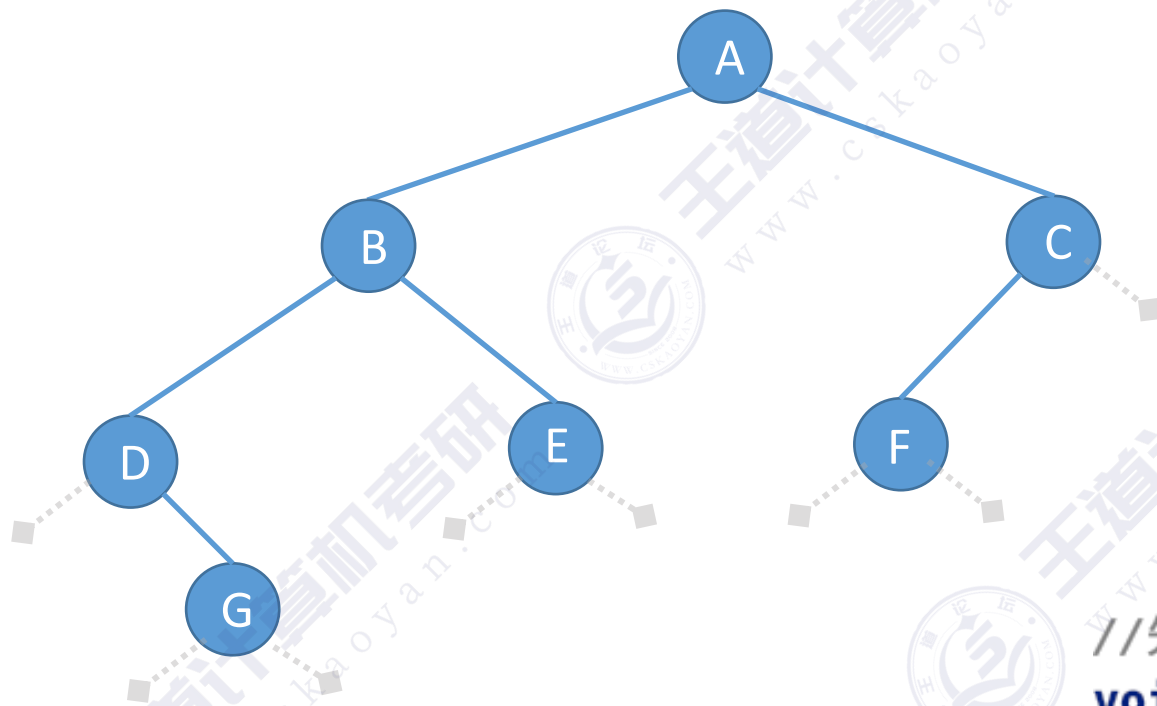
中序遍历 → 中缀表达式（需要加界限符）

后序遍历 → 后缀表达式

算数表达式的“分析树”

$$a + b * (c - d) - e / f$$

先序遍历（代码）



先序遍历（PreOrder）的操作过程如下：

1. 若二叉树为空，则什么也不做；
2. 若二叉树非空：

①访问根结点；

②先序遍历左子树；

③先序遍历右子树。

//先序遍历

```
void PreOrder(BiTree T){
```

```
    if(T!=NULL){
```

```
        visit(T);
```

```
        PreOrder(T->lchild);
```

```
        PreOrder(T->rchild);
```

```
    }
```

```
}
```

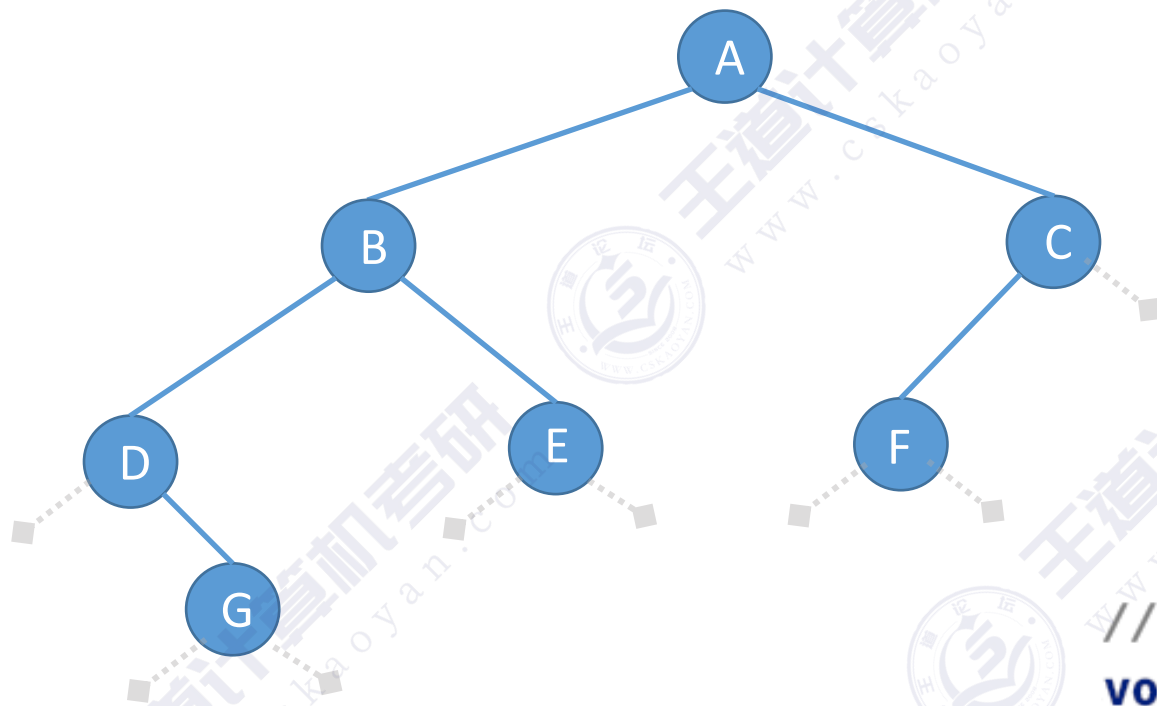
//访问根结点

//递归遍历左子树

//递归遍历右子树

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```


中序遍历（代码）



中序遍历（InOrder）的操作过程如下：

1. 若二叉树为空，则什么也不做；
2. 若二叉树非空：
 - ①中序遍历左子树；
 - ②访问根结点；
 - ③中序遍历右子树。

//中序遍历

```
void InOrder(BiTree T){
```

```
    if(T!=NULL){
```

```
        InOrder(T->lchild);
```

//递归遍历左子树

```
        visit(T);
```

//访问根结点

```
        InOrder(T->rchild);
```

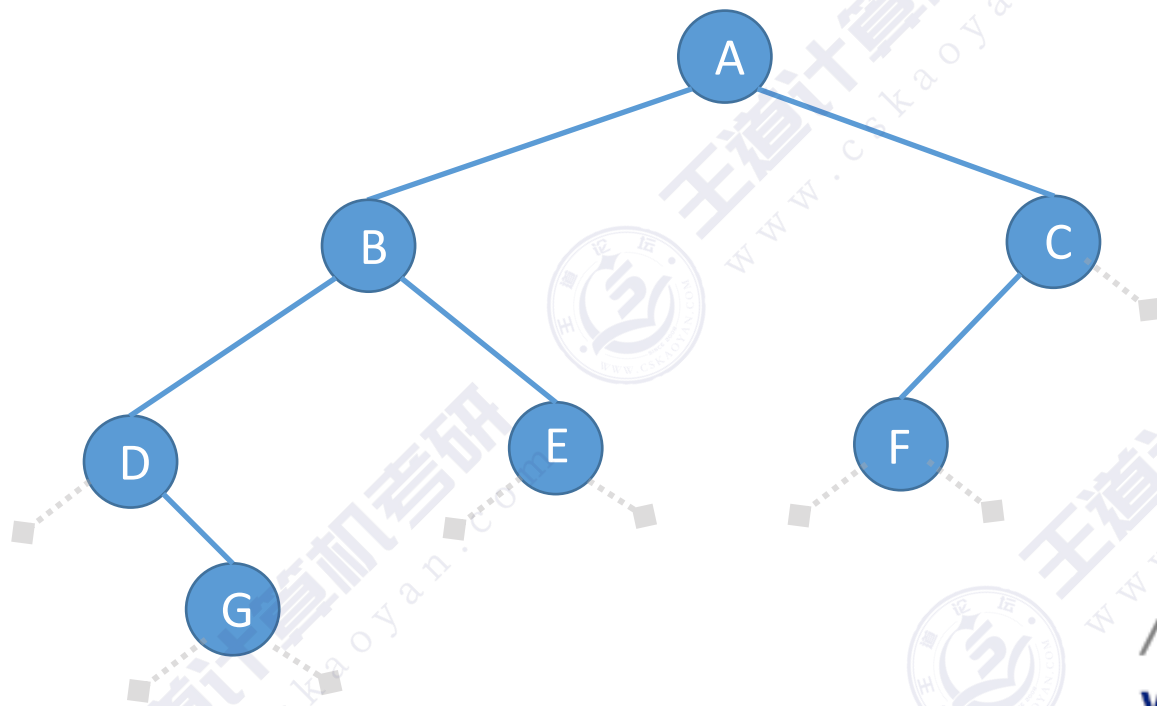
//递归遍历右子树

```
    }
```

```
}
```

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

后序遍历（代码）



后序遍历（InOrder）的操作过程如下：

1. 若二叉树为空，则什么也不做；
2. 若二叉树非空：
 - ①后序遍历左子树；
 - ②后序遍历右子树；
 - ③访问根结点。

//后序遍历

```
void PostOrder(BiTree T){
```

```
    if(T!=NULL){
```

```
        PostOrder(T->lchild);
```

//递归遍历左子树

```
        PostOrder(T->rchild);
```

//递归遍历右子树

```
        visit(T);
```

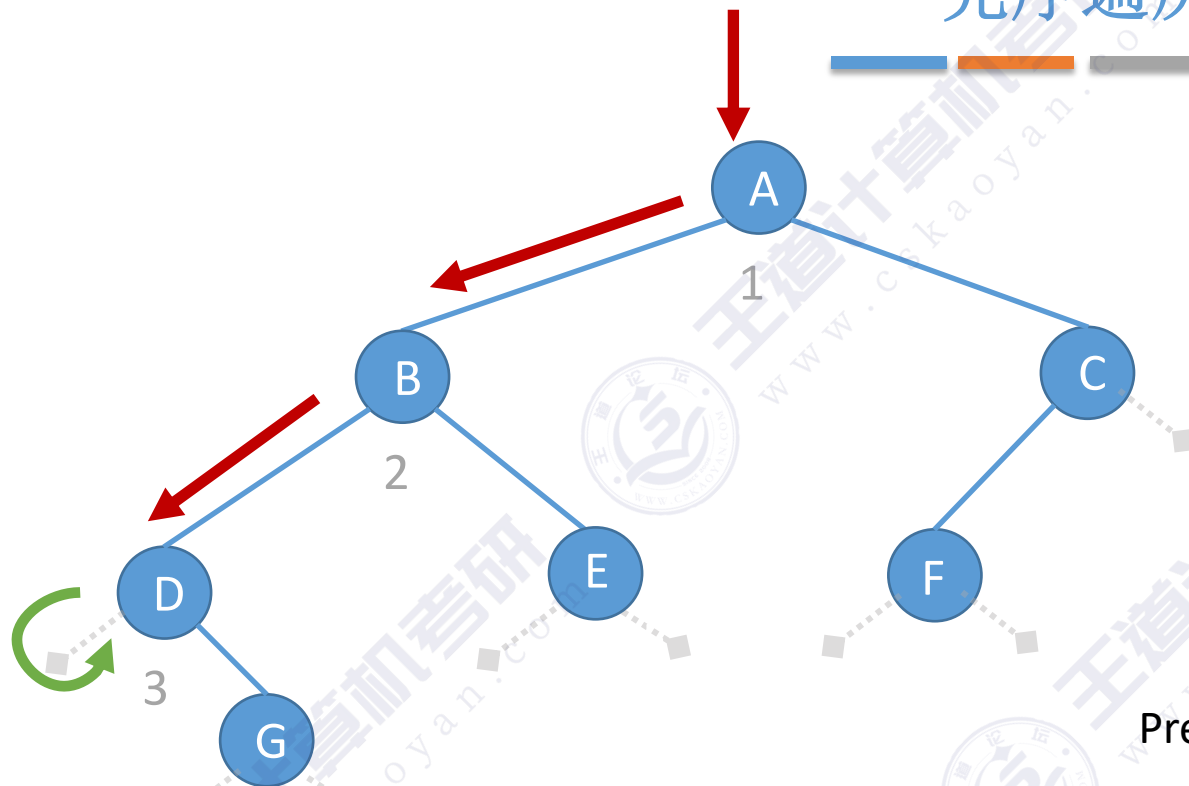
//访问根结点

```
    }
```

```
}
```

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild,*rchild;
}BiTNode,*BiTree;
```

先序遍历（代码）



```
122 //先序遍历
123 void PreOrder(BiTree T){
124     if(T!=NULL){
125         visit(T);
126         PreOrder(T->lchild);
127         PreOrder(T->rchild);
128     }
129 }
```

PreOrder:

PreOrder:

PreOrder:

PreOrder:

T==NULL

T==D #126

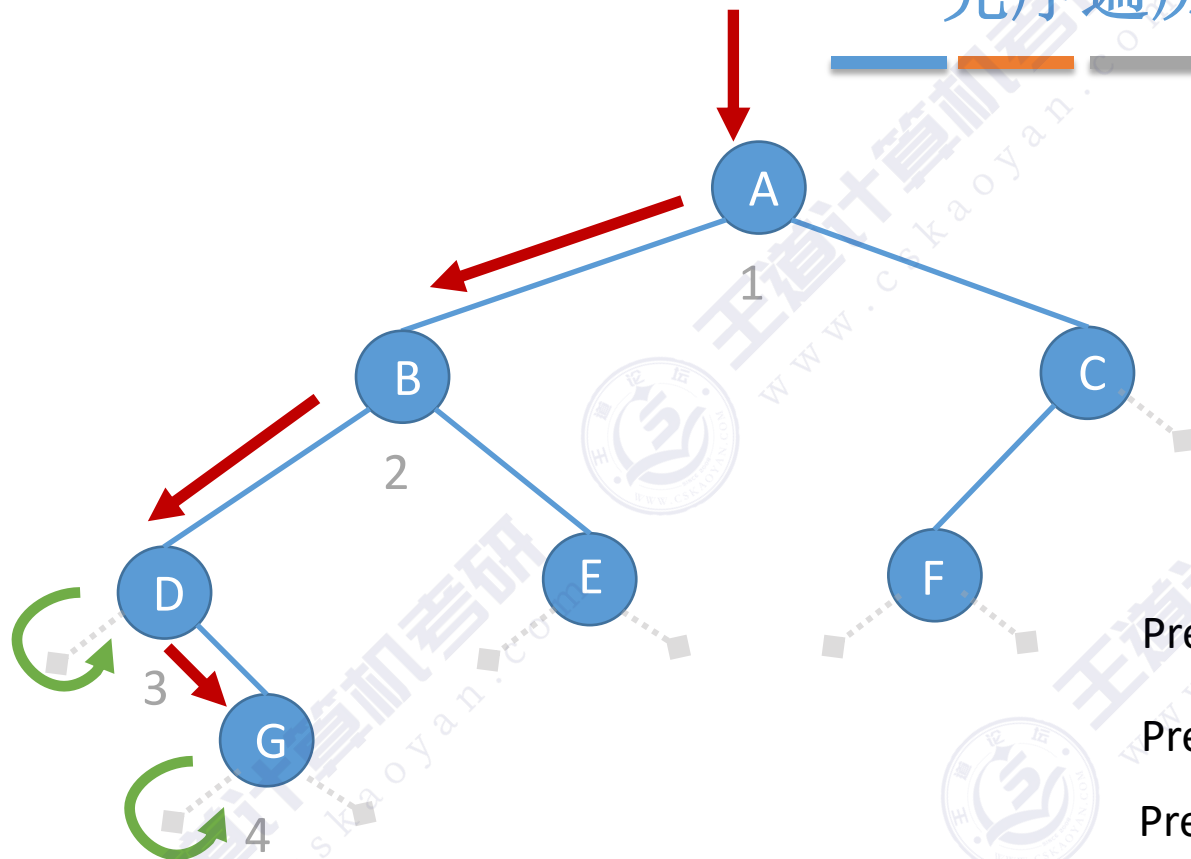
T==B #126

T==A #126

.....

函数调用栈

先序遍历（代码）



```
122 //先序遍历
123 void PreOrder(BiTree T){
124     if(T!=NULL){
125         visit(T);
126         PreOrder(T->lchild);
127         PreOrder(T->rchild);
128     }
129 }
```

PreOrder:

T==NULL

PreOrder:

T==G #126

PreOrder:

T==D #127

PreOrder:

T==B #126

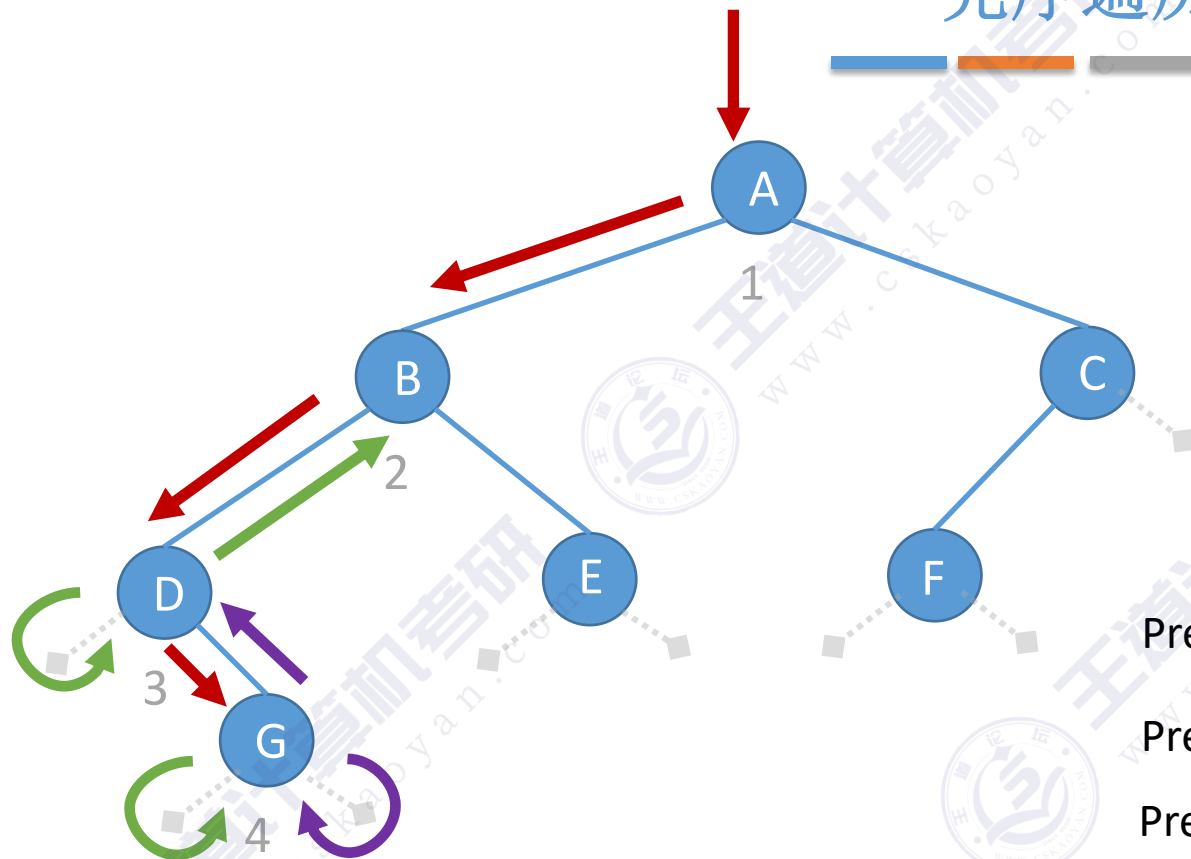
PreOrder:

T==A #126

.....

函数调用栈

先序遍历（代码）



```
122 //先序遍历
123 void PreOrder(BiTree T){
124     if(T!=NULL){
125         visit(T);
126         PreOrder(T->lchild);
127         PreOrder(T->rchild);
128     }
129 }
```

PreOrder:

T==NULL

PreOrder:

T==G #127

PreOrder:

T==D #127

PreOrder:

T==B #126

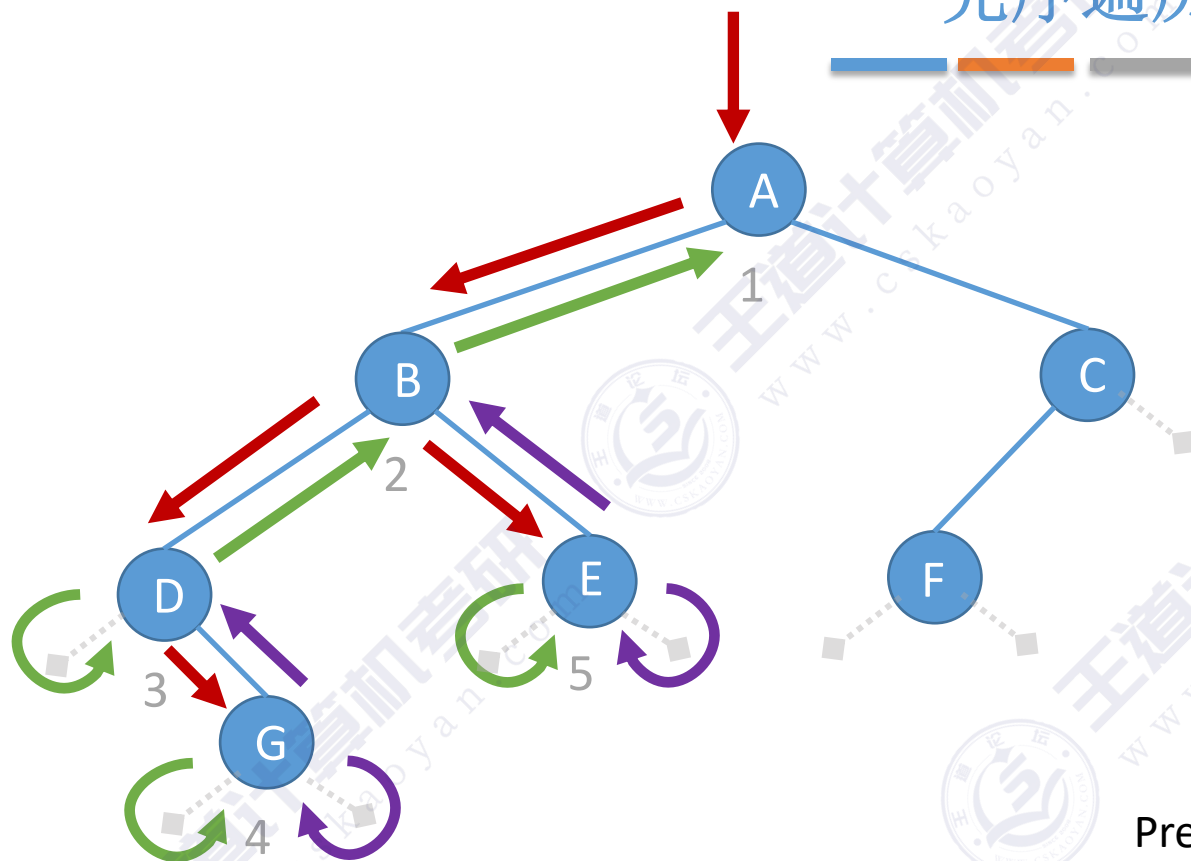
PreOrder:

T==A #126

.....

函数调用栈

先序遍历（代码）



```
122 //先序遍历
123 void PreOrder(BiTree T){
124     if(T!=NULL){
125         visit(T);
126         PreOrder(T->lchild);
127         PreOrder(T->rchild);
128     }
129 }
```

PreOrder:

T==E

PreOrder:

T==B #127

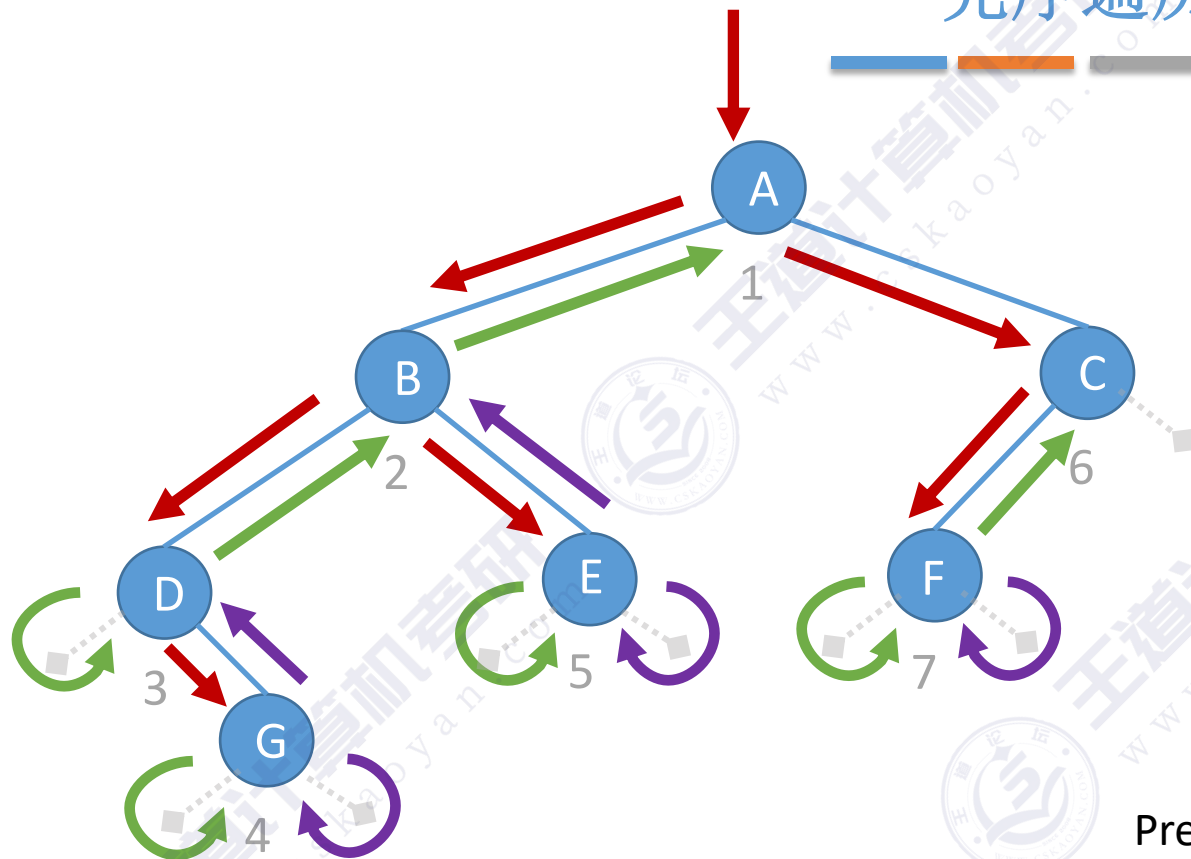
PreOrder:

T==A #126

.....

函数调用栈

先序遍历（代码）

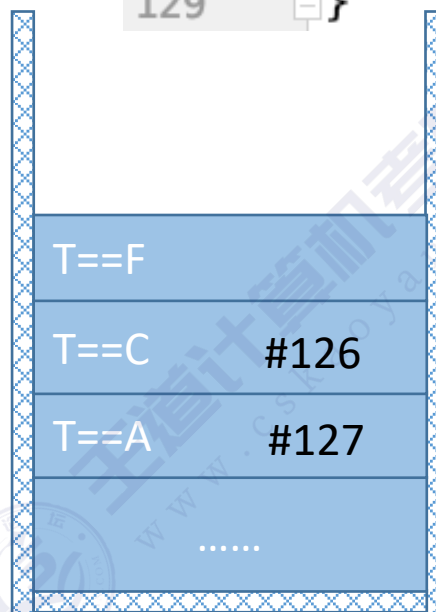


```
122 //先序遍历
123 void PreOrder(BiTree T){
124     if(T!=NULL){
125         visit(T);
126         PreOrder(T->lchild);
127         PreOrder(T->rchild);
128     }
129 }
```

PreOrder:

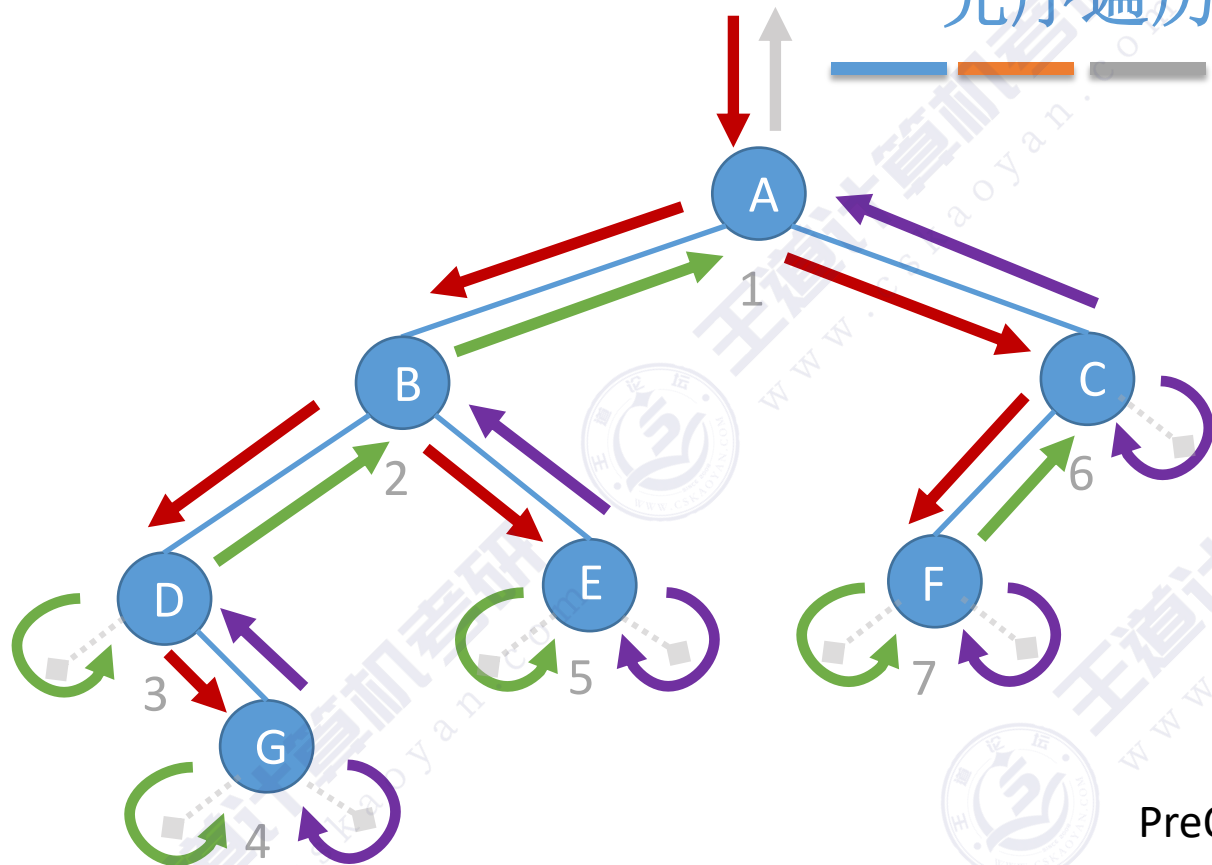
PreOrder:

PreOrder:



函数调用栈

先序遍历（代码）

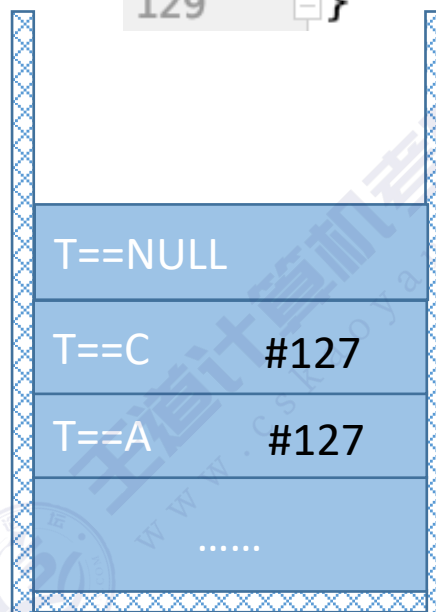


```
122 //先序遍历
123 void PreOrder(BiTree T){
124     if(T!=NULL){
125         visit(T);
126         PreOrder(T->lchild);
127         PreOrder(T->rchild);
128     }
129 }
```

PreOrder:

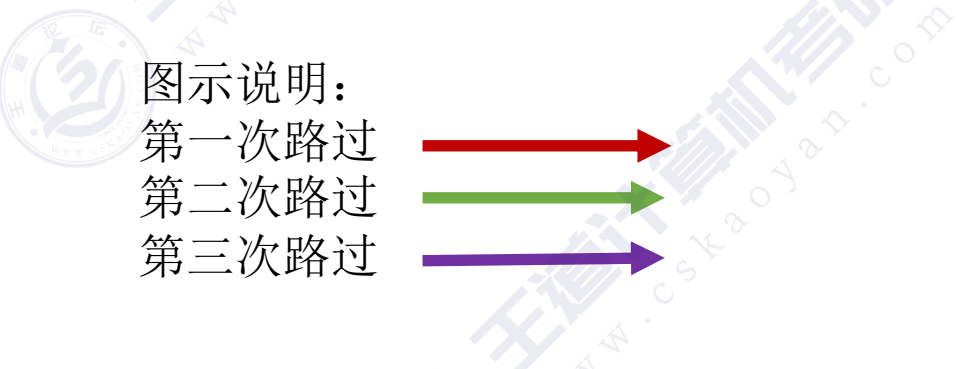
PreOrder:

PreOrder:



空间复杂度:
 $O(h)$

函数调用栈



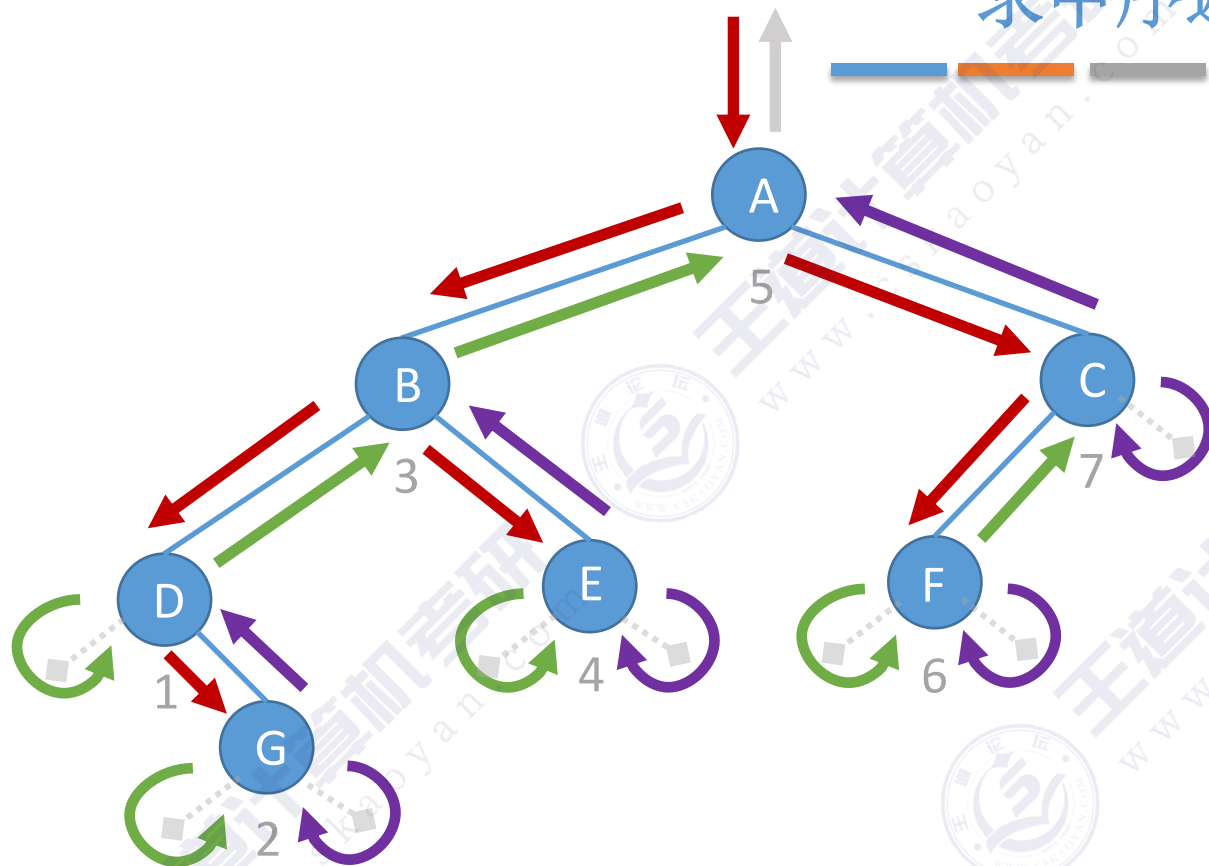
每个结点都会被路过3次

1. 若二叉树为空，则什么也不做；
2. 若二叉树非空：

- ①访问根结点;
- ②先序遍历左子树;
- ③先序遍历右子树。

先序遍历——第一次路过时访问结点

求中序遍历序列



图示说明：
第一次路过
第二次路过
第三次路过



每个结点都
会被路过3次

中序遍历 (InOrder) 的操作过程如下：

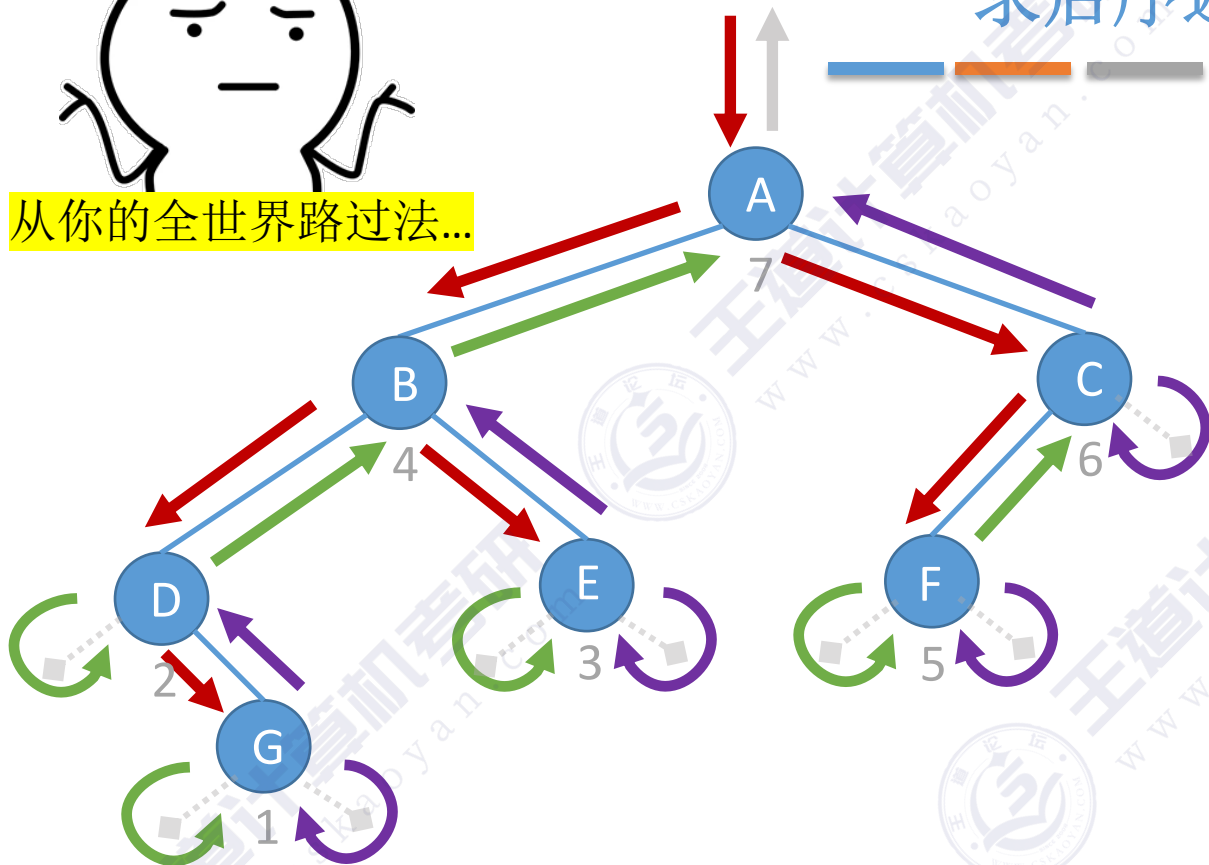
1. 若二叉树为空，则什么也不做；
2. 若二叉树非空：
 - ①中序遍历左子树；
 - ②访问根结点；
 - ③中序遍历右子树。

脑补空结点，从根节点出发，画一条路：
如果左边还有没走的路，优先往左边走
走到路的尽头（空结点）就往回走
如果左边没路了，就往右边走
如果左、右都没路了，则往上面走
中序遍历——第二次路过时访问结点



从你的全世界路过法...

求后序遍历序列



图示说明：
第一次路过
第二次路过
第三次路过

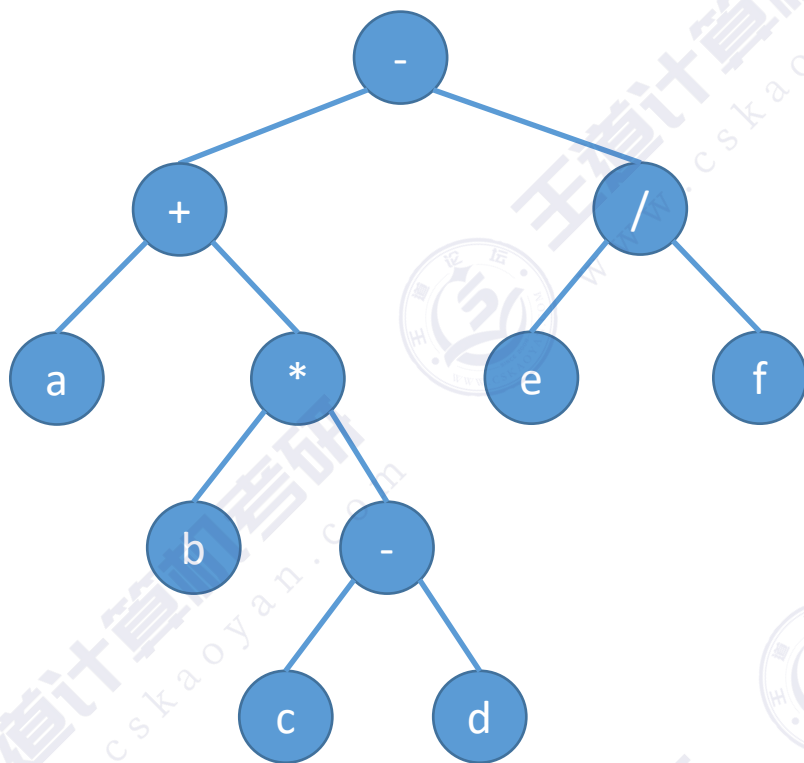
每个结点都会被路过3次

后序遍历 (InOrder) 的操作过程如下：

1. 若二叉树为空，则什么也不做；
2. 若二叉树非空：
 - ①后序遍历左子树；
 - ②后序遍历右子树；
 - ③访问根结点。

脑补空结点，从根节点出发，画一条路：
如果左边还有没走的路，优先往左边走
走到路的尽头（空结点）就往回走
如果左边没路了，就往右边走
如果左、右都没路了，则往上面走
后序遍历——第三次路过时访问结点

二叉树的遍历（手算练习）



先序遍历: $-+a*b-cd/ef$

中序遍历: $a+b*c-d-e/f$

后序遍历: $abcd-*+ef/-$

先序遍历 → 前缀表达式

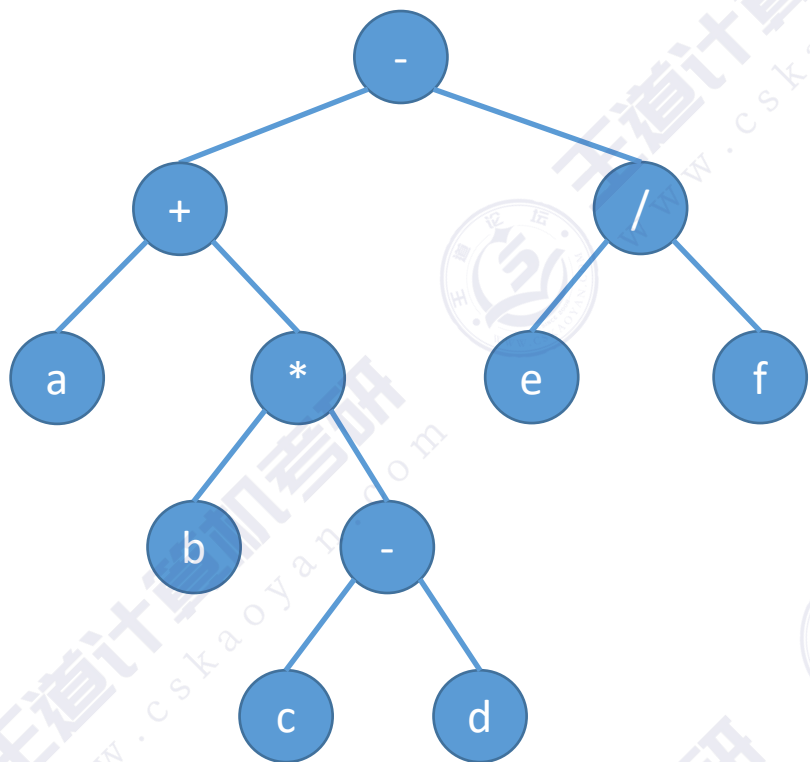
中序遍历 → 中缀表达式（需要加界限符）

后序遍历 → 后缀表达式

算数表达式的“分析树”

$$a + b * (c - d) - e / f$$

例：求树的深度（应用）



```
int treeDepth(BiTree T){  
    if (T == NULL) {  
        return 0;  
    }  
    else {  
        int l = treeDepth(T->lchild);  
        int r = treeDepth(T->rchild);  
        //树的深度=Max(左子树深度, 右子树深度)+1  
        return l>r ? l+1 : r+1;  
    }  
}
```


知识回顾与重要考点

空间复杂度:
 $O(h)$

二叉树的遍历

三种方法

先序遍历 ⊖ 根、左、右

中序遍历 ⊖ 左、根、右

后序遍历 ⊖ 左、右、根

遍历算数表达式树

先序遍历得前缀表达式

中序遍历得中缀表达式 (没有括号)

后序遍历得后缀表达式

分支结点逐层展开法...

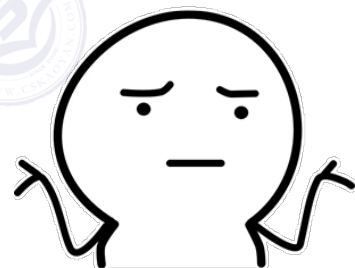
考点: 求遍历序列

从你的全世界路过法

先序——第一次路过时访问

中序——第二次路过时访问

后序——第三次路过时访问



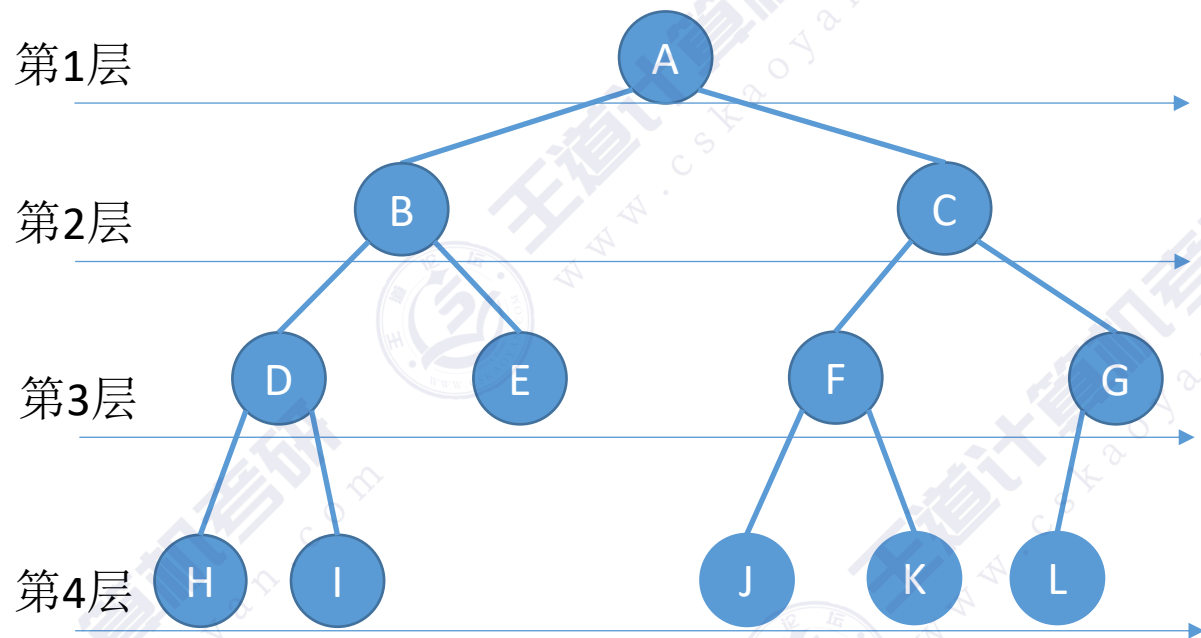
脑补空结点, 从根节点出发, 画一条路:
如果左边还有没走的路, 优先往左边走
走到路的尽头 (空结点) 就往回走
如果左边没路了, 就往右边走
如果左、右都没路了, 则往上面走

本节内容

二叉树

层序遍历

二叉树的层序遍历



算法思想:

- ①初始化一个辅助**队列**
- ②根结点入队
- ③若队列非空, 则队头结点出队, 访问该结点, 并将其左、右孩子插入队尾 (如果有的话)
- ④重复③直至队列为空

代码实现

算法思想:

- ①初始化一个辅助**队列**
- ②根结点入队
- ③若队列非空, 则队头结点出队, 访问该结点, 并将其左、右孩子插入队尾 (如果有的话)
- ④重复③直至队列为空

//层序遍历

```
void LevelOrder(BiTree T){
```

```
    LinkQueue Q;
```

```
    InitQueue(Q);
```

```
    BiTree p;
```

```
    EnQueue(Q,T);
```

```
    while(!IsEmpty(Q)){
```

```
        DeQueue(Q, p);
```

```
        visit(p);
```

```
        if(p->lchild!=NULL)
```

```
            EnQueue(Q,p->lchild);
```

```
        if(p->rchild!=NULL)
```

```
            EnQueue(Q,p->rchild);
```

```
    }
```

```
}
```

//初始化辅助队列

//将根结点入队

//队列不空则循环

//队头结点出队

//访问出队结点

//左孩子入队

//右孩子入队

//二叉树的结点 (链式存储)

```
typedef struct BiTNode{
```

```
    char data;
```

```
    struct BiTNode *lchild,*rchild;
```

```
}BiTNode,*BiTree;
```

//链式队列结点

```
typedef struct LinkNode{
```

```
    BiTNode * data;
```

```
    struct LinkNode *next;
```

```
}LinkNode;
```

```
typedef struct{
```

```
    LinkNode *front,*rear; //队头队尾
```

```
}LinkQueue;
```

存指针而
不是结点

知识回顾与重要考点

树的层次遍历算法思想:

- ①初始化一个辅助**队列**
- ②根结点入队
- ③若队列非空，则队头结点出队，访问该结点，并将其左、右孩子插入队尾（如果有的话）
- ④重复③直至队列为空

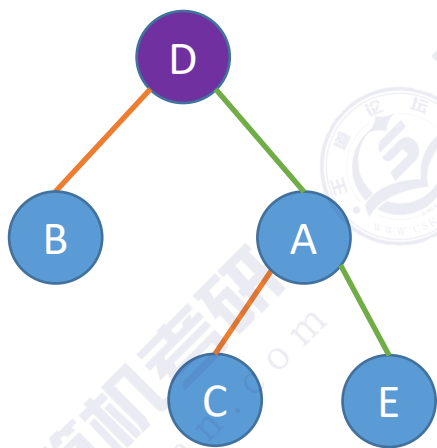
本节内容

由遍历序列

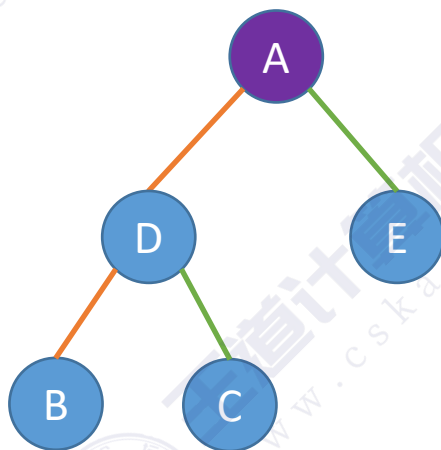
构造二叉树

不同二叉树的中序遍历序列

中序遍历：中序遍历左子树、根结点、中序遍历右子树

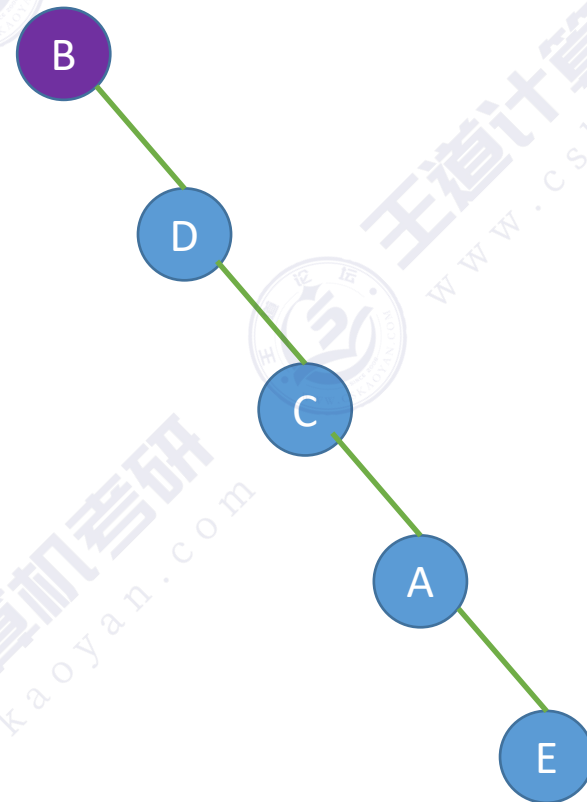


中序遍历序列：BDCAE



中序遍历序列：BDCAE

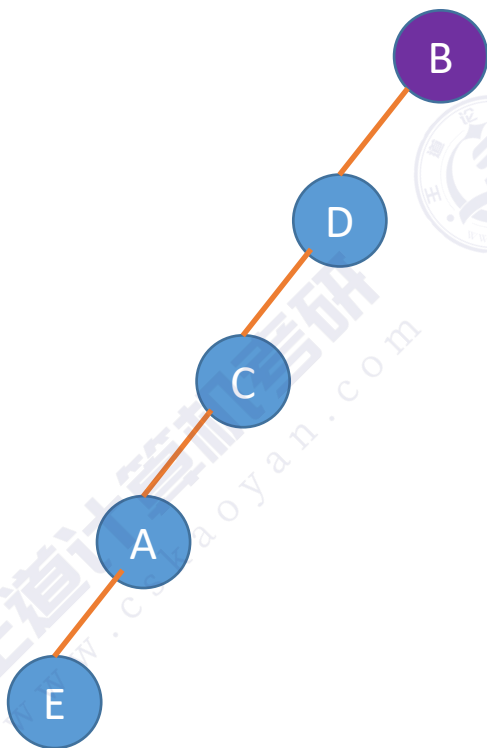
结论：一个中序遍历序列可能对应多种二叉树形态



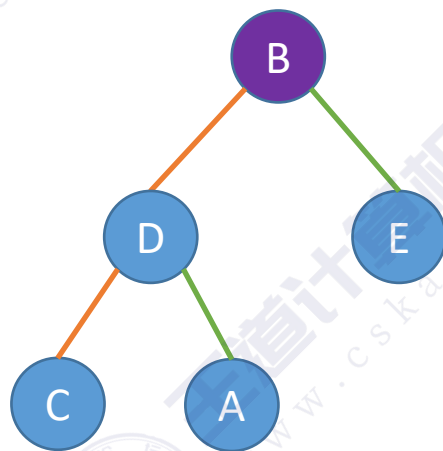
中序遍历序列：BDCAE

不同二叉树的前序遍历序列

前序遍历：根结点、前序遍历左子树、前序遍历右子树

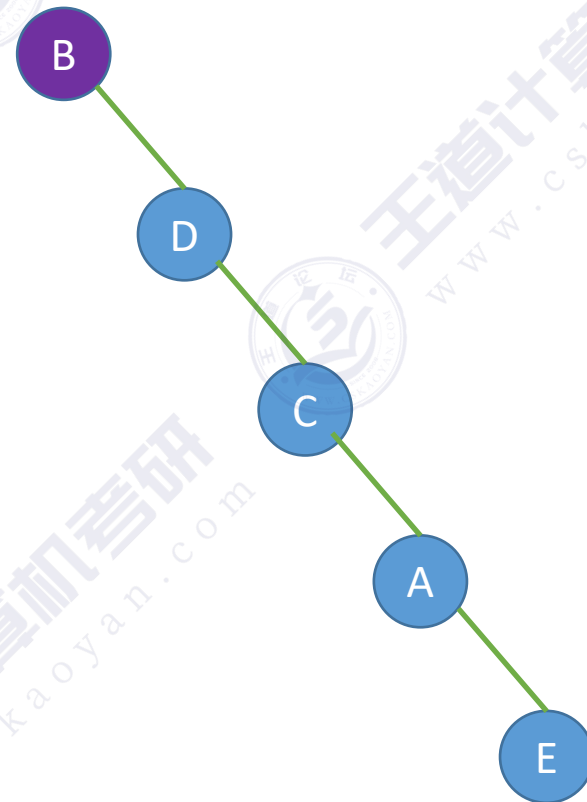


前序遍历序列：BDCAE



前序遍历序列：BDCAE

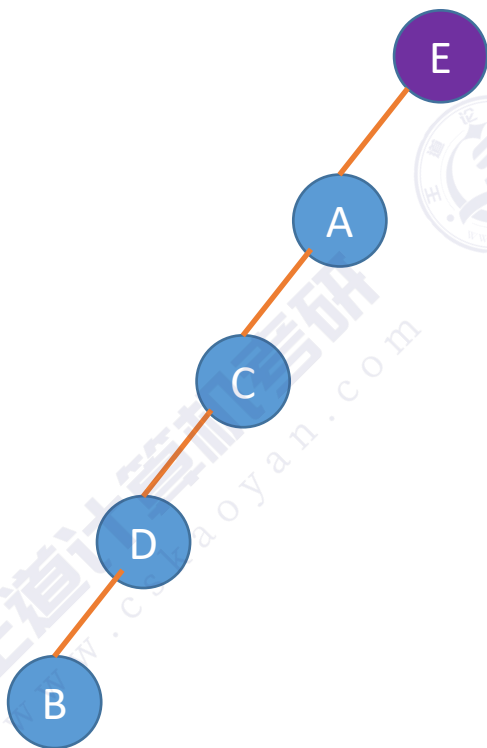
结论：一个前序遍历序列
可能对应多种二叉树形态



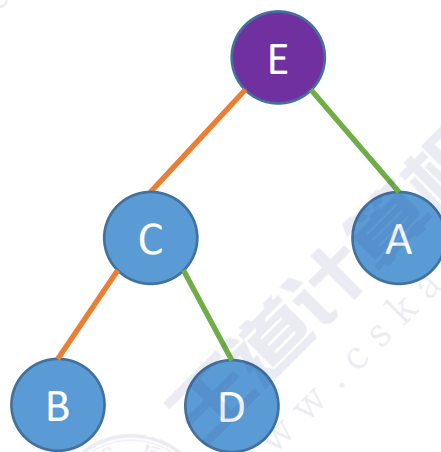
前序遍历序列：BDCAE

不同二叉树的后序遍历序列

后序遍历：前序遍历左子树、前序遍历右子树、根结点

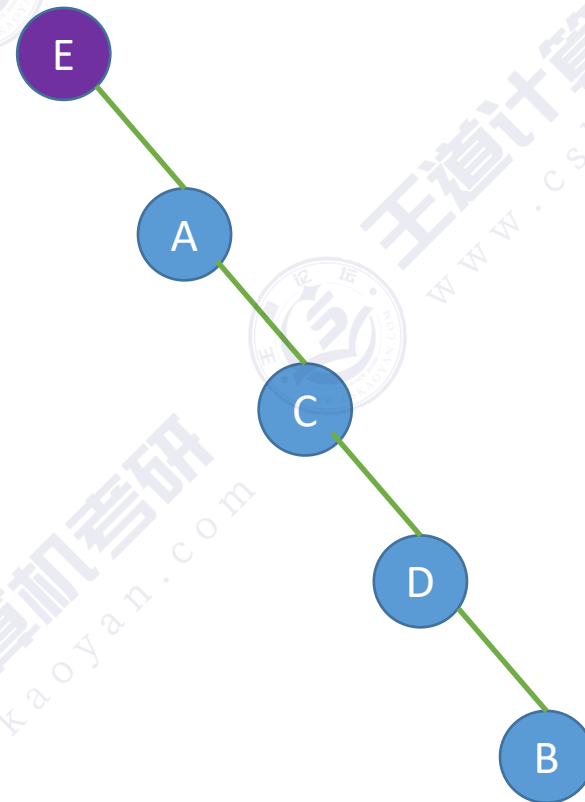


后序遍历序列：BDCAE



后序遍历序列：BDCAE

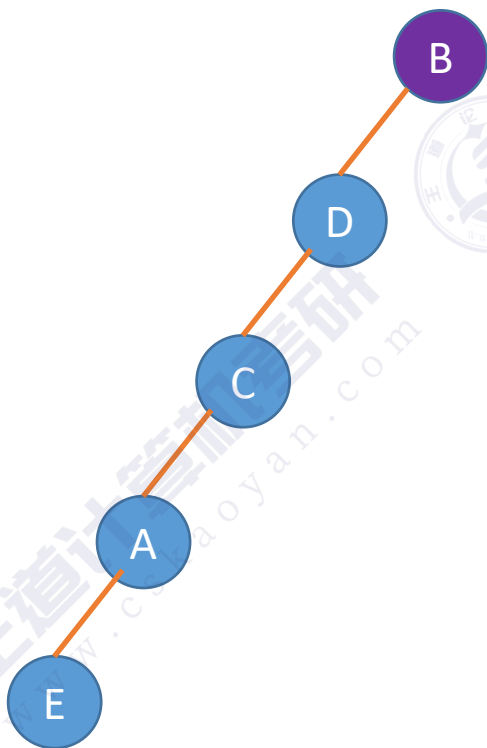
结论：一个后序遍历序列
可能对应多种二叉树形态



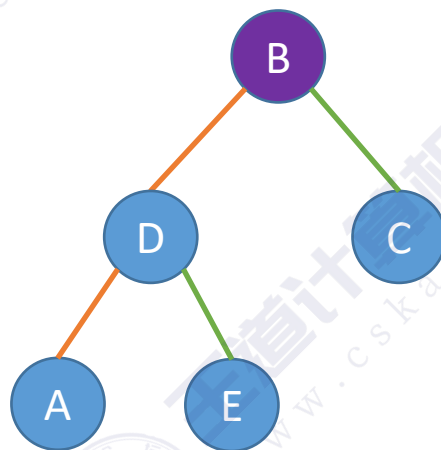
后序遍历序列：BDCAE

不同二叉树的层序遍历序列

层序遍历:

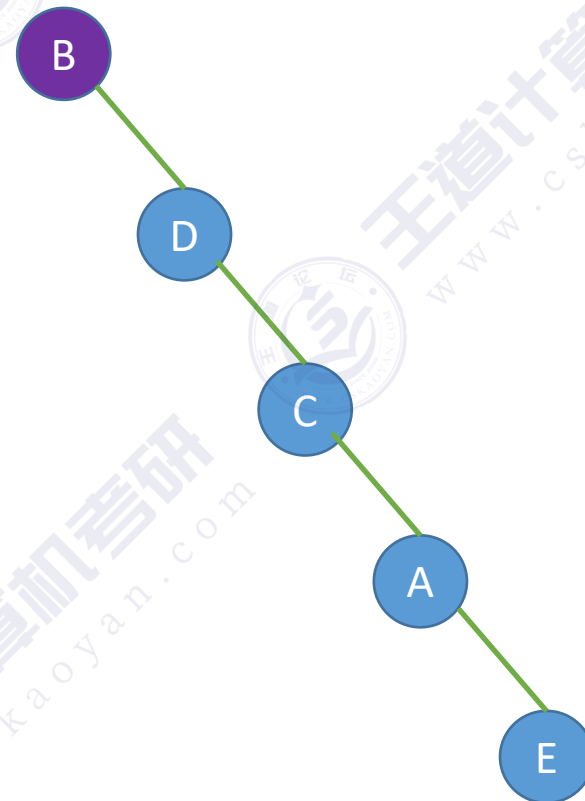


层序遍历序列: BDCAE



层序遍历序列: BDCAE

结论: 一个层序遍历序列
可能对应多种二叉树形态



层序遍历序列: BDCAE

由遍历序列构造二叉树

结论：若只给出一棵二叉树的前/中/后/层序遍历序列中的一种，不能唯一确定一棵二叉树

由二叉树的遍历序列构造二叉树

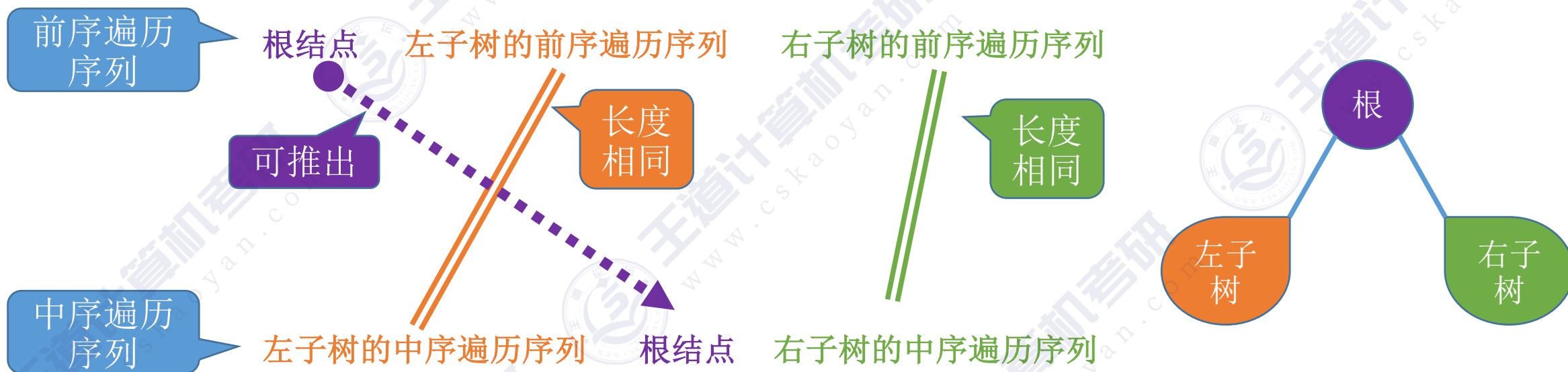
前序+中序遍历序列

后序+中序遍历序列

层序+中序遍历序列

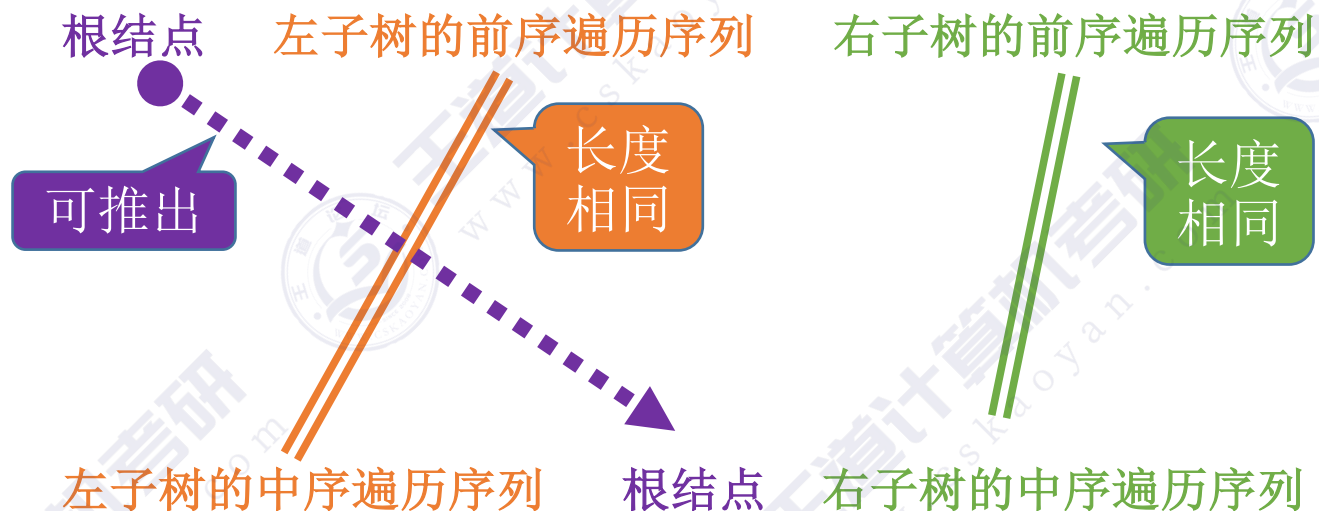
前序 + 中序遍历序列

前序遍历：根结点、前序遍历左子树、前序遍历右子树



中序遍历：中序遍历左子树、根结点、中序遍历右子树

例1: 前序 + 中序遍历序列

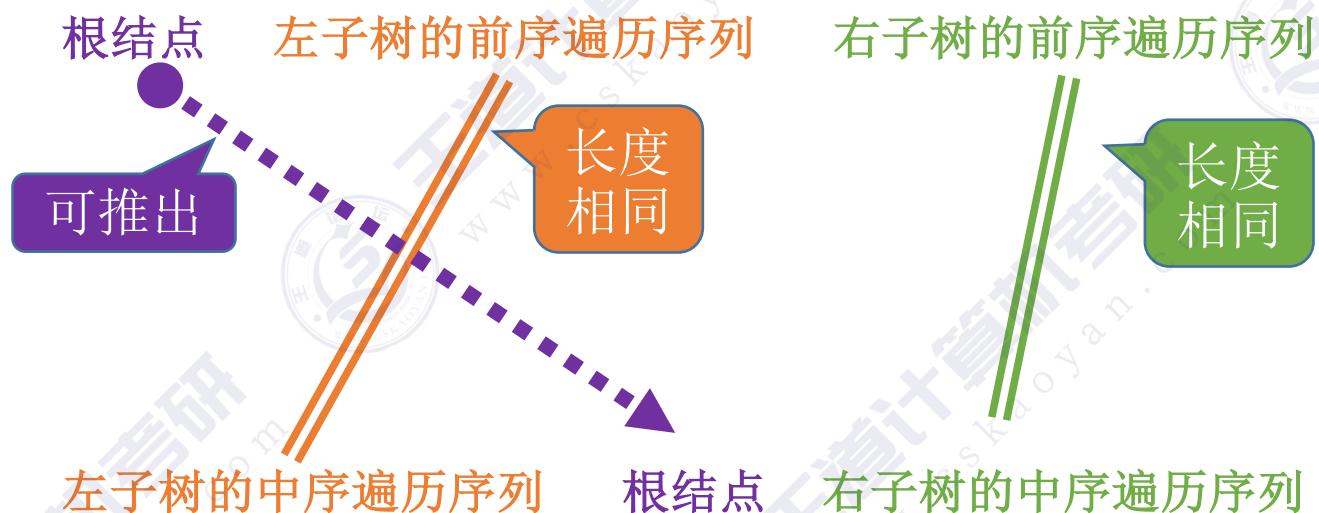


前序遍历序列: A D B C E

中序遍历序列: B D C A E

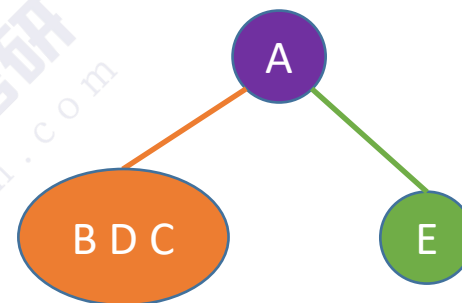
B D C A E

例1: 前序 + 中序遍历序列

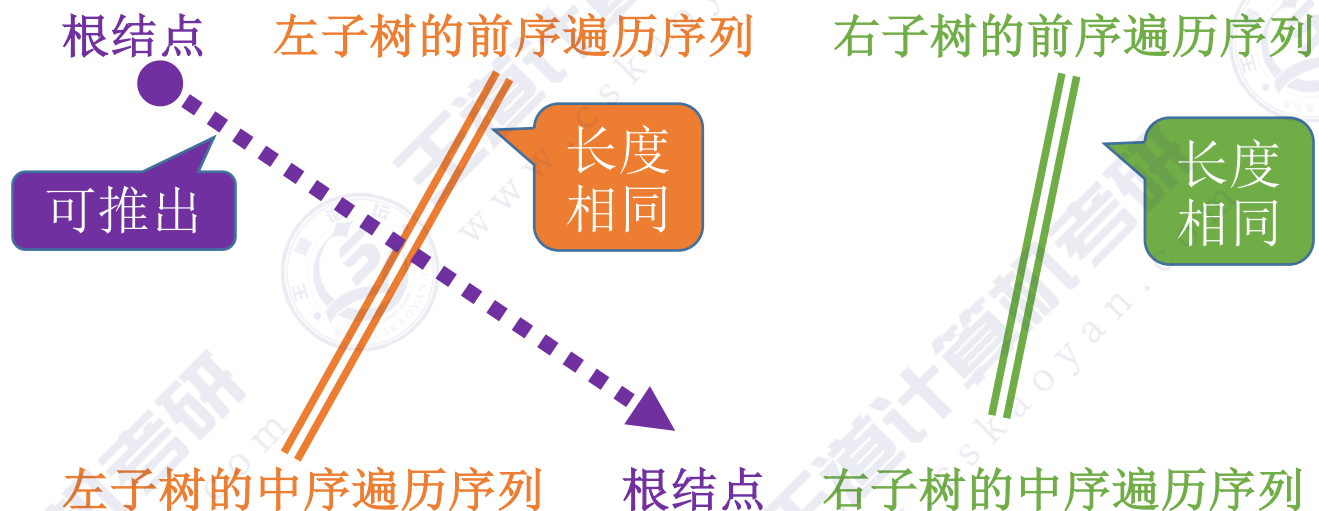


前序遍历序列: A D B C E

中序遍历序列: B D C A E

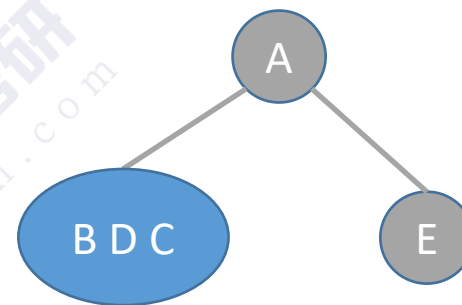


例1: 前序 + 中序遍历序列

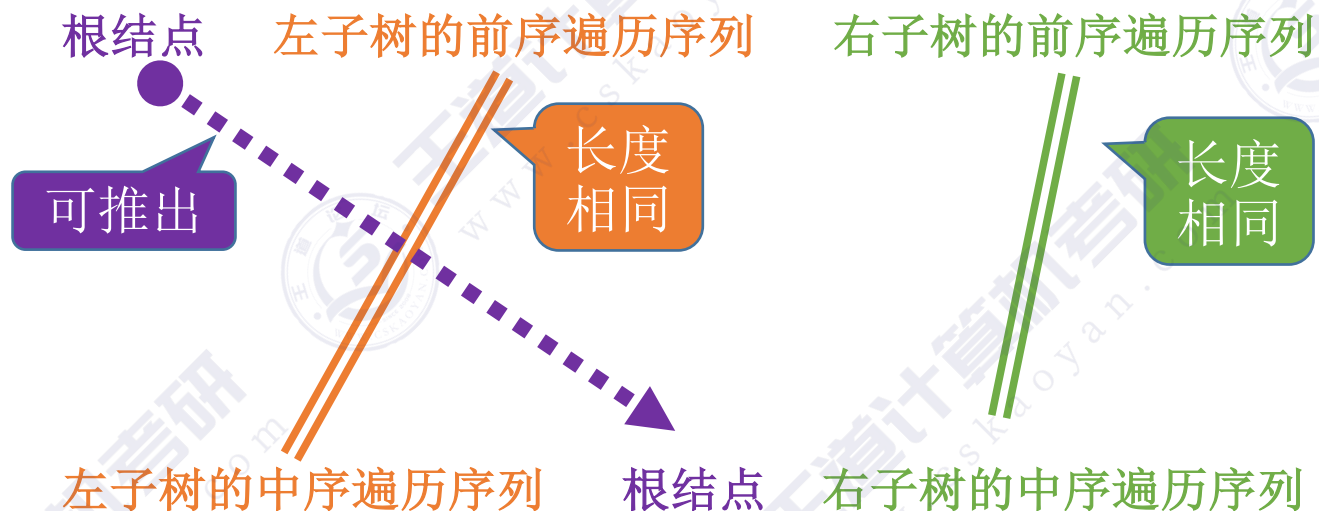


前序遍历序列: A D B C E

中序遍历序列: B D C A E

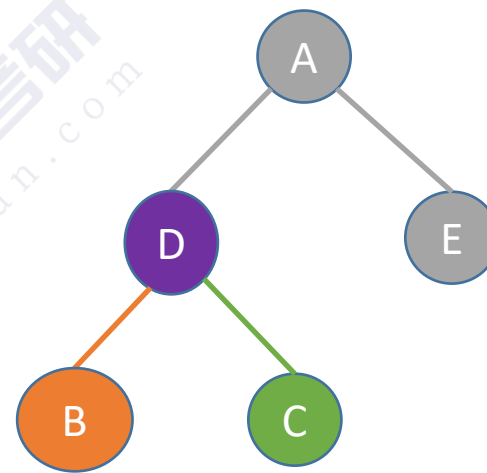


例1: 前序 + 中序遍历序列

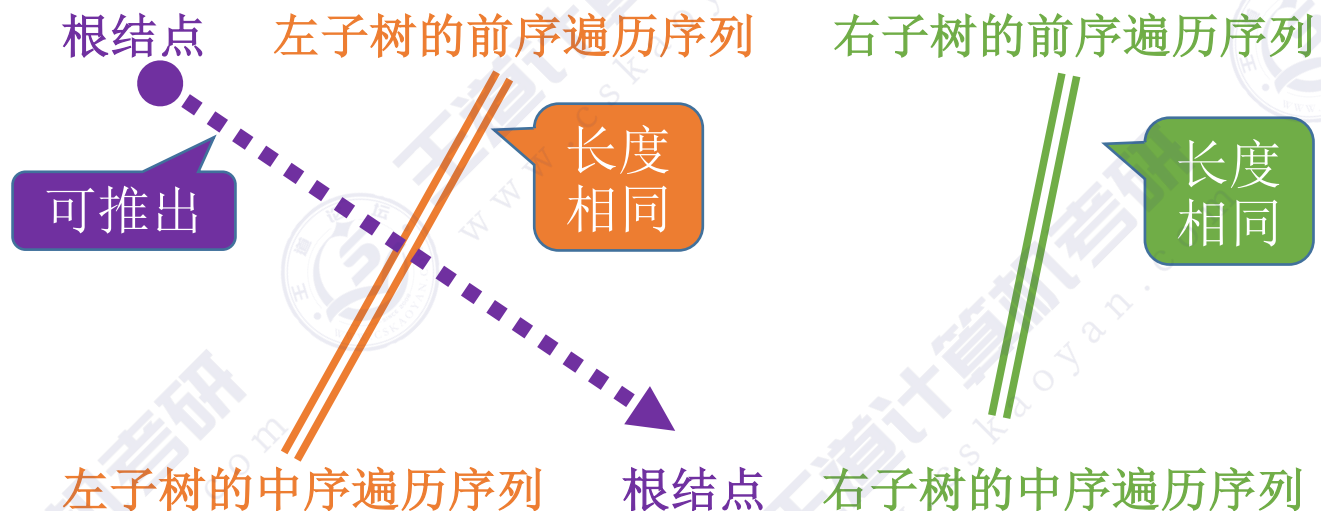


前序遍历序列: A D B C E

中序遍历序列: B D C A E



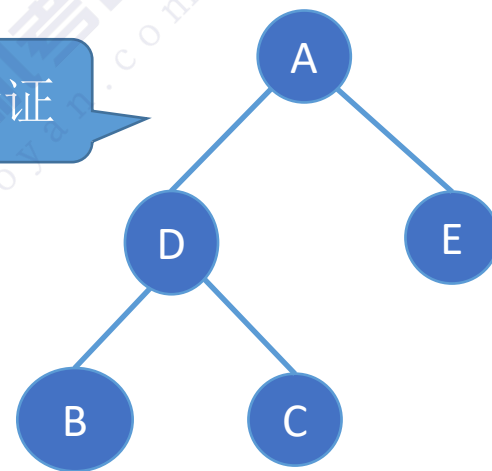
例1: 前序 + 中序遍历序列



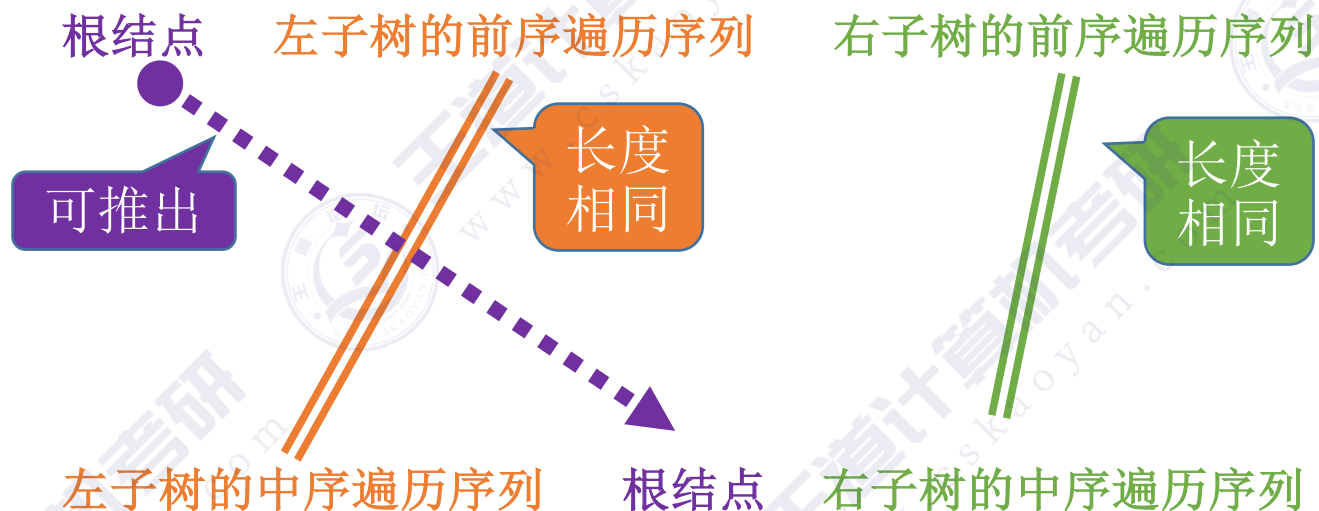
前序遍历序列: A D B C E

中序遍历序列: B D C A E

注意验证



例2：前序 + 中序遍历序列

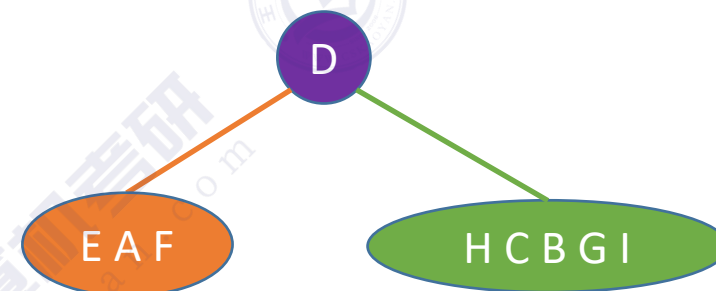
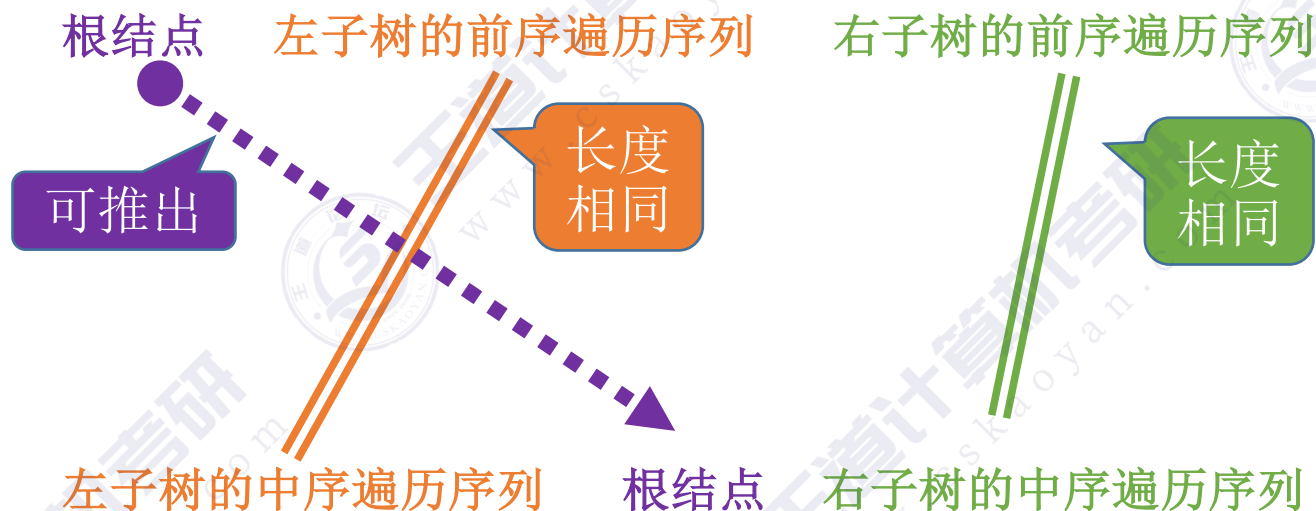


EAFDHCBI

前序遍历序列: DAEFBCHGI

中序遍历序列: EAFDHCBI

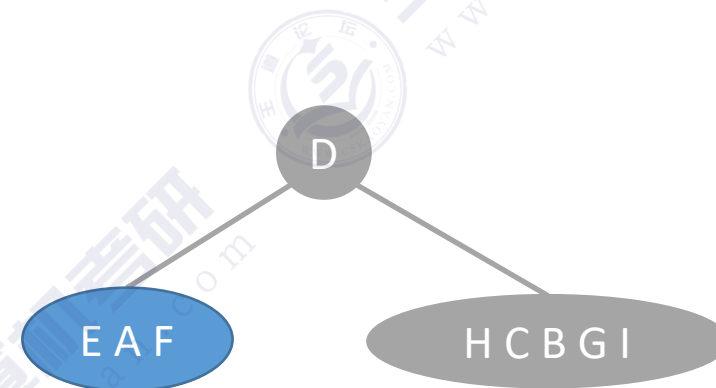
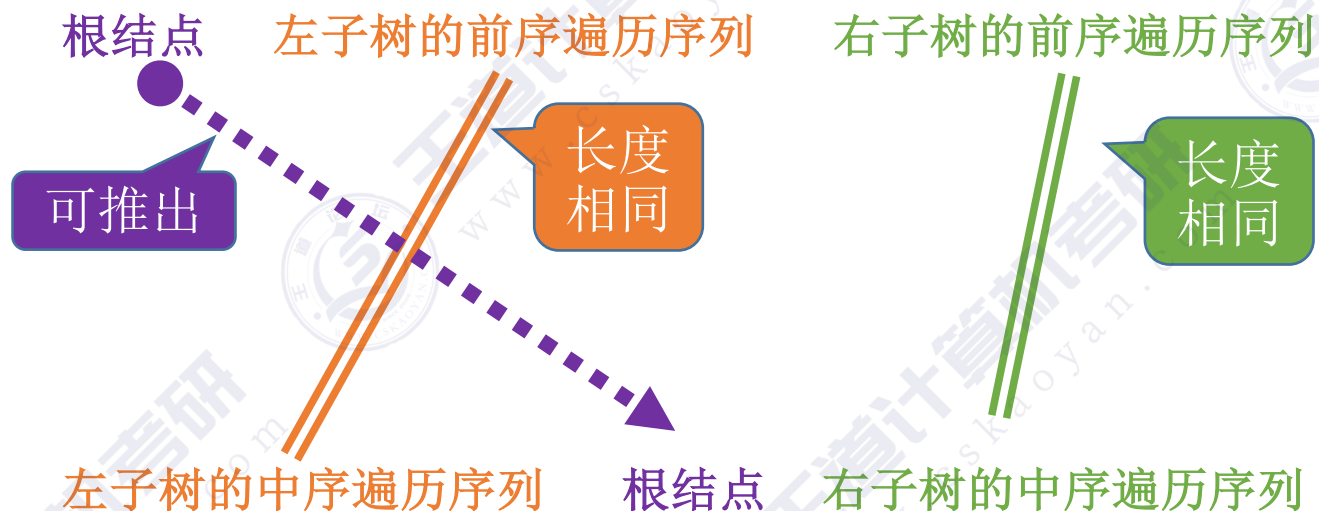
例2：前序 + 中序遍历序列



前序遍历序列: **D A E F B C H G I**

中序遍历序列: **E A F D H C B G I**

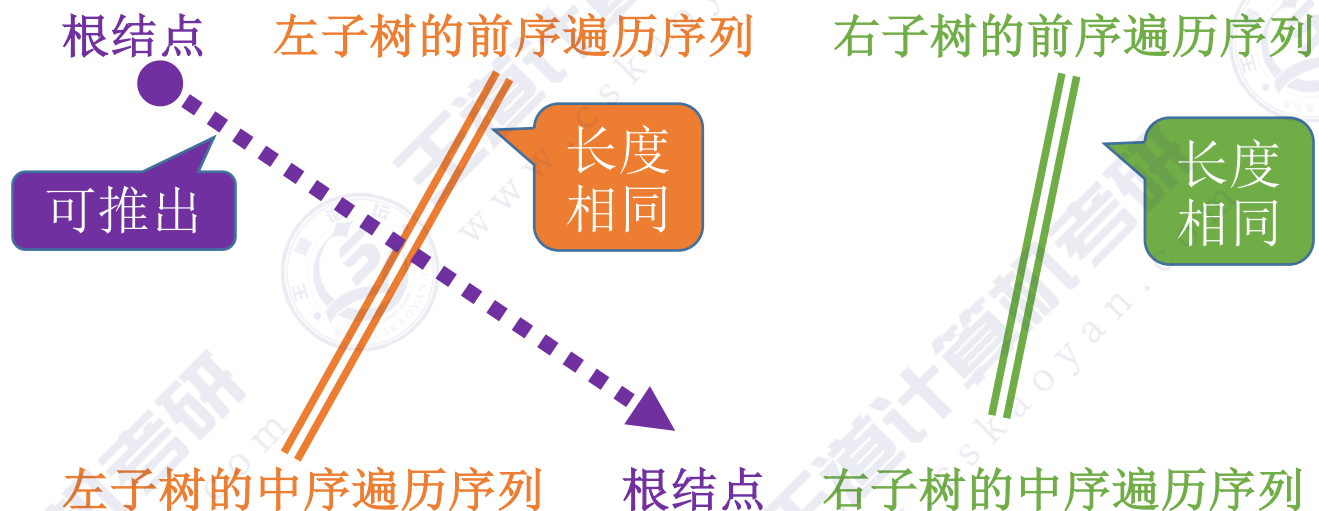
例2: 前序 + 中序遍历序列



前序遍历序列: D A E F B C H G I

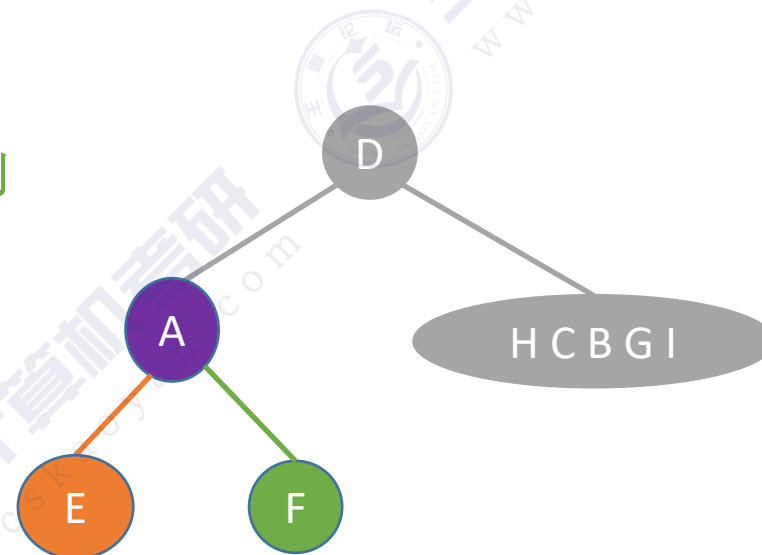
中序遍历序列: E A F D H C B G I

例2: 前序 + 中序遍历序列

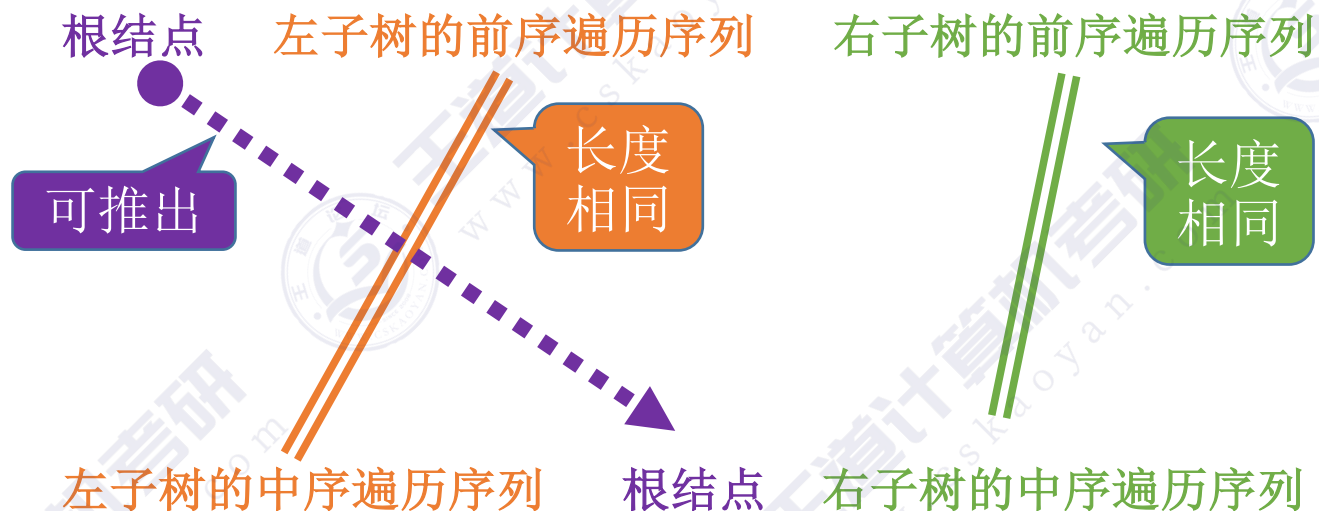


前序遍历序列: D A E F B C H G I

中序遍历序列: E A F D H C B G I

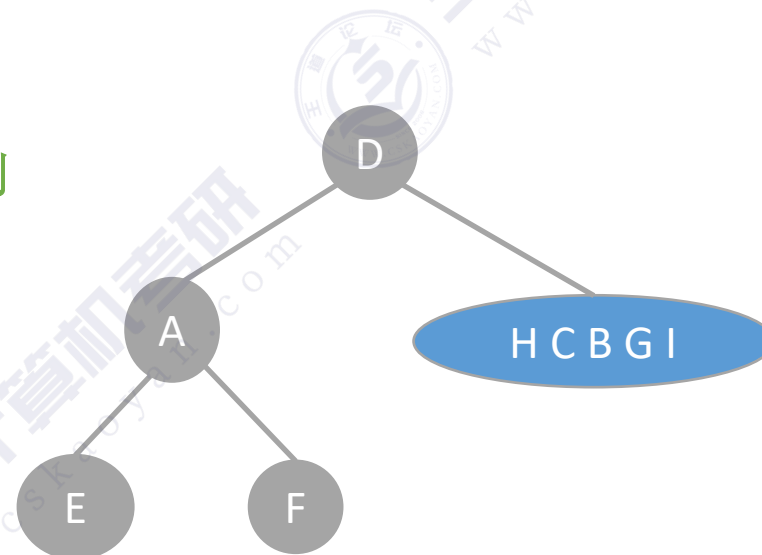


例2：前序 + 中序遍历序列

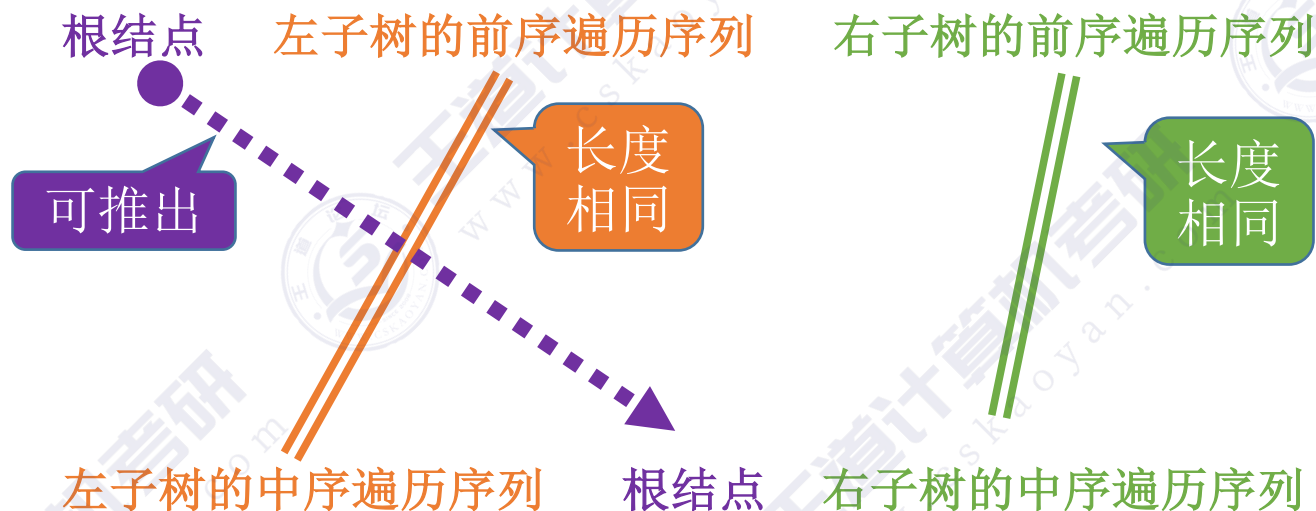


前序遍历序列：D A E F B C H G I

中序遍历序列：E A F D H C B G I

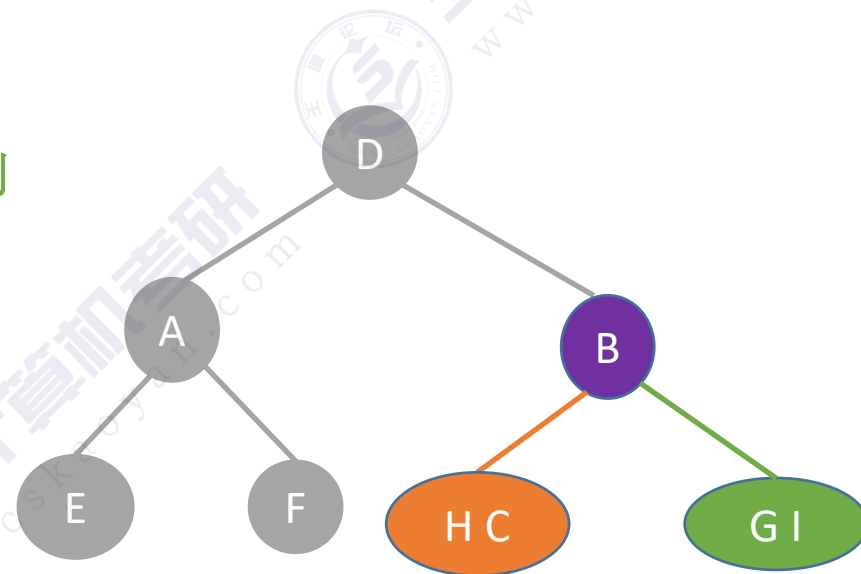


例2：前序 + 中序遍历序列

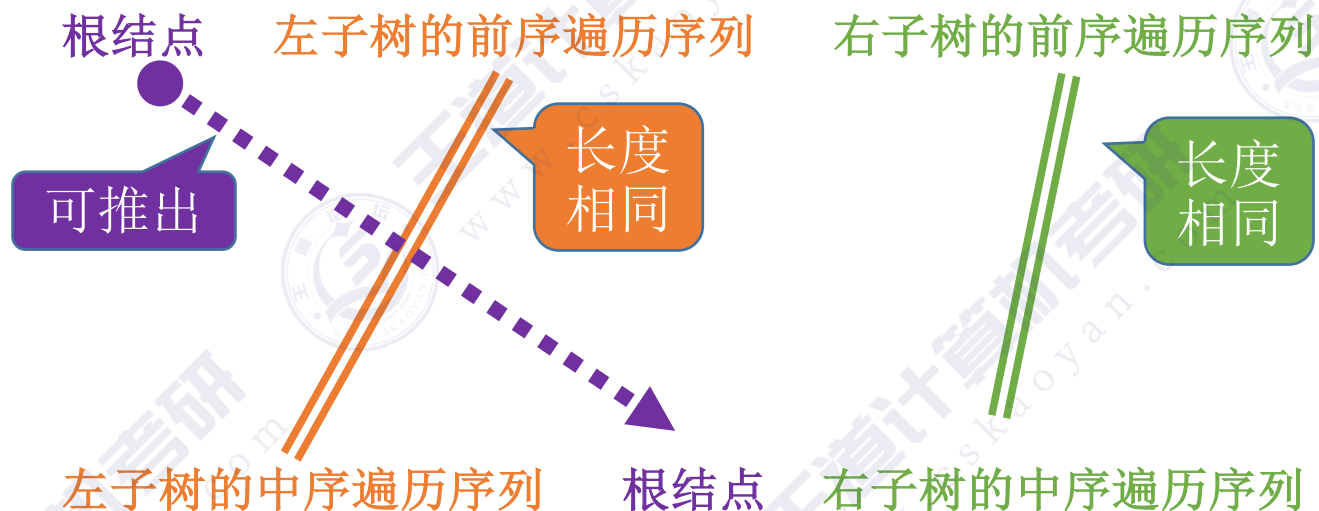


前序遍历序列: D A E F B C H G I

中序遍历序列: E A F D H C B G I

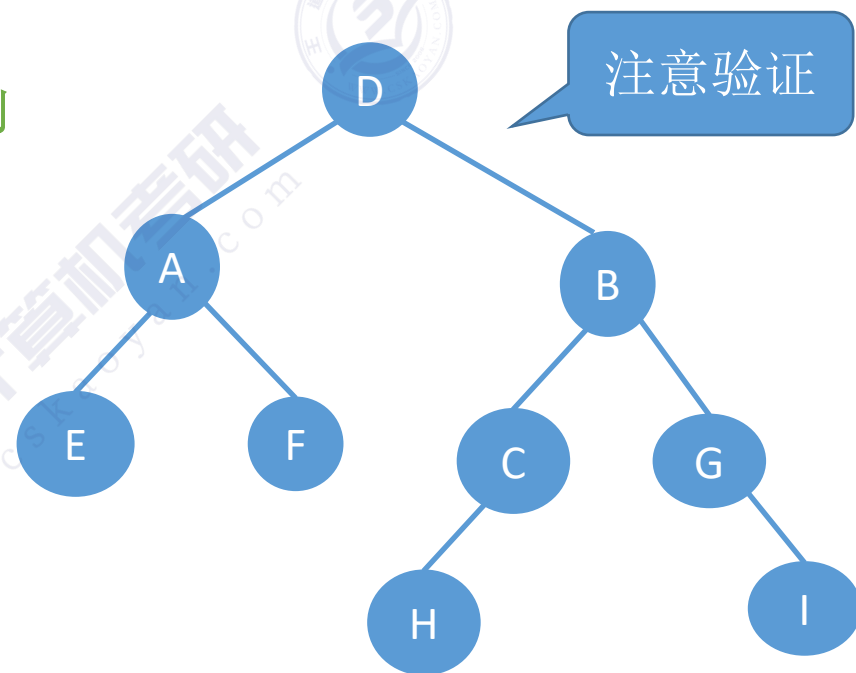


例2：前序 + 中序遍历序列



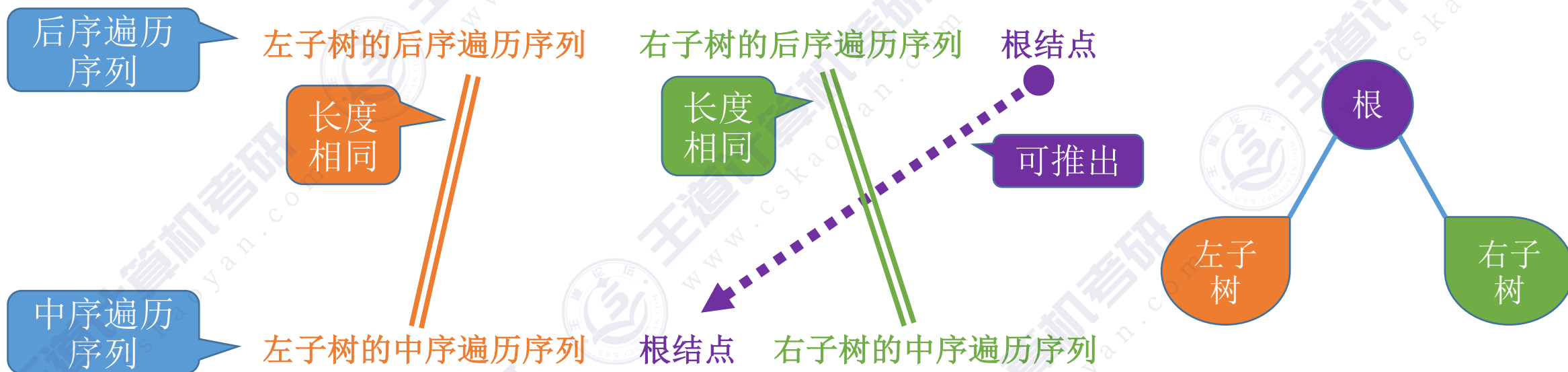
前序遍历序列: D A E F B C H G I

中序遍历序列: E A F D H C B G I



后序 + 中序遍历序列

后序遍历：前序遍历左子树、前序遍历右子树、根结点



中序遍历：中序遍历左子树、根结点、中序遍历右子树

例3：后序 + 中序遍历序列

左子树的后序遍历序列

右子树的后序遍历序列

根结点

长度
相同

长度
相同

可推出

左子树的中序遍历序列

根结点

右子树的中序遍历序列

EAFDHCBI

后序遍历序列：EFAHCIGBD

中序遍历序列：EAFDHCBI

例3：后序 + 中序遍历序列

左子树的后序遍历序列

右子树的后序遍历序列

根结点

长度
相同

长度
相同

可推出

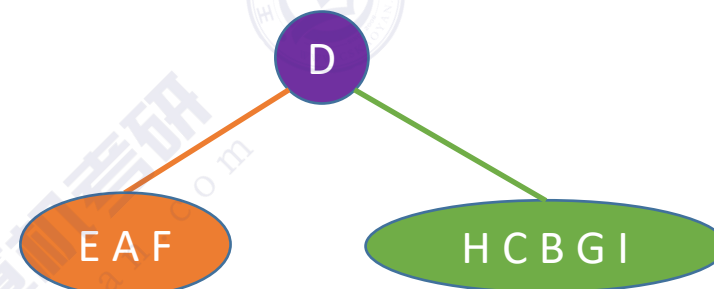
左子树的中序遍历序列

根结点

右子树的中序遍历序列

后序遍历序列：E F A H C I G B D

中序遍历序列：E A F D H C B G I



例3：后序 + 中序遍历序列

左子树的后序遍历序列

右子树的后序遍历序列

根结点

长度
相同

长度
相同

可推出

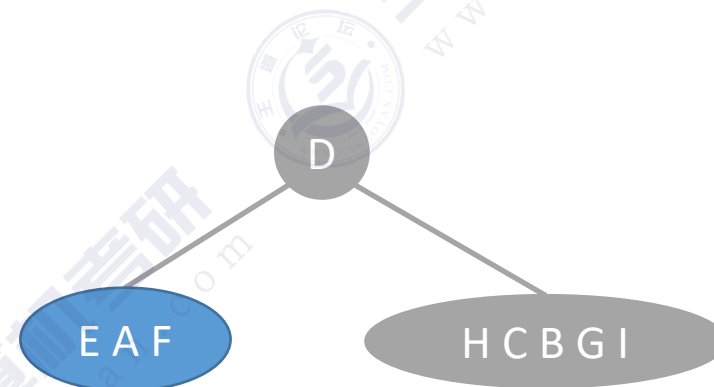
左子树的中序遍历序列

根结点

右子树的中序遍历序列

后序遍历序列：E F A H C I G B D

中序遍历序列：E A F D H C B G I



例3：后序 + 中序遍历序列

左子树的后序遍历序列

右子树的后序遍历序列

根结点

长度
相同

长度
相同

可推出

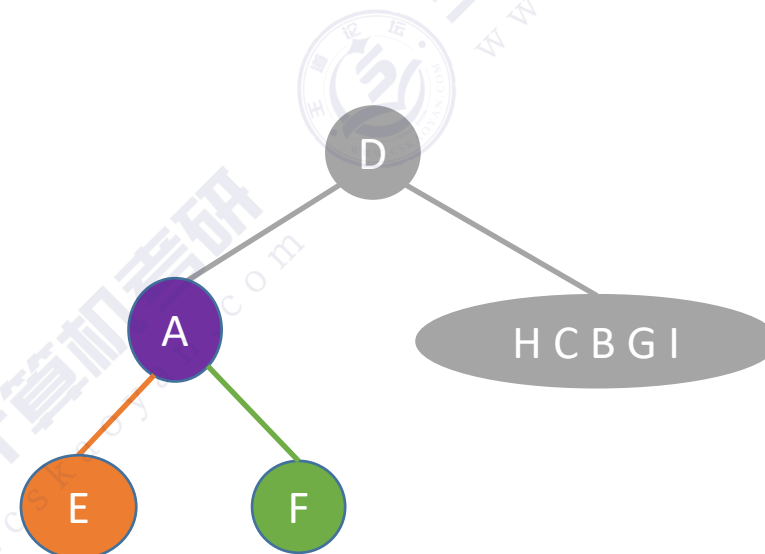
左子树的中序遍历序列

根结点

右子树的中序遍历序列

后序遍历序列：E F A H C I G B D

中序遍历序列：E A F D H C B G I



例3：后序 + 中序遍历序列

左子树的后序遍历序列

右子树的后序遍历序列

根结点

长度
相同

长度
相同

可推出

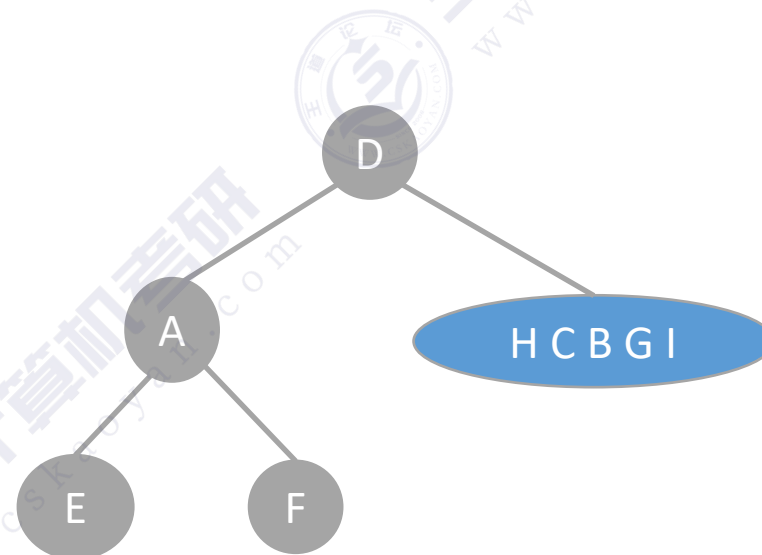
左子树的中序遍历序列

根结点

右子树的中序遍历序列

后序遍历序列：E F A H C I G B D

中序遍历序列：E A F D H C B G I



例3：后序 + 中序遍历序列

左子树的后序遍历序列

右子树的后序遍历序列

根结点

长度
相同

长度
相同

可推出

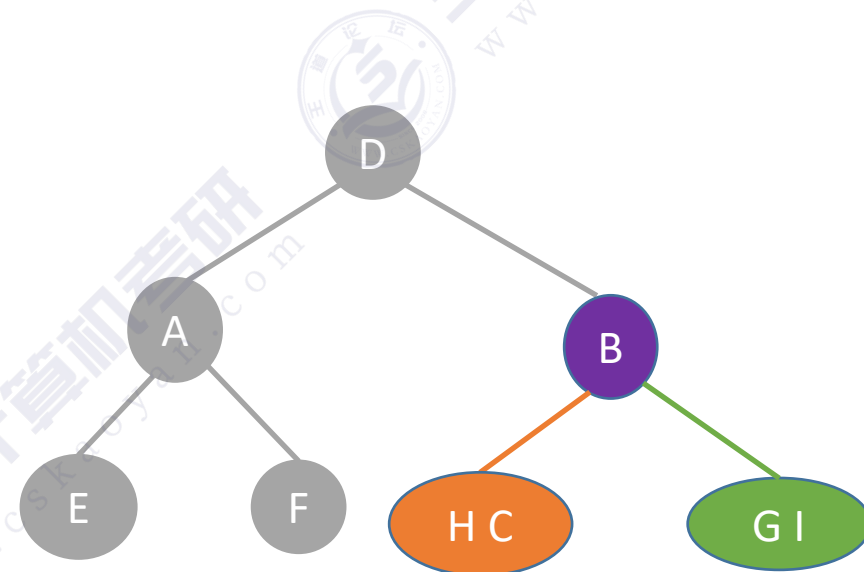
左子树的中序遍历序列

根结点

右子树的中序遍历序列

后序遍历序列：E F A H C I G B D

中序遍历序列：E A F D H C B G I



例3: 后序 + 中序遍历序列

左子树的后序遍历序列

右子树的后序遍历序列

根结点

长度
相同

长度
相同

可推出

左子树的中序遍历序列

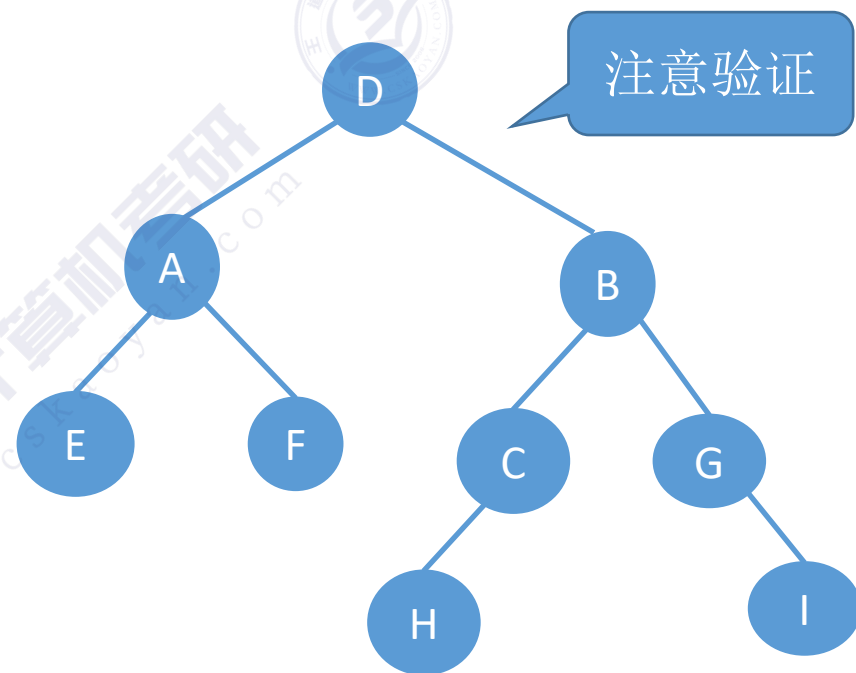
根结点

右子树的中序遍历序列

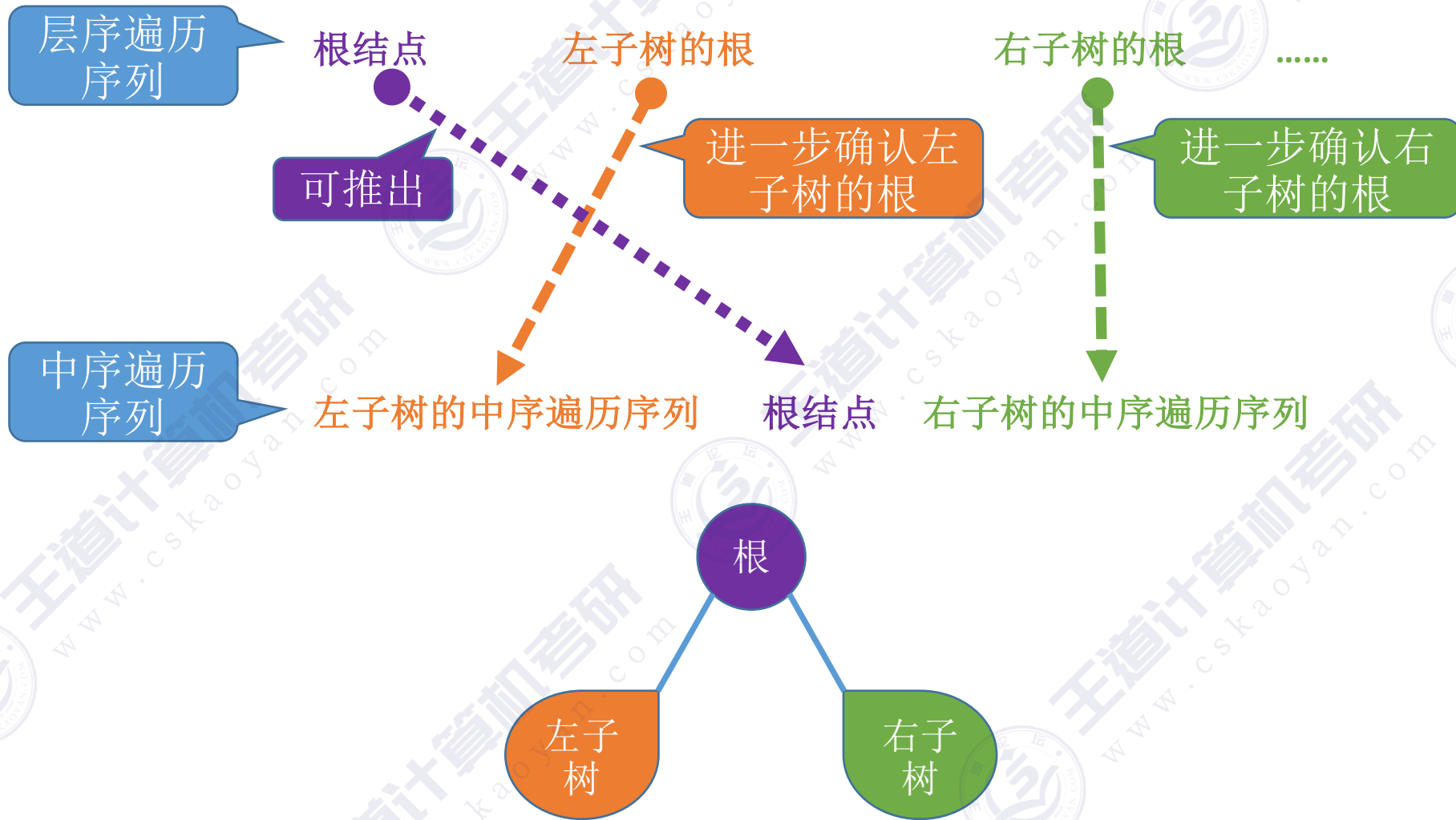
注意验证

后序遍历序列: E F A H C I G B D

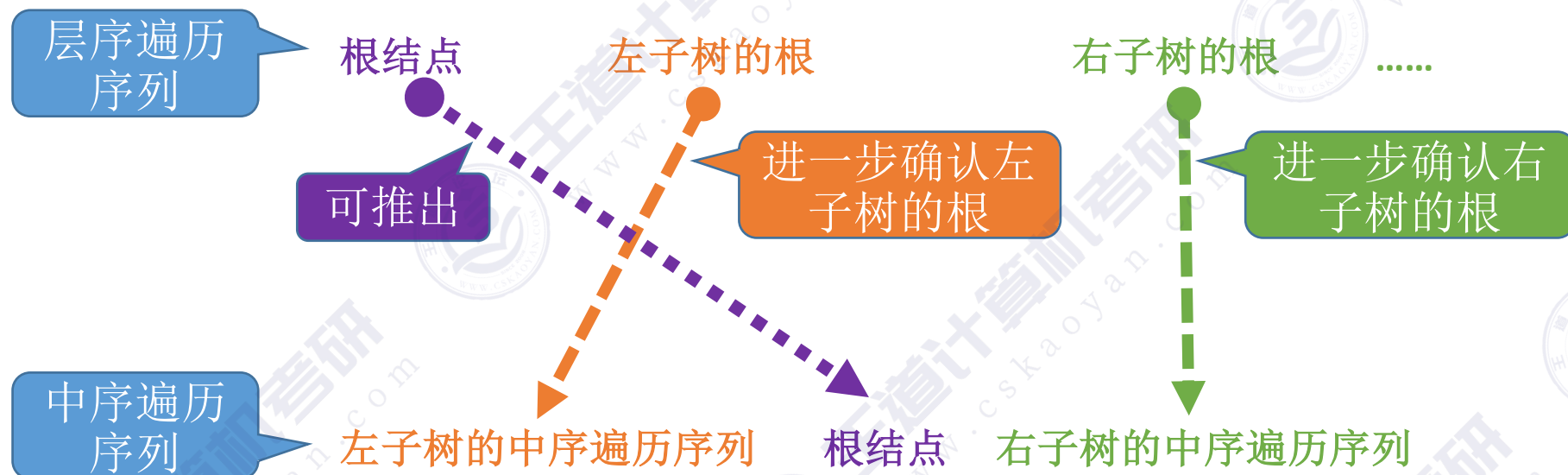
中序遍历序列: E A F D H C B G I



层序 + 中序遍历序列



例4：层序 + 中序遍历序列

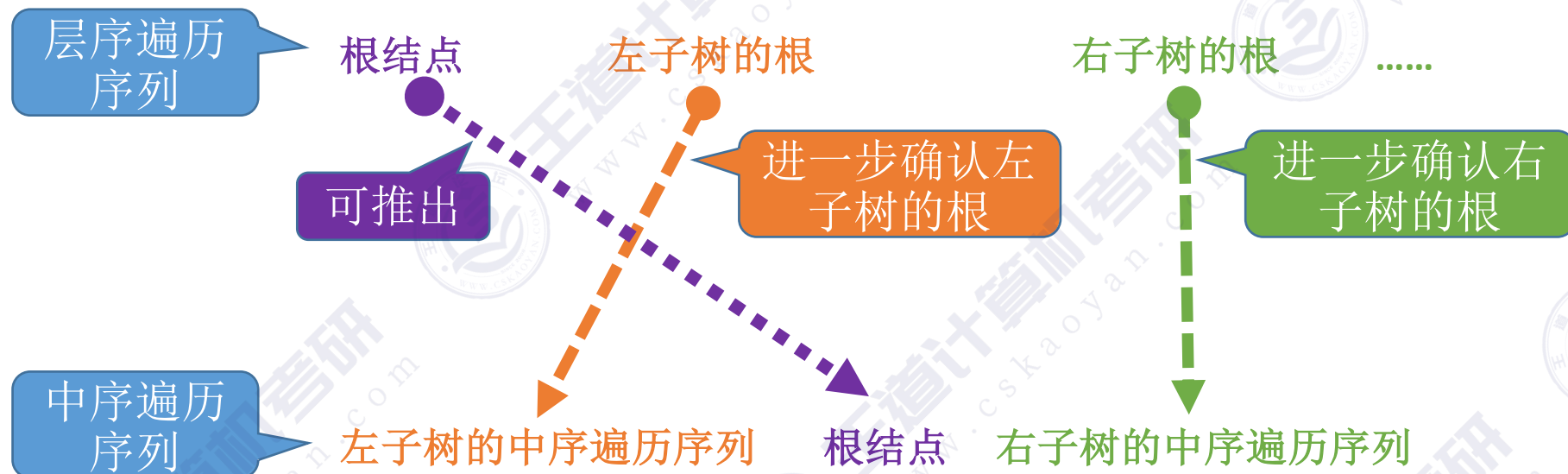


层序遍历序列: D A B E F C G H I

E A F D H C B G I

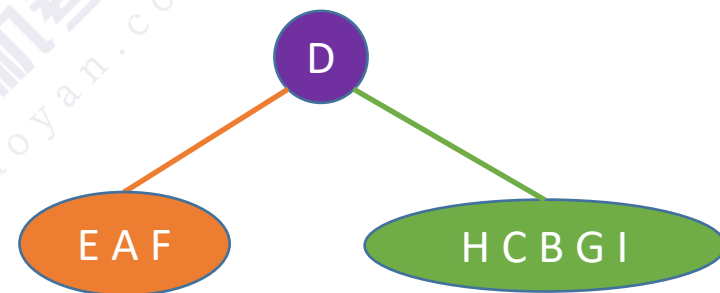
中序遍历序列: E A F D H C B G I

例4：层序 + 中序遍历序列

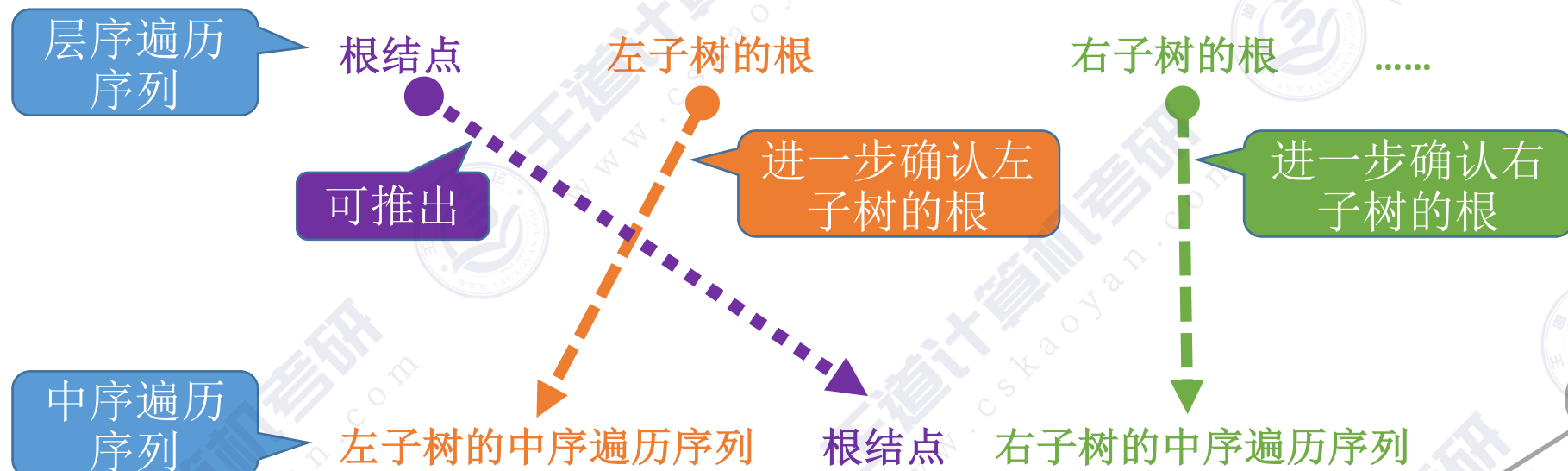


层序遍历序列: **D** A B E F C G H I

中序遍历序列: E A F **D** H C B G I

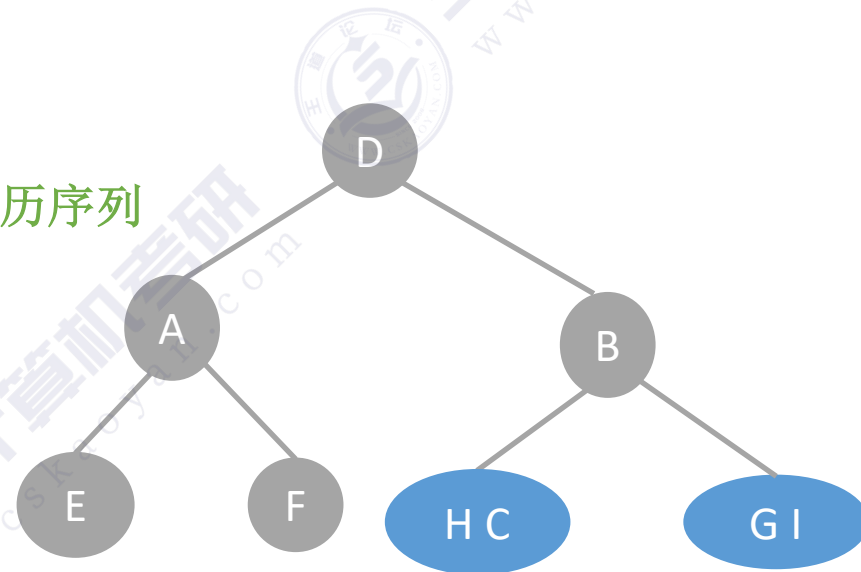


例4：层序 + 中序遍历序列

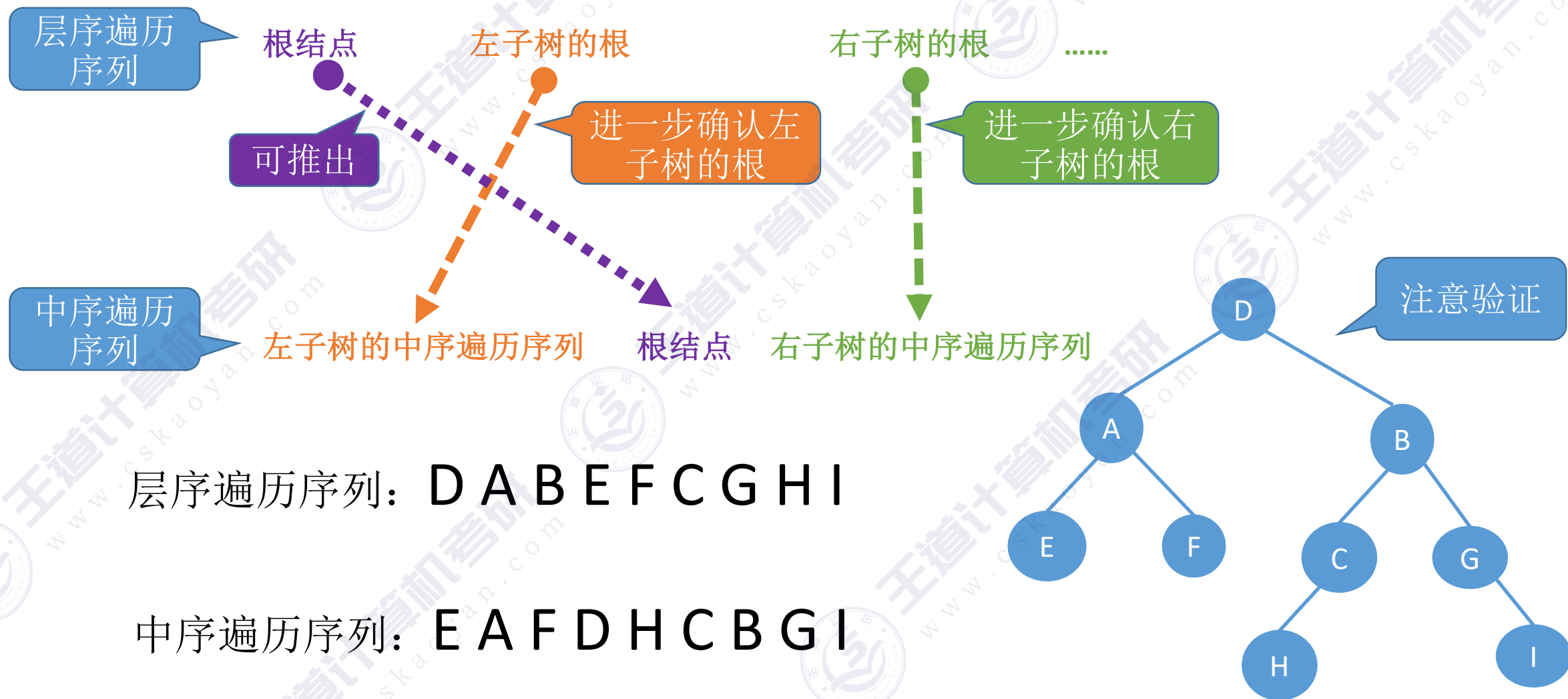


层序遍历序列: D A B E F C G H I

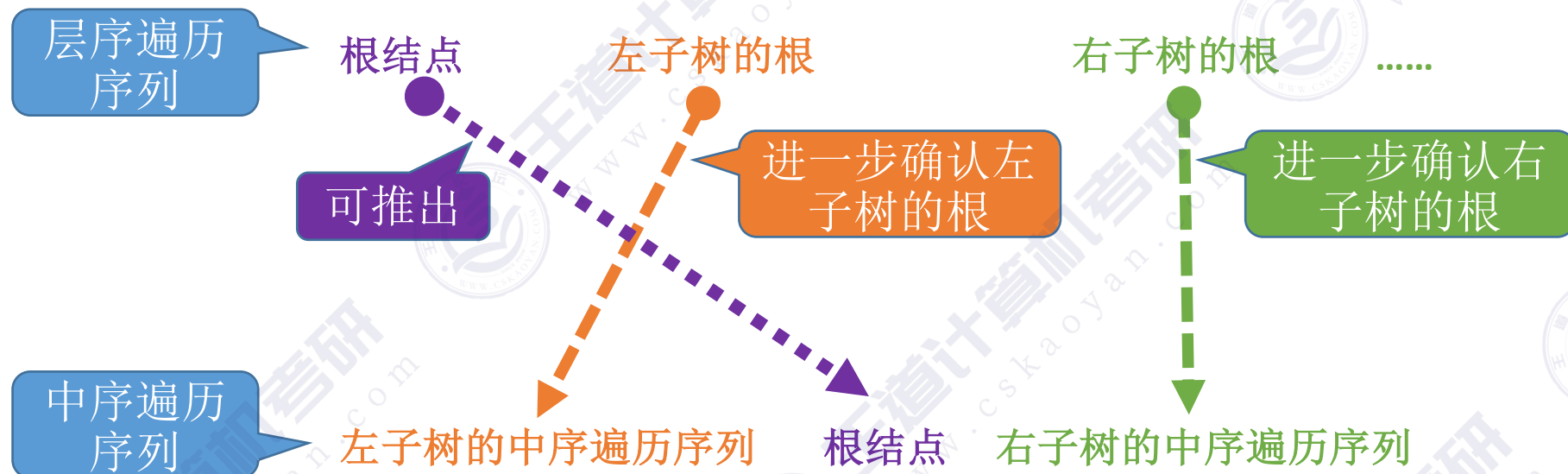
中序遍历序列: E A F D H C B G I



例4：层序 + 中序遍历序列



例5：层序 + 中序遍历序列

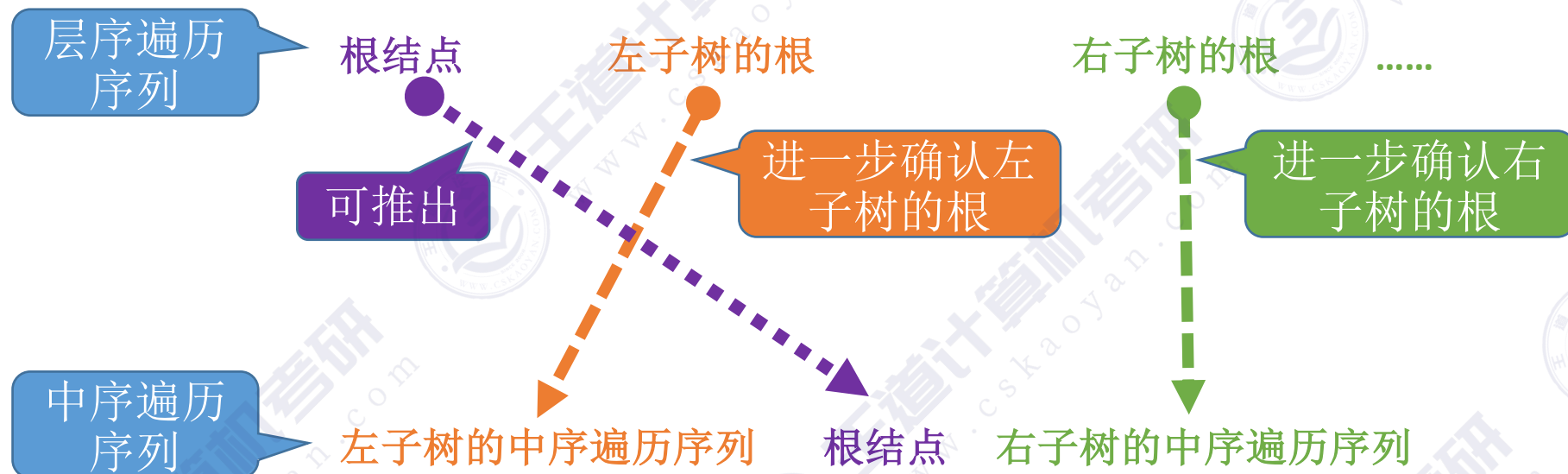


ACBED

层序遍历序列: A B C D E

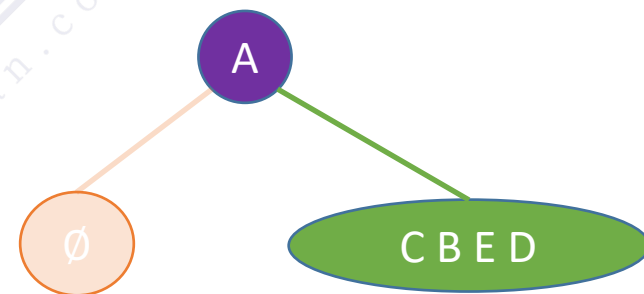
中序遍历序列: A C B E D

例5: 层序 + 中序遍历序列

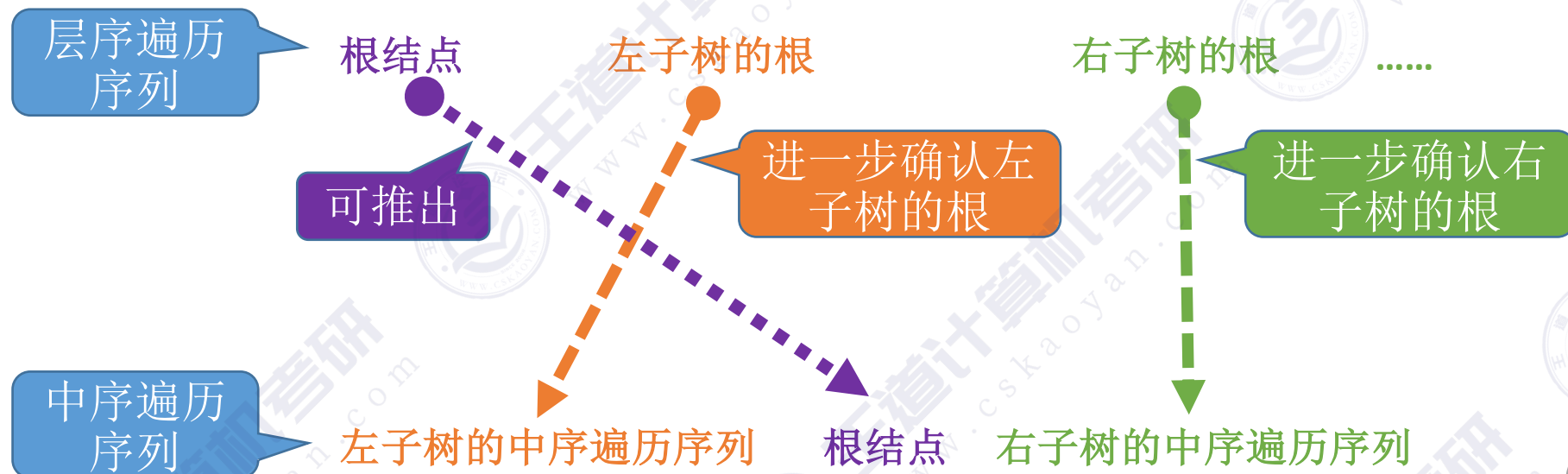


层序遍历序列: A B C D E

中序遍历序列: A C B E D

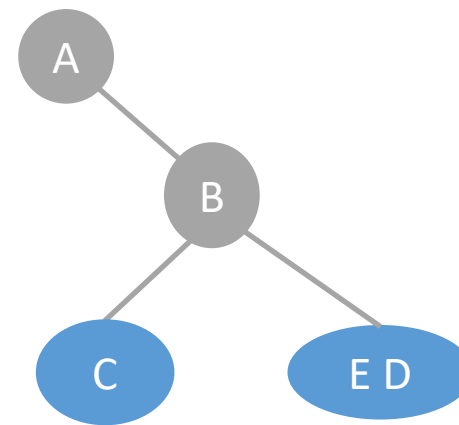


例5: 层序 + 中序遍历序列

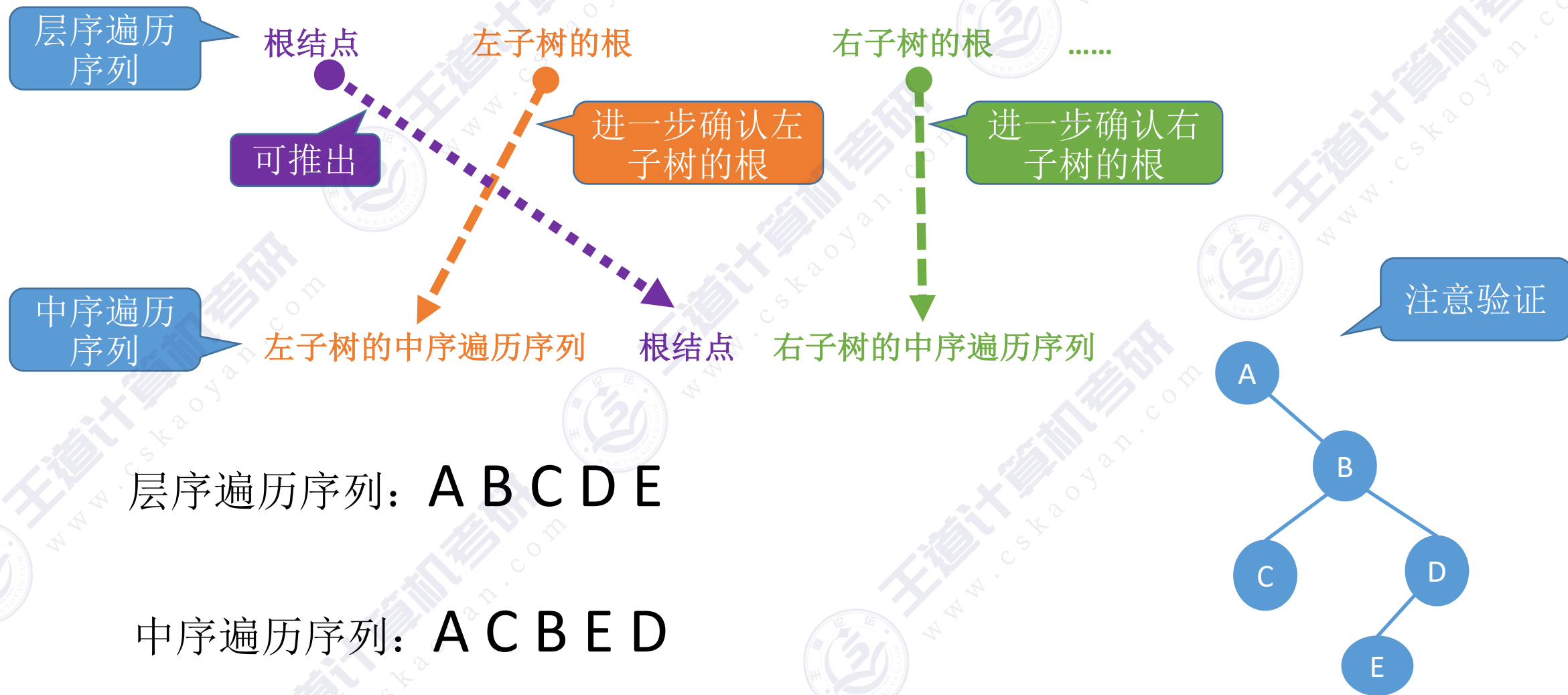


层序遍历序列: A B C D E

中序遍历序列: A C B E D



例5: 层序 + 中序遍历序列



知识回顾与重要考点

由二叉树的遍历序列构造二叉树

前序+中序遍历序列

后序+中序遍历序列

层序+中序遍历序列

前序遍历
序列

根结点 左子树的前序遍历序列 右子树的前序遍历序列

后序遍历
序列

左子树的后序遍历序列 右子树的后序遍历序列 根结点

中序遍历
序列

左子树的中序遍历序列 根结点 右子树的中序遍历序列

层序遍历
序列

根结点 左子树的根 右子树的根

Key: 找到树的根节点，并根据中序序列划分左右子树，再找到左右子树根节点

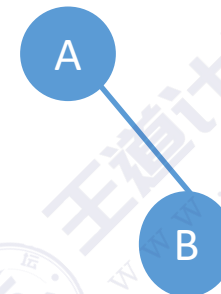
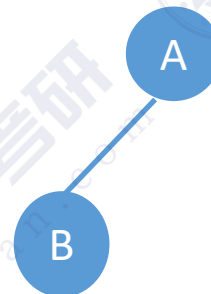
若前序、后序、层序序列两两组合？



前序遍历序列：A B

后序遍历序列：B A

层序遍历序列：A B



结论：前序、后序、层序序列的两两组合无法唯一确定一棵二叉树



抱歉 不能

本节内容

线索二叉树

概念

知识总览

线索二叉树

线索二叉树的作用

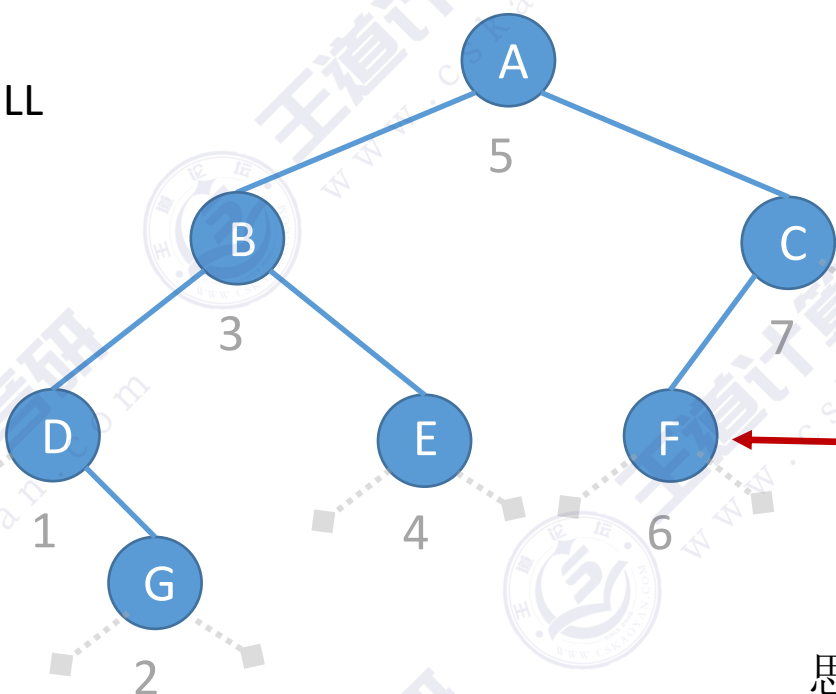
线索二叉树的存储结构

三种线索二叉树

二叉树的中序遍历序列

pre
→ NULL

pre
q →



中序遍历序列: D G B E A F C

能否从一个指定结点开始中序遍历?

//中序遍历

```
void InOrder(BiTree T){
```

```
    if(T!=NULL){
```

```
        InOrder(T->lchild);
```

//递归遍历左子树

```
        visit(T);
```

//访问根结点

```
        InOrder(T->rchild);
```

//递归遍历右子树

```
    }
```

```
}
```

①如何找到指定结点p在中序遍历序列中的前驱?

②如何找到p的中序后继?

思路:

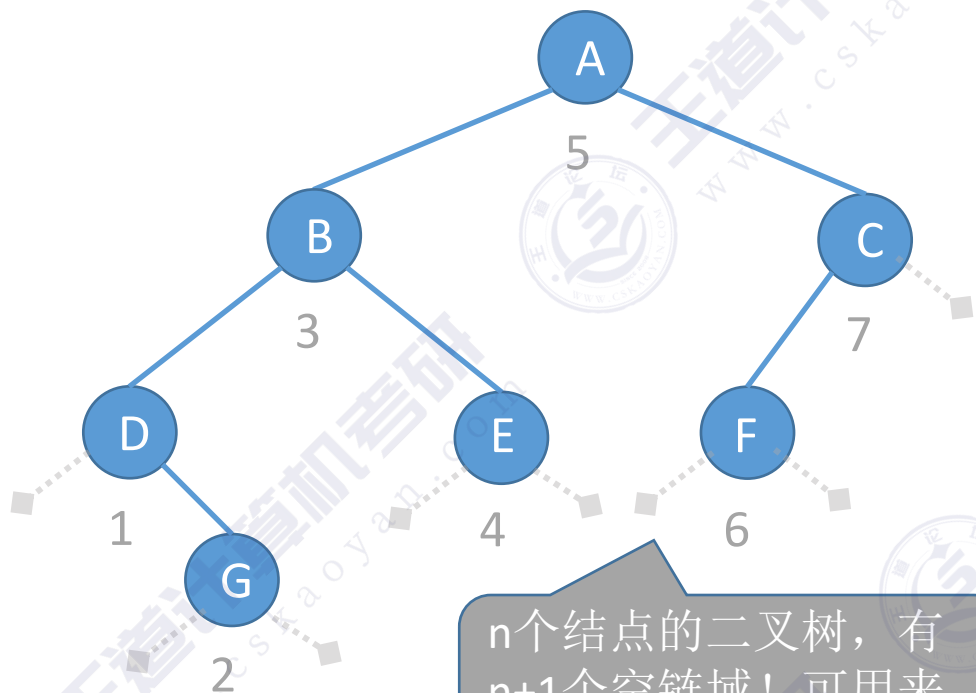
从根节点出发, 重新进行一次中序遍历, 指针q记录当前访问的结点, 指针pre记录上一个被访问的结点

①当q==p时, pre为前驱

②当pre==p时, q为后继

缺点: 找前驱、后继很不方便; 遍历操作必须从根开始

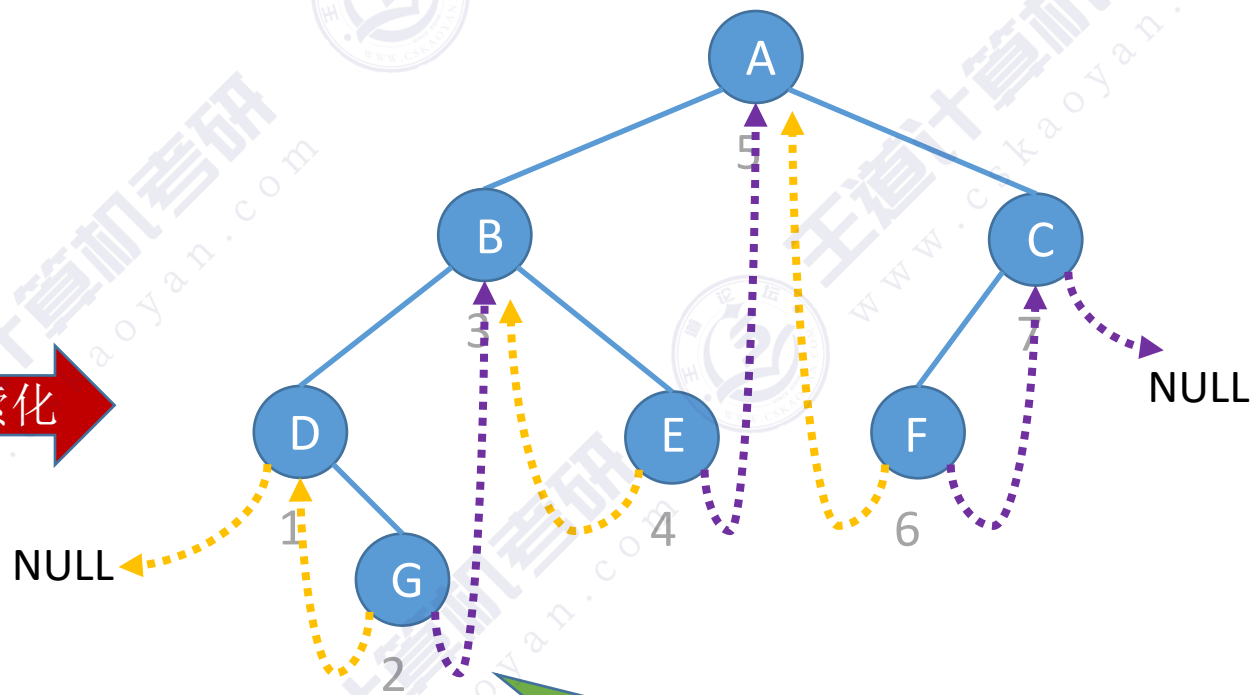
中序线索二叉树



n个结点的二叉树，有n+1个空链域！可用来记录前驱、后继的信息

中序遍历序列：D G B E A F C

线索化



问题：如何找到G的后继？

图示说明

前驱线索（由左孩子指针充当）：

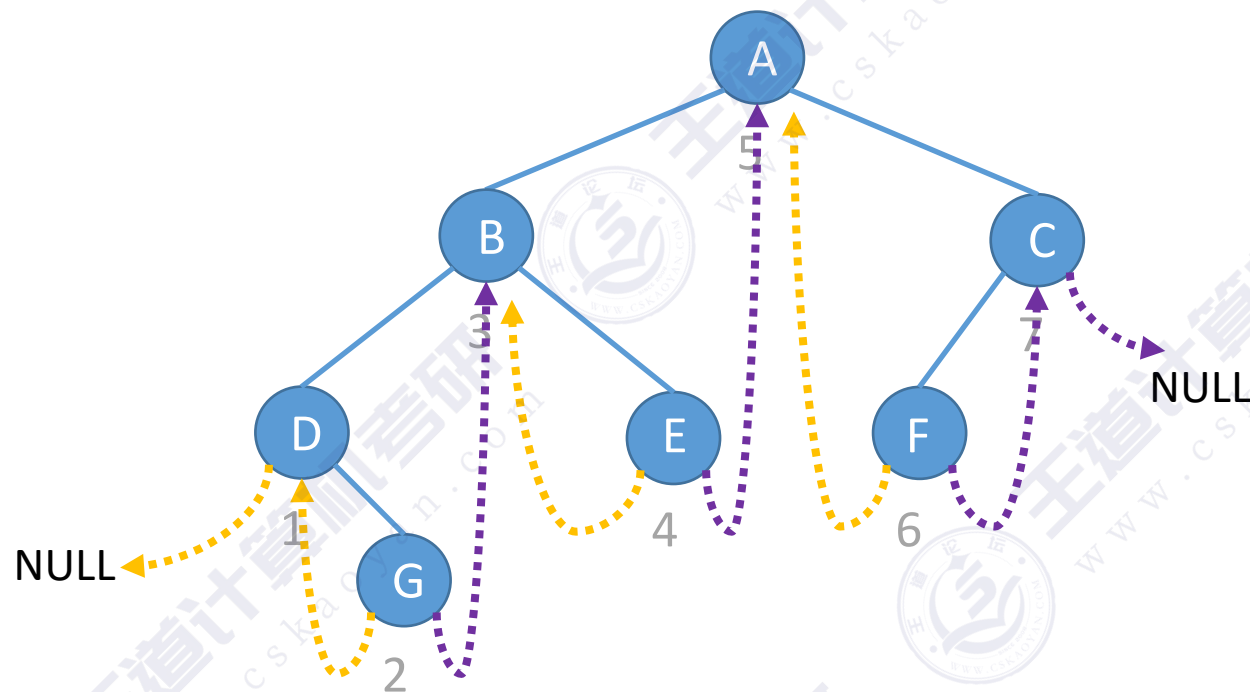


后继线索（由右孩子指针充当）：



指向前驱、后继的指针称为“线索”

线索二叉树的存储结构



图示说明

前驱线索 (由左孩子指针充当):



后继线索 (由右孩子指针充当):



//二叉树的结点 (链式存储)

```
typedef struct BiTNode{  
    ElemType data;  
    struct BiTNode *lchild,*rchild;  
}BiTNode,*BiTree;
```

术语: 二叉链表

//线索二叉树结点

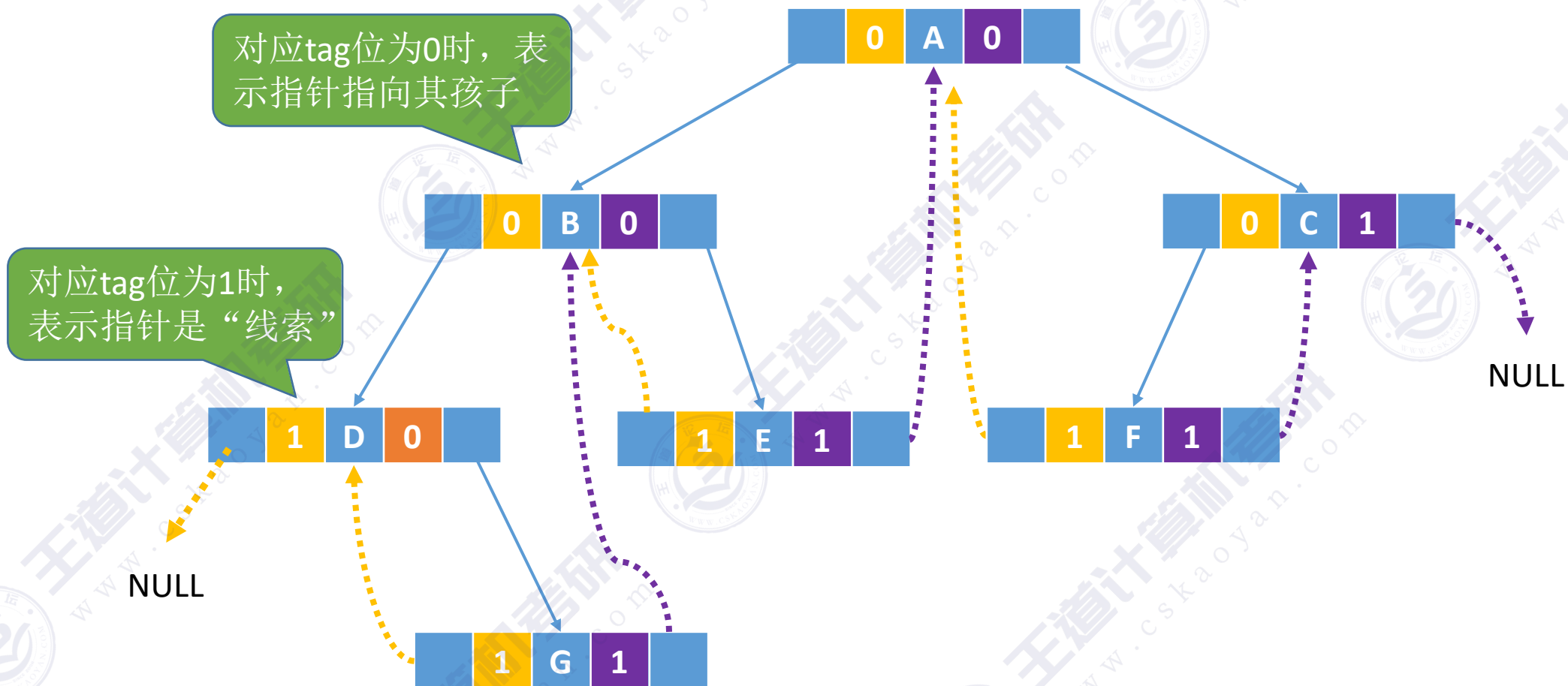
```
typedef struct ThreadNode{  
    ElemType data;  
    struct ThreadNode *lchild,*rchild;  
    int ltag,rtag; //左、右线索标志  
}ThreadNode,*ThreadTree;
```

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------

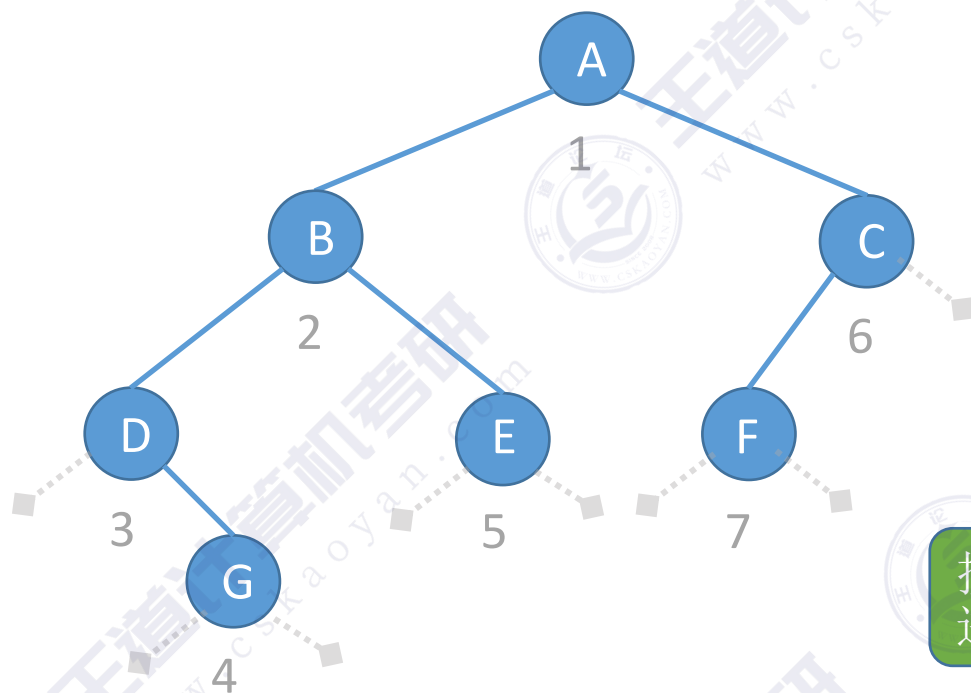
tag==0, 表示指针指向孩子
tag==1, 表示指针是“线索”

术语: 线索链表

中序线索二叉树的存储

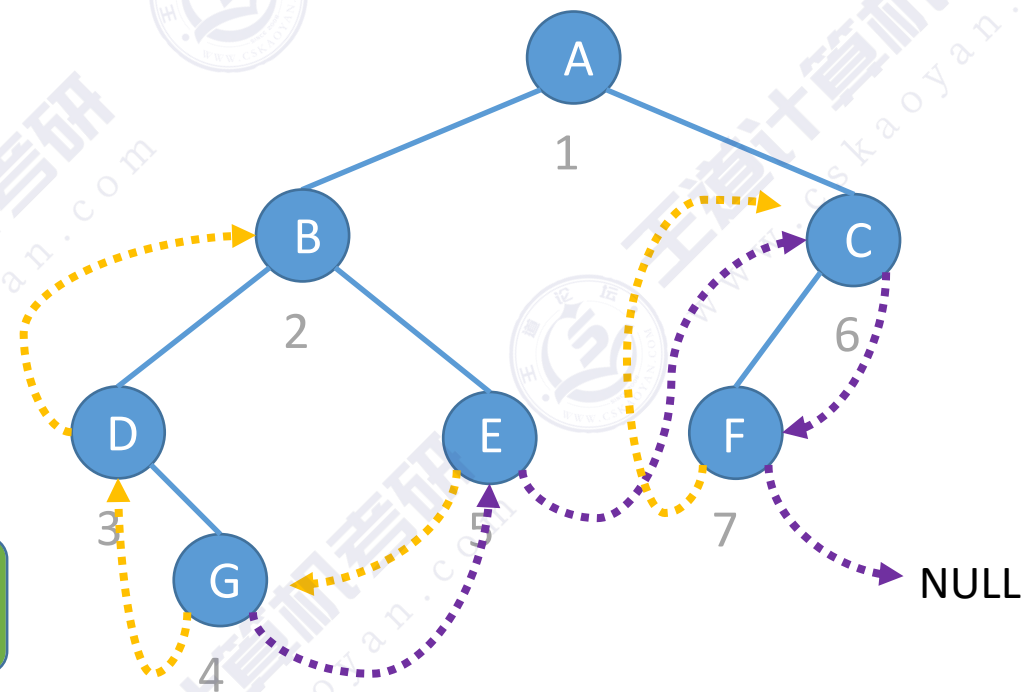


先序线索二叉树



线索化

按照“先序遍历”
进行线索化



先序遍历序列: A B D G E C F
先序遍历序列: A B D G E C F

图示说明

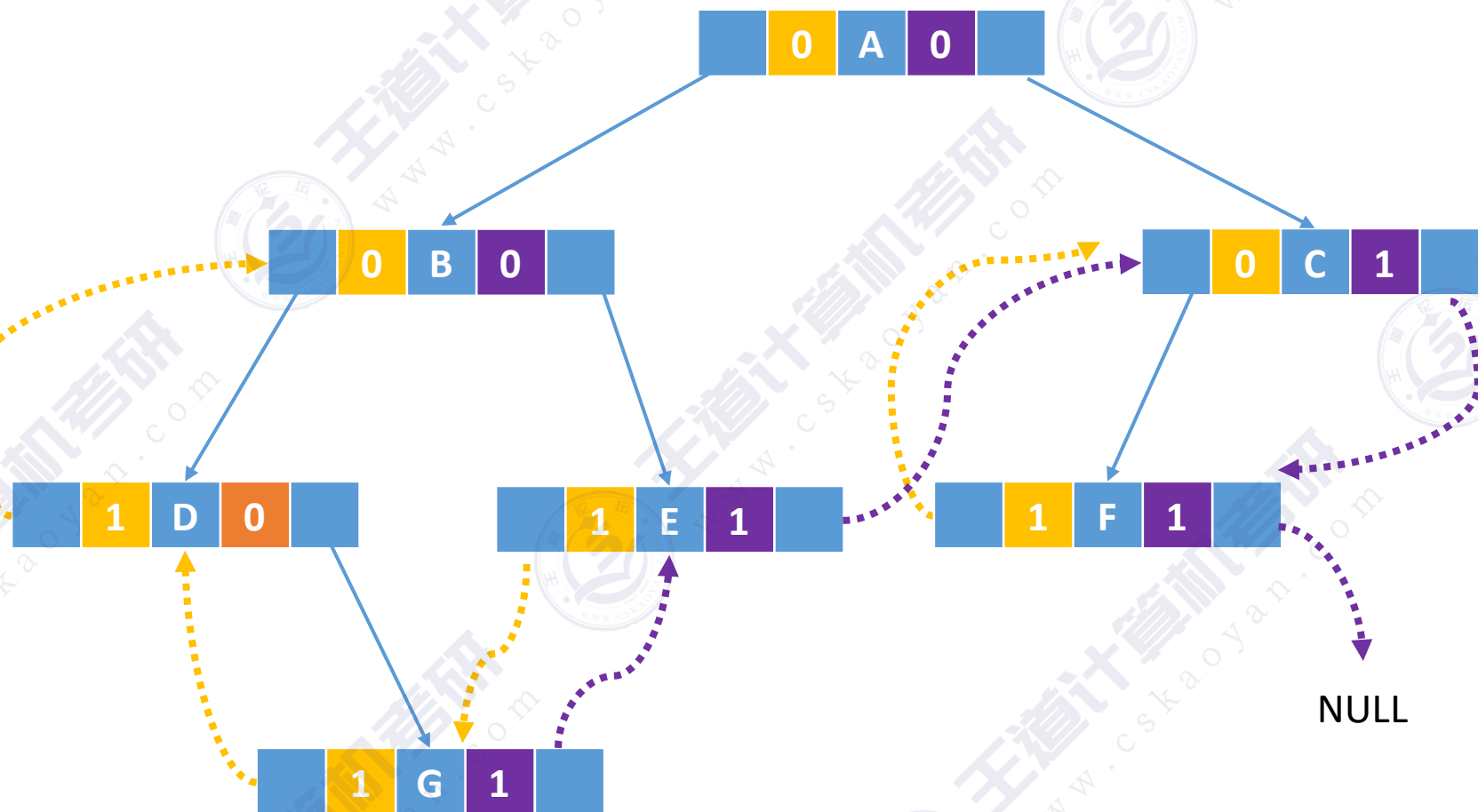
前驱**线索** (由左孩子指针充当):



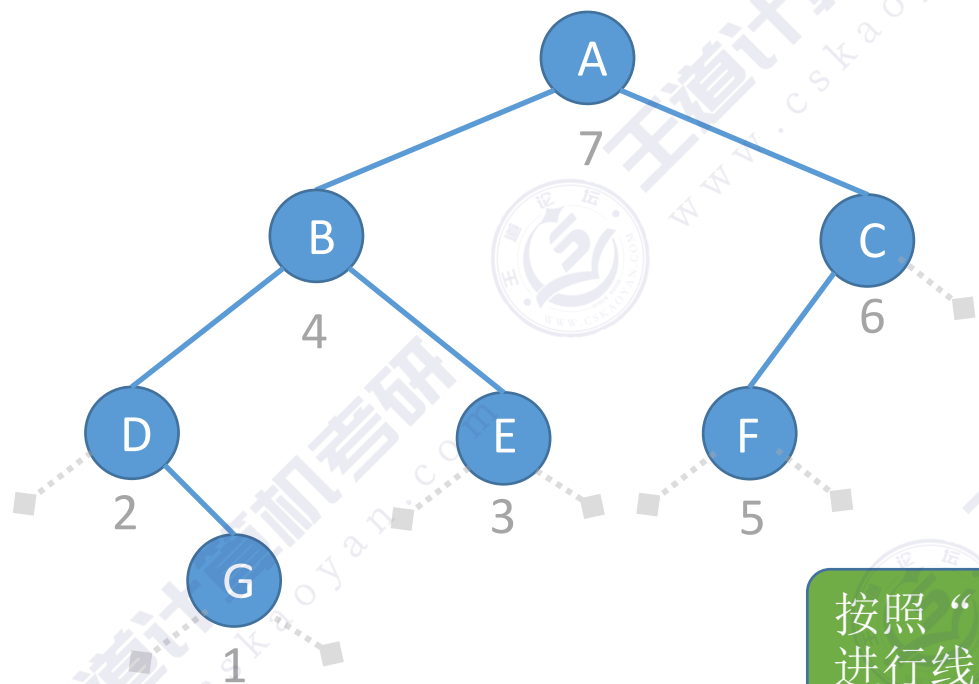
后继**线索** (由右孩子指针充当):



先序线索二叉树的存储

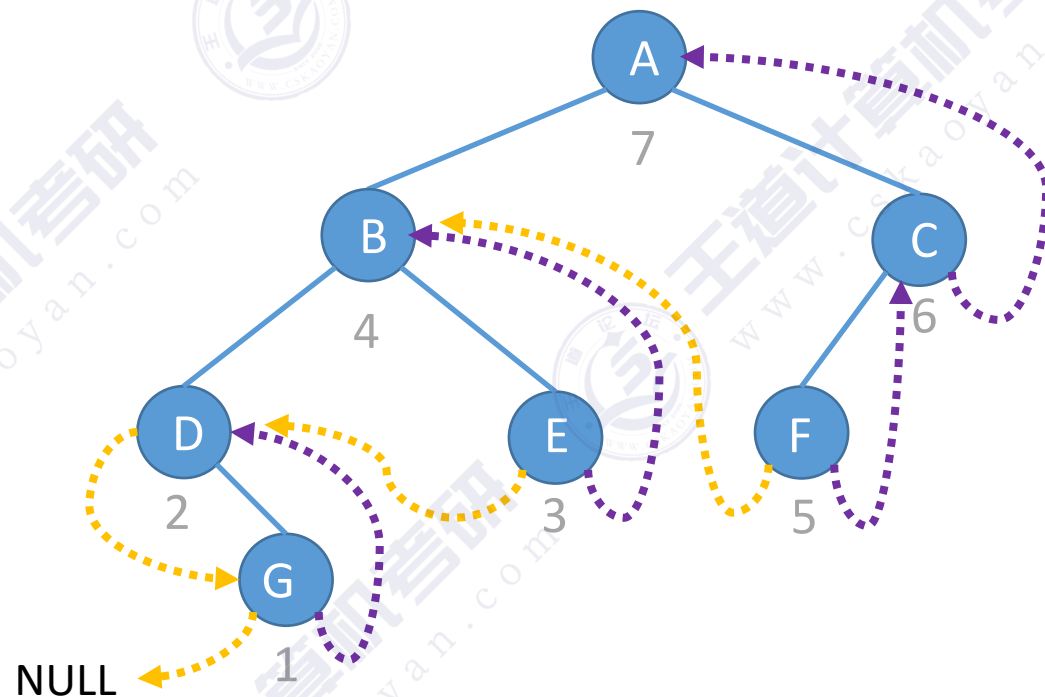


后序线索二叉树



线索化

按照“后序遍历”
进行线索化



后序遍历序列：G D E B F C A

图示说明

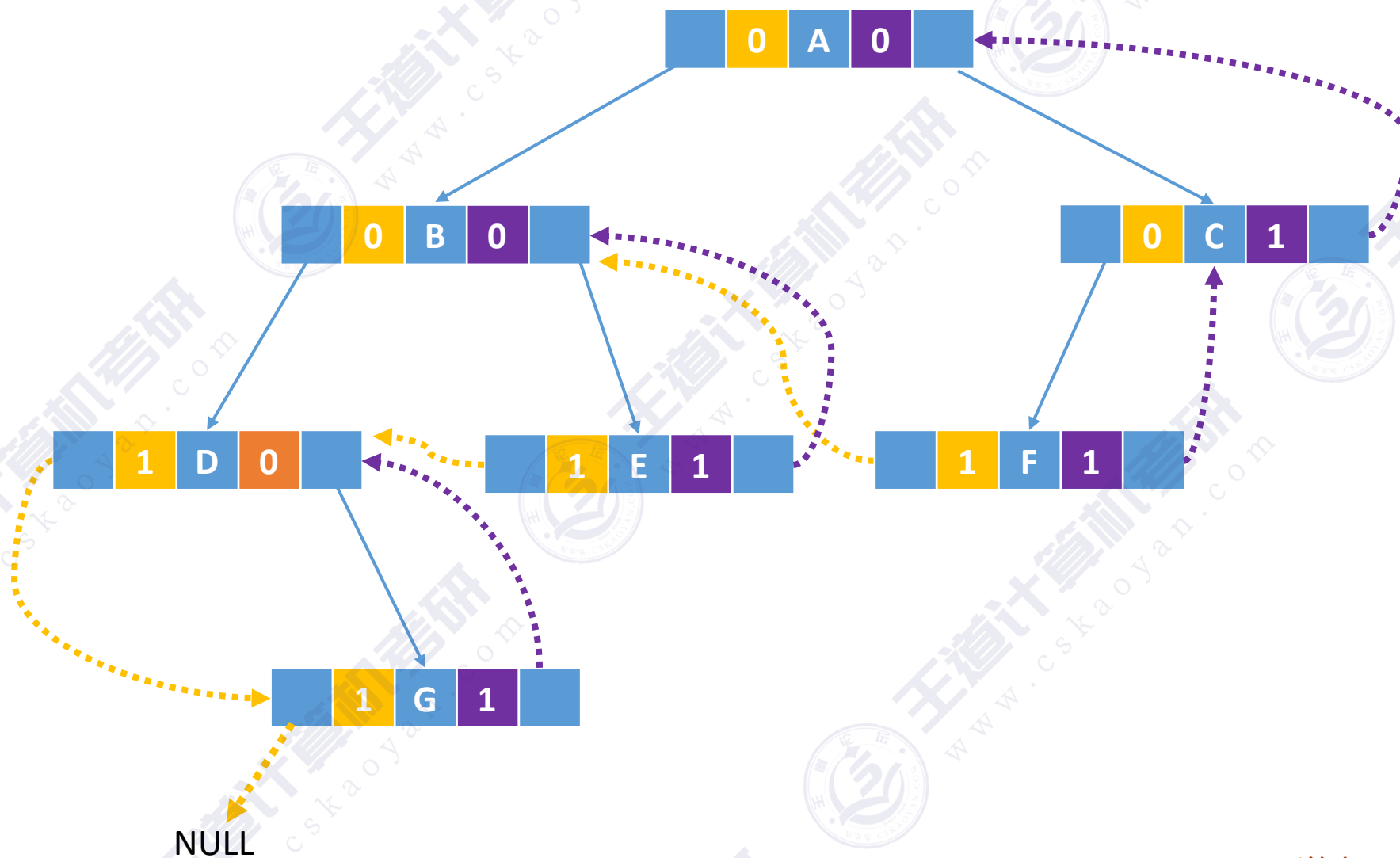
前驱线索（由左孩子指针充当）：



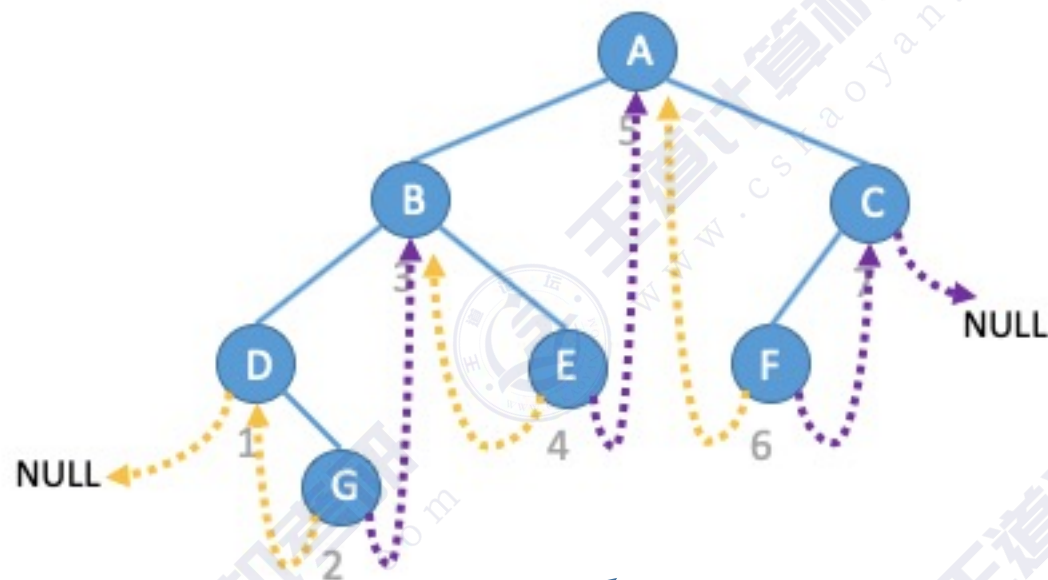
后继线索（由右孩子指针充当）：



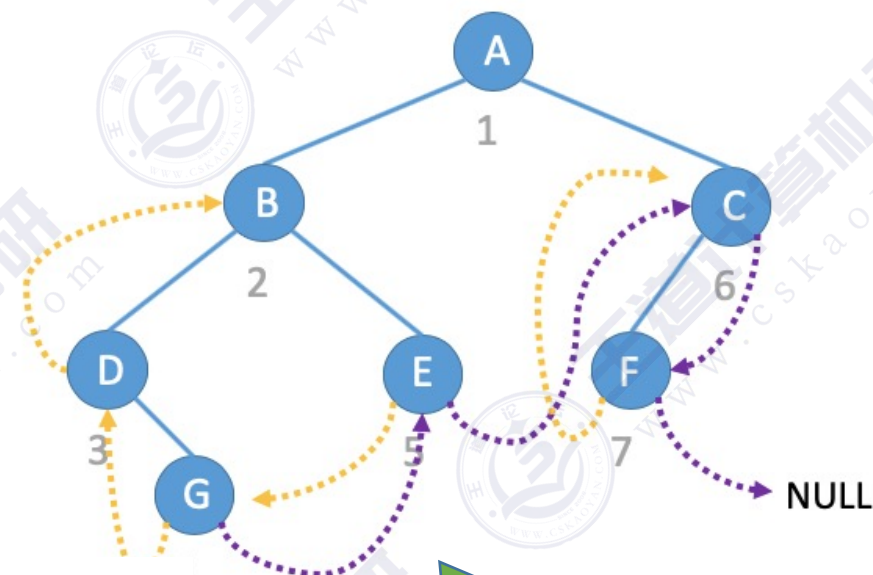
后序线索二叉树的存储



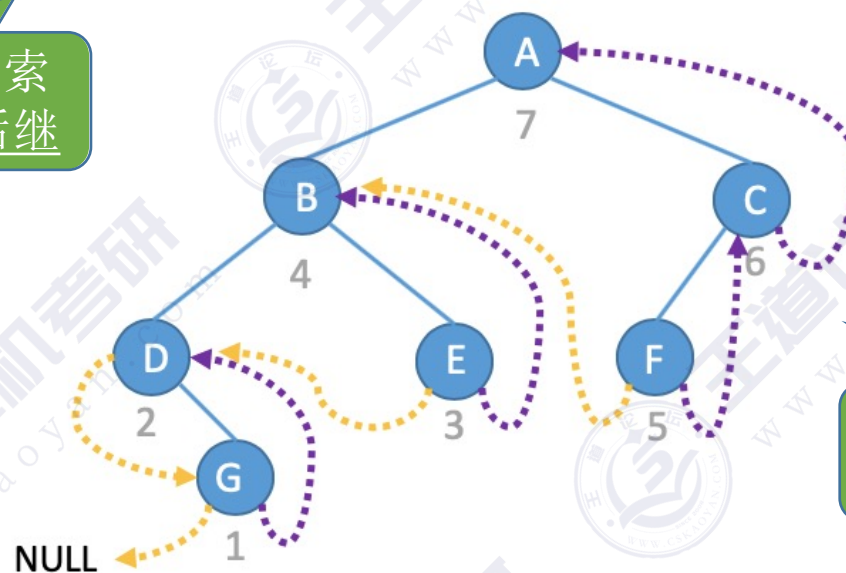
三种线索二叉树的对比



中序线索二叉树——线索指向中序前驱、中序后继



先序线索二叉树——线索指向先序前驱、先序后继



后序线索二叉树——线索指向后序前驱、后序后继

知识回顾与重要考点

作用：方便从一个指定结点出发，找到其前驱、后继；方便遍历

线索二叉树

存储结构

在普通二叉树结点的基础上，增加两个标志位 **ltag 和 rtag**

ltag==1时，表示lchild指向前驱；ltag==0时，表示lchild指向左孩子

rtag==1时，表示rchild指向后继；rtag==0时，表示rchild指向右孩子

三种线索二叉树

中序线索二叉树 以中序遍历序列为依据进行“线索化”

先序线索二叉树 以先序遍历序列为依据进行“线索化”

后序线索二叉树 以后序遍历序列为依据进行“线索化”

几个概念

线索 指向前驱/后继的指针称为线索

中序前驱/中序后继；先序前驱/先序后继；后序前驱/后序后继

手算画出线索二叉树

①确定线索二叉树类型——中序、先序、or 后序？

②按照对应遍历规则，确定各个结点的访问顺序，并写上编号

③将 $n+1$ 个空链域连上前驱、后继

本节内容

二叉树的 线索化

知识总览

二叉树线索化

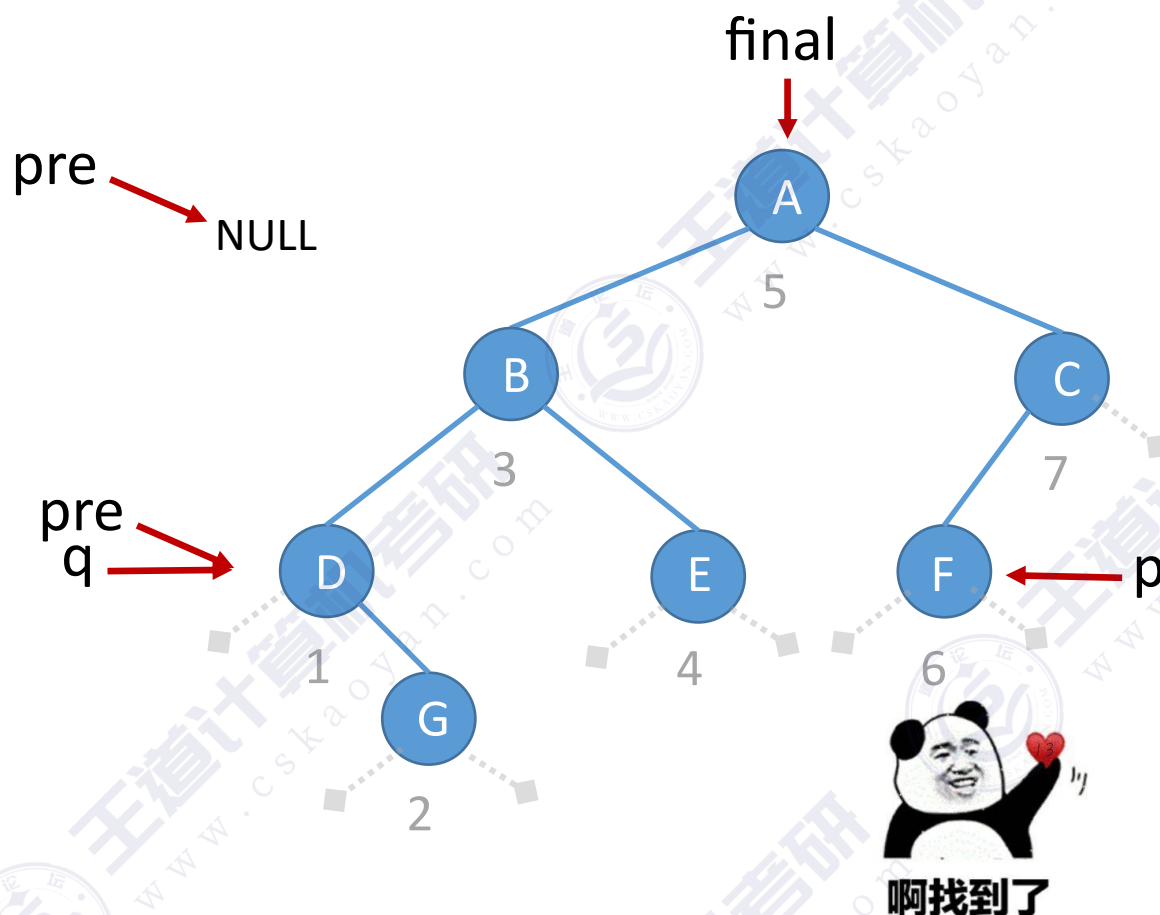
中序线索化

先序线索化

后序线索化

用土办法找到中序前驱

最好改一个函数名，如 findPre



中序遍历序列: D G B E A F C

//中序遍历

```
void InOrder(BiTree T){
```

```
    if(T!=NULL){
```

```
        InOrder(T->lchild);
```

//递归遍历左子树

```
        visit(T);
```

//访问根结点

```
        InOrder(T->rchild);
```

//递归遍历右子树

```
    }
```

```
}
```

//访问结点q

```
void visit(BiTNode * q){
```

```
    if (q==p)
```

//当前访问结点刚好是结点p

```
        final = pre;
```

//找到p的前驱

```
    else
```

```
        pre = q;
```

//pre指向当前访问的结点

```
}
```

//辅助全局变量，用于查找结点p的前驱

```
BiTNode *p;
```

//p指向目标结点

```
BiTNode * pre=NULL;
```

//指向当前访问结点的前驱

```
BiTNode * final=NULL;
```

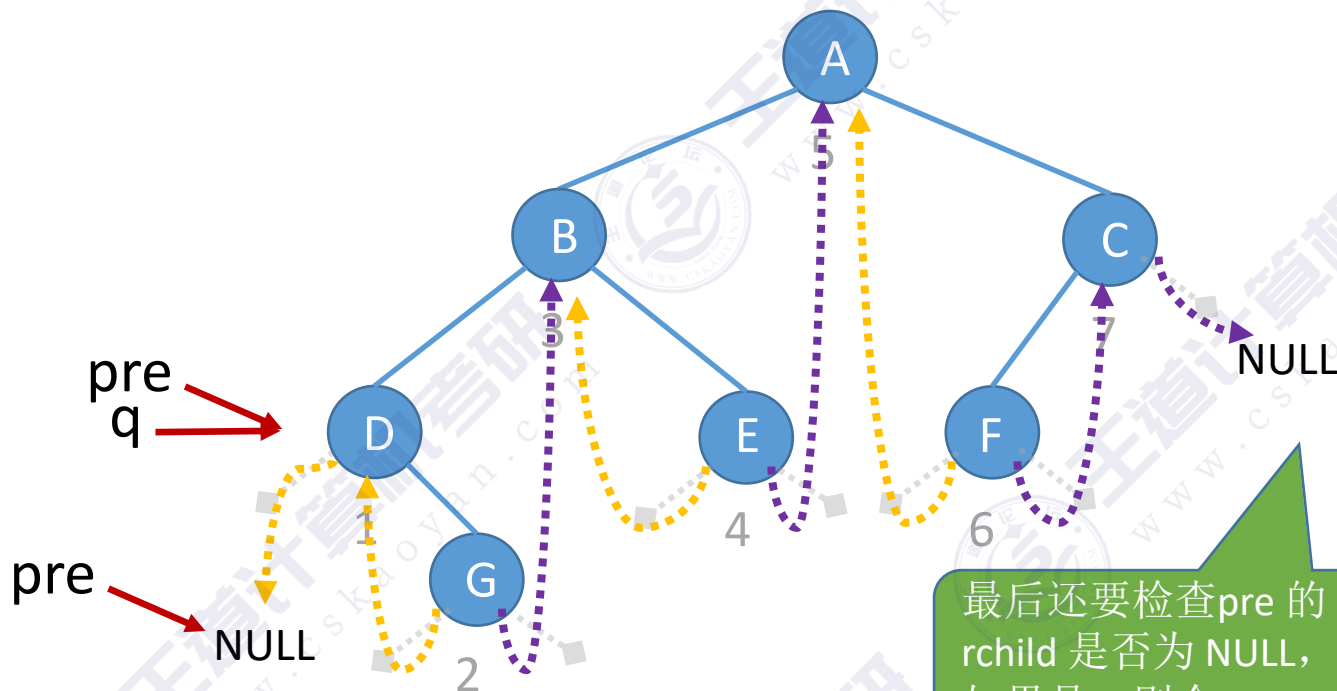
//用于记录最终结果

王道考研/CSKAOYAN.COM

初步建成的树，
ltag、rtag=0

中序线索化

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------



//线索二叉树结点

```
typedef struct ThreadNode{  
    ElemType data;  
    struct ThreadNode *lchild,*rchild;  
    int ltag,rtag;    //左、右线索标志  
}ThreadNode,* ThreadTree;
```

//中序遍历二叉树，一边遍历一边线索化

```
void InThread(ThreadTree T){  
    if(T!=NULL){  
        InThread(T->lchild);    //中序遍历左子树  
        visit(T);                //访问根节点  
        InThread(T->rchild);    //中序遍历右子树  
    }  
}
```

```
void visit(ThreadNode *q) {  
    if(q->lchild==NULL){//左子树为空，建立前驱线索  
        q->lchild=pre;  
        q->ltag=1;  
    }  
    if(pre!=NULL&&pre->rchild==NULL){  
        pre->rchild=q;    //建立前驱结点的后继线索  
        pre->rtag=1;  
    }  
    pre=q;  
}
```

//全局变量 pre，指向当前访问结点的前驱

```
ThreadNode *pre=NULL;
```


初步建成的树,
ltag、rtag=0

中序线索化

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------

//全局变量 pre, 指向当前访问结点的前驱

ThreadNode *pre=NULL;

//中序线索化二叉树T

```
void CreateInThread(ThreadTree T){
    pre=NULL;           //pre初始为NULL
    if(T!=NULL){        //非空二叉树才能线索化
        InThread(T);     //中序线索化二叉树
        if (pre->rchild==NULL)
            pre->rtag=1; //处理遍历的最后一个结点
    }
}
```

//线索二叉树结点

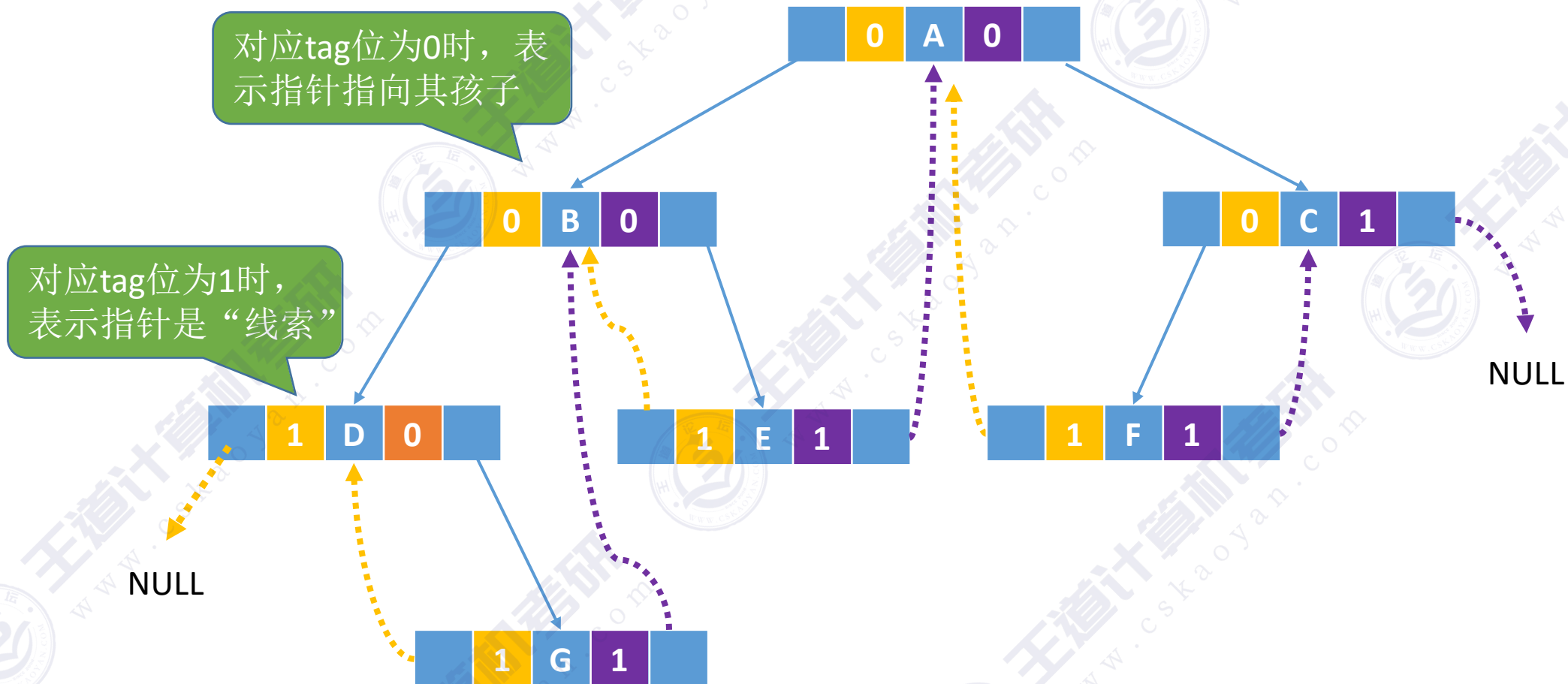
```
typedef struct ThreadNode{
    ElemType data;
    struct ThreadNode *lchild,*rchild;
    int ltag,rtag;      //左、右线索标志
}ThreadNode,* ThreadTree;
```

//中序遍历二叉树, 一边遍历一边线索化

```
void InThread(ThreadTree T){
    if(T!=NULL){
        InThread(T->lchild); //中序遍历左子树
        visit(T);           //访问根节点
        InThread(T->rchild); //中序遍历右子树
    }
}
```

```
void visit(ThreadNode *q) {
    if(q->lchild==NULL){ //左子树为空, 建立前驱线索
        q->lchild=pre;
        q->ltag=1;
    }
    if(pre!=NULL&&pre->rchild==NULL){
        pre->rchild=q; //建立前驱结点的后继线索
        pre->rtag=1;
    }
    pre=q;
}
```


中序线索二叉树



中序线索化（王道教材版）

//中序线索化

```
void InThread(ThreadTree p, ThreadTree &pre){
    if(p!=NULL){
        InThread(p->lchild, pre); //递归，线索化左子树
        if(p->lchild==NULL){ //左子树为空，建立前驱线索
            p->lchild=pre;
            p->ltag=1;
        }
        if(pre!=NULL && pre->rchild==NULL){ //建立前驱结点的后继线索
            pre->rchild=p;
            pre->rtag=1;
        }
        pre=p;
        InThread(p->rchild, pre);
    } //if(p!=NULL)
}
```

思考：为什么 pre 参数是引用类型？

思考：处理遍历的最后一个结点时，为什么没有判断 rchild 是否为NULL？

答：中序遍历的最后一个结点右孩子指针必为空。

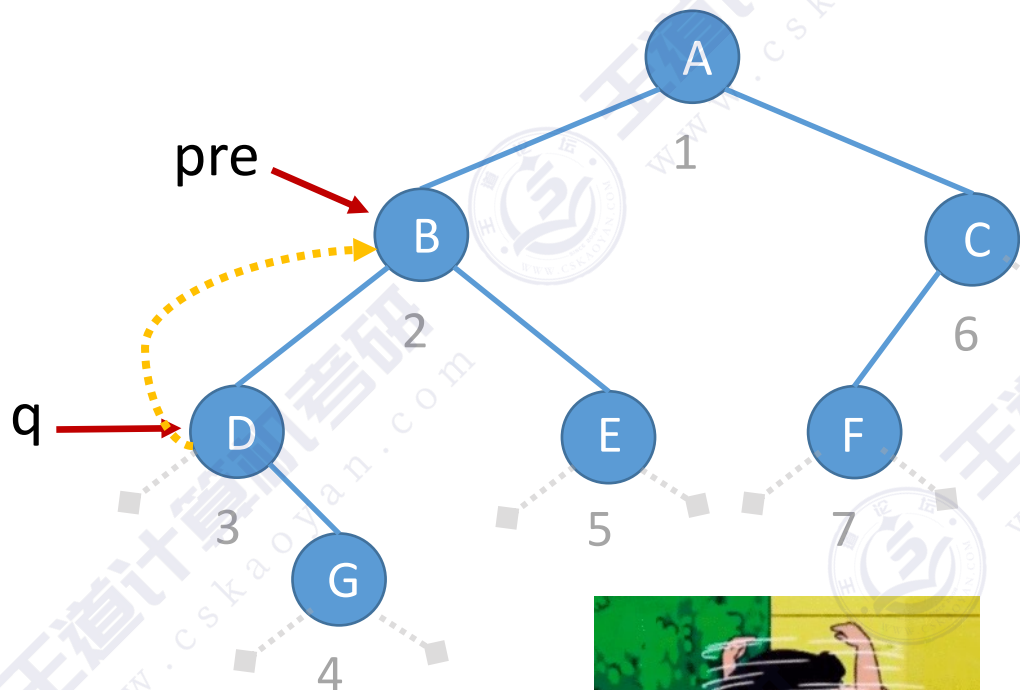
//中序线索化二叉树T

```
void CreateInThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){
        InThread(T, pre); //非空二叉树，线索化
        pre->rchild=NULL; //线索化二叉树
        pre->rtag=1; //处理遍历的最后一个结点
    }
}
```

初步建成的树，
ltag、rtag=0

先序线索化

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------



爱滴魔力转圈圈

//先序遍历二叉树，一边遍历一边线索化

```
void PreThread(ThreadTree T){
```

```
    if(T!=NULL){
```

```
        visit(T);          //先处理根节点
```

```
        PreThread(T->lchild);
```

```
        PreThread(T->rchild);
```

```
    }
```

```
void visit(ThreadNode *q) {
```

```
    if(q->lchild==NULL){//左子树为空，建立前驱线索
```

```
        q->lchild=pre;
```

```
        q->ltag=1;
```

```
    }
```

```
    if(pre!=NULL&&pre->rchild==NULL){
```

```
        pre->rchild=q;  //建立前驱结点的后继线索
```

```
        pre->rtag=1;
```

```
    }
```

```
    pre=q;
```

```
}
```

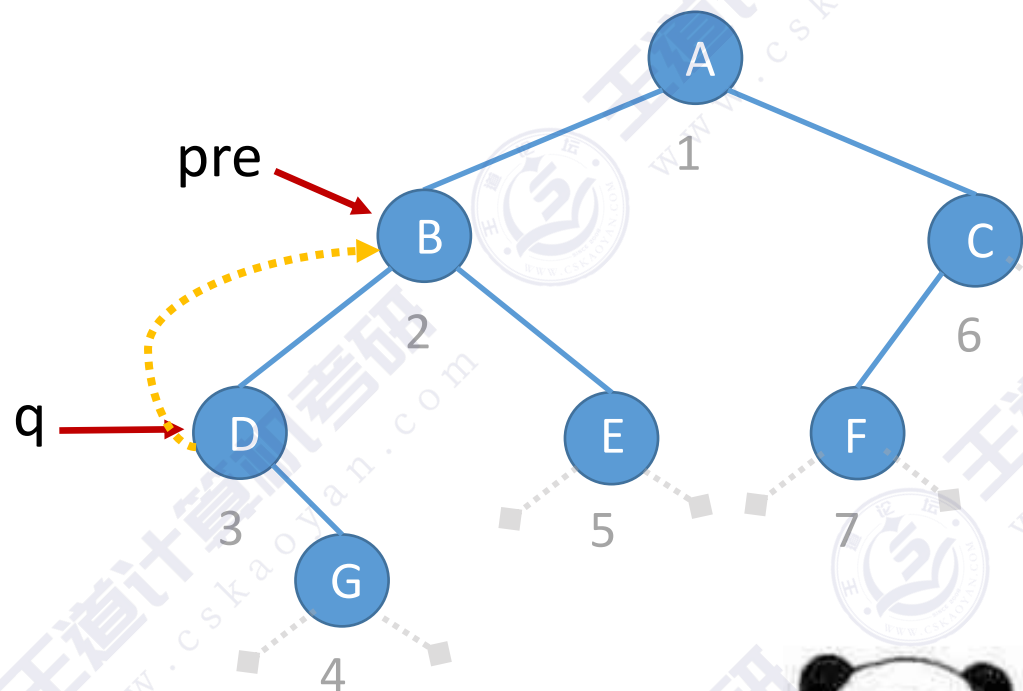
//全局变量 pre，指向当前访问结点的前驱

```
ThreadNode *pre=NULL;
```

初步建成的树，
ltag、rtag=0

先序线索化

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------



可以

//先序遍历二叉树，一边遍历一边线索化

```
void PreThread(ThreadTree T){  
    if(T!=NULL){  
        visit(T);           //先处理根节点  
        PreThread(T->lchild);  
        PreThread(T->rchild);  
    }  
}
```

//先序遍历二叉树，一边遍历一边线索化

```
void PreThread(ThreadTree T){  
    if(T!=NULL){  
        visit(T);           //先处理根节点  
        if (T->ltag==0) //lchild不是前驱线索  
            PreThread(T->lchild);  
        PreThread(T->rchild);  
    }  
}
```

先序线索化

初步建成的树，
ltag、rtag=0

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------

//全局变量 pre, 指向当前访问结点的前驱
ThreadNode *pre=NULL;

//先序线索化二叉树T

```
void CreatePreThread(ThreadTree T){
    pre=NULL;           //pre初始为NULL
    if(T!=NULL){        //非空二叉树才能线索化
        PreThread(T);    //先序线索化二叉树
        if(pre->rchild==NULL)
            pre->rtag=1; //处理遍历的最后一个结点
    }
}
```

//先序遍历二叉树，一边遍历一边线索化

```
void PreThread(ThreadTree T){
    if(T!=NULL){
        visit(T);           //先处理根节点
        if(T->ltag==0) //lchild不是前驱线索
            PreThread(T->lchild);
        PreThread(T->rchild);
    }
}

void visit(ThreadNode *q) {
    if(q->lchild==NULL){ //左子树为空，建立前驱线索
        q->lchild=pre;
        q->ltag=1;
    }
    if(pre!=NULL&&pre->rchild==NULL){
        pre->rchild=q; //建立前驱结点的后继线索
        pre->rtag=1;
    }
    pre=q;
}
```


先序线索化（王道教材Style）

//先序线索化

```
void PreThread(ThreadTree p, ThreadTree &pre){
    if(p!=NULL){
        if(p->lchild==NULL){           //左子树为空，建立前驱线索
            p->lchild=pre;
            p->ltag=1;
        }
        if(pre!=NULL&&pre->rchild==NULL){
            pre->rchild=p;             //建立前驱结点的后继线索
            pre->rtag=1;
        }
        pre=p;
        if(p->ltag==0)
            PreThread(p->lchild, pre); //递归，
        PreThread(p->rchild, pre);     //递归，
    } //if(p!=NULL)
}
```



//先序线索化二叉树T

```
void CreatePreThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){
        PreThread(T, pre);
        if(pre->rchild==NULL)
            pre->rtag=1;
    }
}
```


后序线索化

初步建成的树，
ltag、rtag=0

* lchild	ltag	data	rtag	* rchild
----------	------	------	------	----------

//全局变量 pre, 指向当前访问结点的前驱
ThreadNode *pre=NULL;

//后序线索化二叉树T

```
void CreatePostThread(ThreadTree T){
    pre=NULL;           //pre初始为NULL
    if(T!=NULL){        //非空二叉树才能线索化
        PostThread(T);  //后序线索化二叉树
        if (pre->rchild==NULL)
            pre->rtag=1; //处理遍历的最后一个结点
    }
}
```

//后遍历二叉树，一边遍历一边线索化

```
void PostThread(ThreadTree T){
    if(T!=NULL){
        PostThread(T->lchild); //后序遍历左子树
        PostThread(T->rchild); //后序遍历右子树
        visit(T);              //访问根节点
    }
}

void visit(ThreadNode *q) {
    if(q->lchild==NULL){ //左子树为空，建立前驱线索
        q->lchild=pre;
        q->ltag=1;
    }
    if(pre!=NULL&&pre->rchild==NULL){
        pre->rchild=q; //建立前驱结点的后继线索
        pre->rtag=1;
    }
    pre=q;
}
```

后序线索化（王道教材Style）

//后序线索化

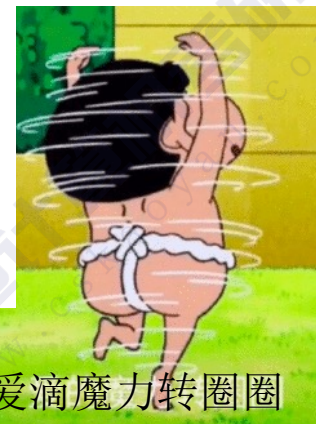
```
void PostThread(ThreadTree p, ThreadTree &pre){
    if(p!=NULL){
        PostThread(p->lchild, pre); //递归，线索化左子树
        PostThread(p->rchild, pre); //递归，线索化右子树
        if(p->lchild==NULL){ //左子树为空，建立前驱线索
            p->lchild=pre;
            p->ltag=1;
        }
        if(pre!=NULL&&pre->rchild==NULL){
            pre->rchild=p;
            pre->rtag=1;
        }
        pre=p;
    } //if(p!=NULL)
}
```

//后序线索化二叉树T

```
void CreatePostThread(ThreadTree T){
    ThreadTree pre=NULL;
    if(T!=NULL){ //非空二叉树，线索化
        PostThread(T, pre); //线索化二叉树
        if(pre->rchild==NULL) //处理遍历的最后一个结点
            pre->rtag=1;
    }
}
```



不存在的



爱滴魔力转圈圈

知识回顾与重要考点

二叉树线索化

中序线索化 得到中序线索二叉树

先序线索化 得到先序线索二叉树

后序线索化 得到后序线索二叉树

核心

中序/先序/后序遍历算法的改造，当访问一个结点时，连接该结点与前驱结点的线索信息

用一个指针 pre 记录当前访问结点的前驱结点

易错点

最后一个结点的 rchild、rtag 的处理

先序线索化中，注意处理爱滴魔力转圈圈问题，当 ltag==0时，才能对左子树先序线索化

本节内容

线索二叉树

找前驱/后继

知识总览

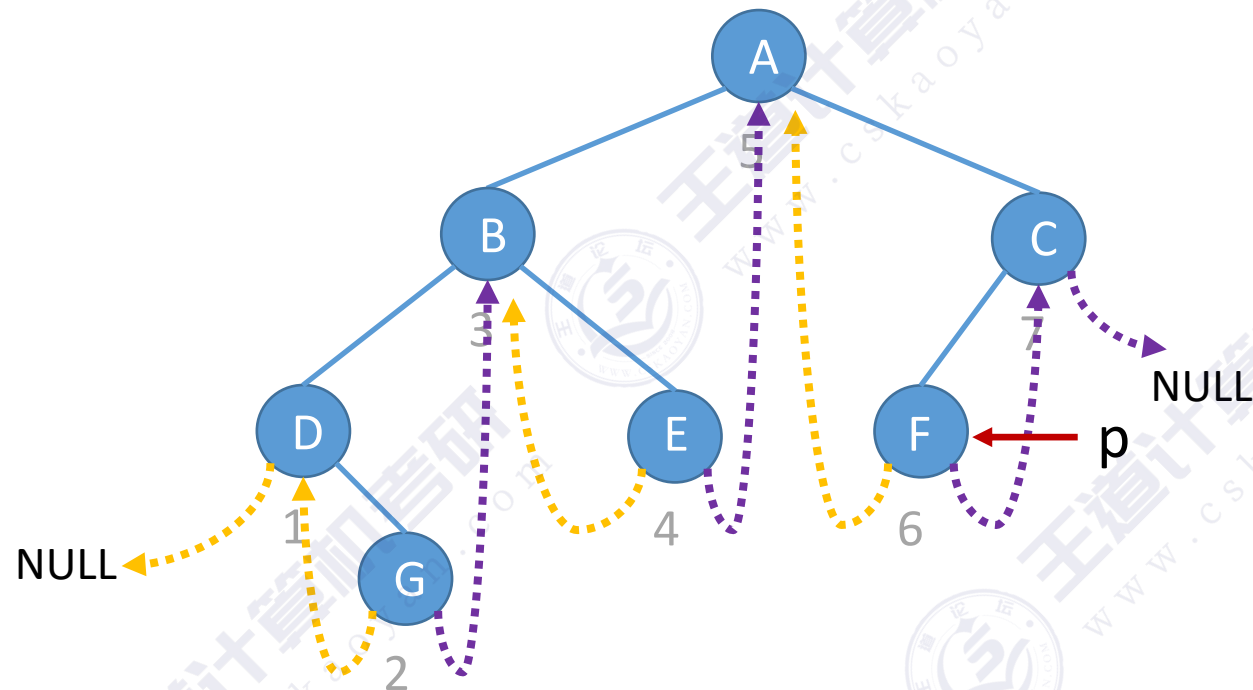
线索二叉树找前驱/后继

中序线索二叉树

先序线索二叉树

后序线索二叉树

中序线索二叉树找中序后继

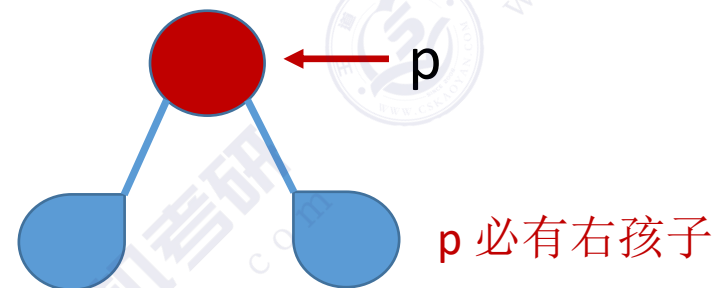


中序遍历序列: D G B E A F C

在中序线索二叉树中找到指定结点*p
的中序后继 next

①若 $p \rightarrow rtag == 1$, 则 $next = p \rightarrow rchild$

②若 $p \rightarrow rtag == 0$



中序遍历——左 根 右

左 根 (左 根 右)

左 根 ((左 根 右) 根 右)

$next = p$ 的右子树中最左下结点

中序线索二叉树找中序后继

//找到以P为根的子树中，第一个被中序遍历的结点

```
ThreadNode *Firstnode(ThreadNode *p){  
    //循环找到最左下结点(不一定是叶结点)  
    while(p->ltag==0) p=p->lchild;  
    return p;  
}
```

//在中序线索二叉树中找到结点p的后继结点

```
ThreadNode *Nextnode(ThreadNode *p){  
    //右子树中最左下结点  
    if(p->rtag==0) return Firstnode(p->rchild);  
    else return p->rchild; //rtag==1直接返回后继线索  
}
```

//对中序线索二叉树进行中序遍历（利用线索实现的非递归算法）

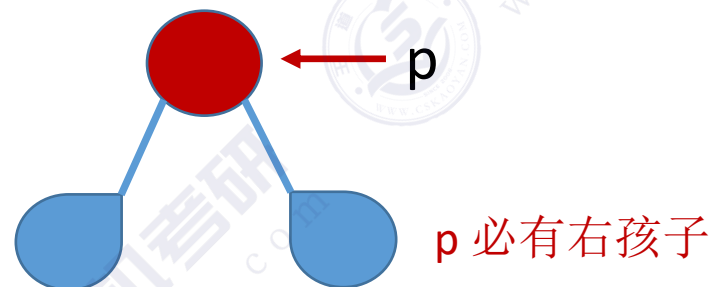
```
void Inorder(ThreadNode *T){  
    for(ThreadNode *p=Firstnode(T);p!=NULL; p=Nextnode(p))  
        visit(p);  
}
```

空间复杂度O(1)

在中序线索二叉树中找到指定结点*p的中序后继 next

①若 $p \rightarrow rtag == 1$ ，则 $next = p \rightarrow rchild$

②若 $p \rightarrow rtag == 0$



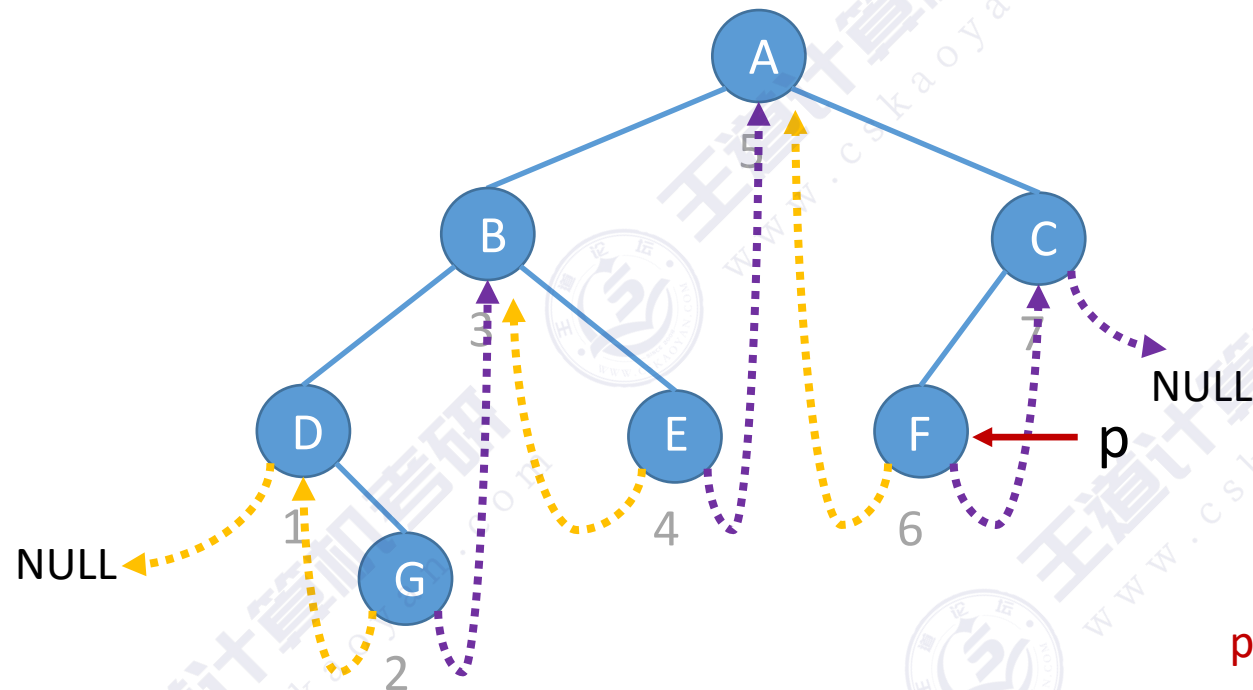
中序遍历——左 根 右

左 根 (左 根 右)

左 根 ((左 根 右) 根 右)

next = p的右子树中最左下结点

中序线索二叉树找中序前驱



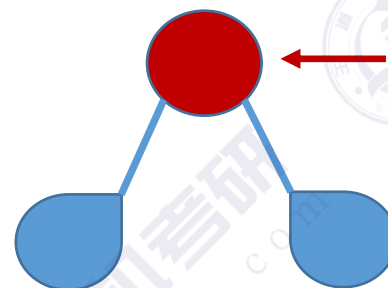
中序遍历序列: D G B E A F C

在中序线索二叉树中找到指定结点*p
的中序前驱 pre

①若 $p \rightarrow ltag == 1$, 则 $pre = p \rightarrow lchild$

②若 $p \rightarrow ltag == 0$

p 必有左孩子



中序遍历——左 根 右

(左 根 右) 根 右

(左 根 (左 根 右)) 根 右

pre = p 的左子树中最右下结点

中序线索二叉树找中序前驱

//找到以P为根的子树中，最后一个被中序遍历的结点

```
ThreadNode *Lastnode(ThreadNode *p){  
    //循环找到最右下结点(不一定是叶结点)  
    while(p->rtag==0) p=p->rchild;  
    return p;  
}
```

//在中序线索二叉树中找到结点p的前驱结点

```
ThreadNode *Prenode(ThreadNode *p){  
    //左子树中最右下结点  
    if(p->ltag==0) return Lastnode(p->lchild);  
    else return p->lchild;    //ltag==1直接返回前驱线索  
}
```

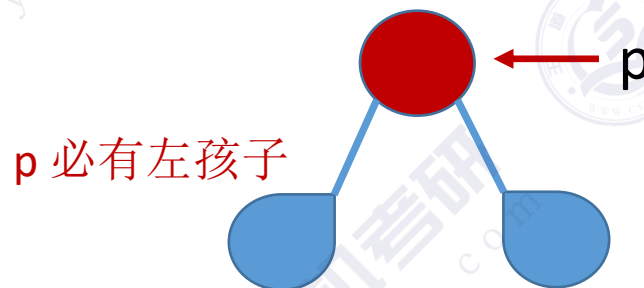
//对中序线索二叉树进行逆向中序遍历

```
void RevInorder(ThreadNode *T){  
    for(ThreadNode *p=Lastnode(T);p!=NULL;p=Prenode(p))  
        visit(p);  
}
```

在中序线索二叉树中找到指定结点*p
的中序前驱 pre

①若 $p \rightarrow ltag == 1$ ，则 $pre = p \rightarrow lchild$

②若 $p \rightarrow ltag == 0$



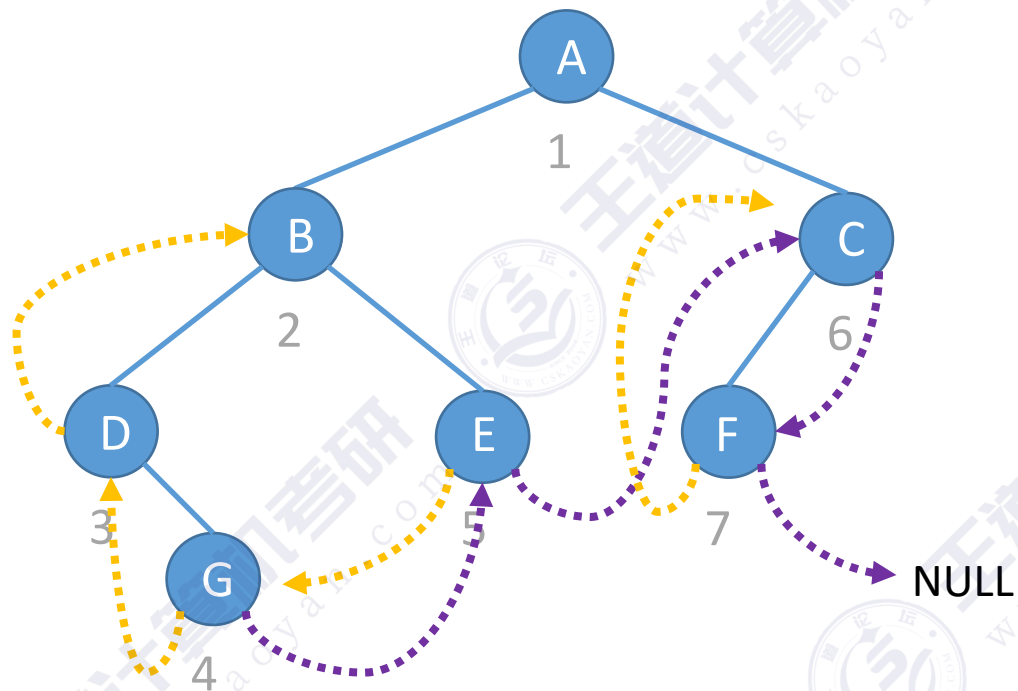
中序遍历——左 根 右

(左 根 右) 根 右

(左 根 (左 根 右)) 根 右

pre = p的左子树中最右下结点

先序线索二叉树找先序后继



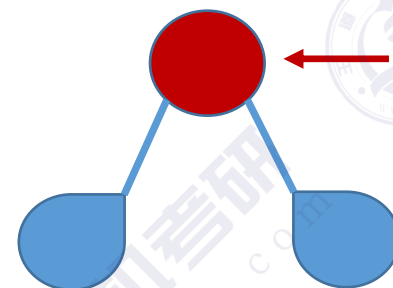
先序遍历序列: A B D G E C F



在先序线索二叉树中找到指定结点*p
的先序后继 next

①若 $p \rightarrow rtag == 1$, 则 $next = p \rightarrow rchild$

②若 $p \rightarrow rtag == 0$



p 必有右孩子

若p有左孩子,
则先序后继为
左孩子

先序遍历——根 左 右
根 (根 左 右) 右

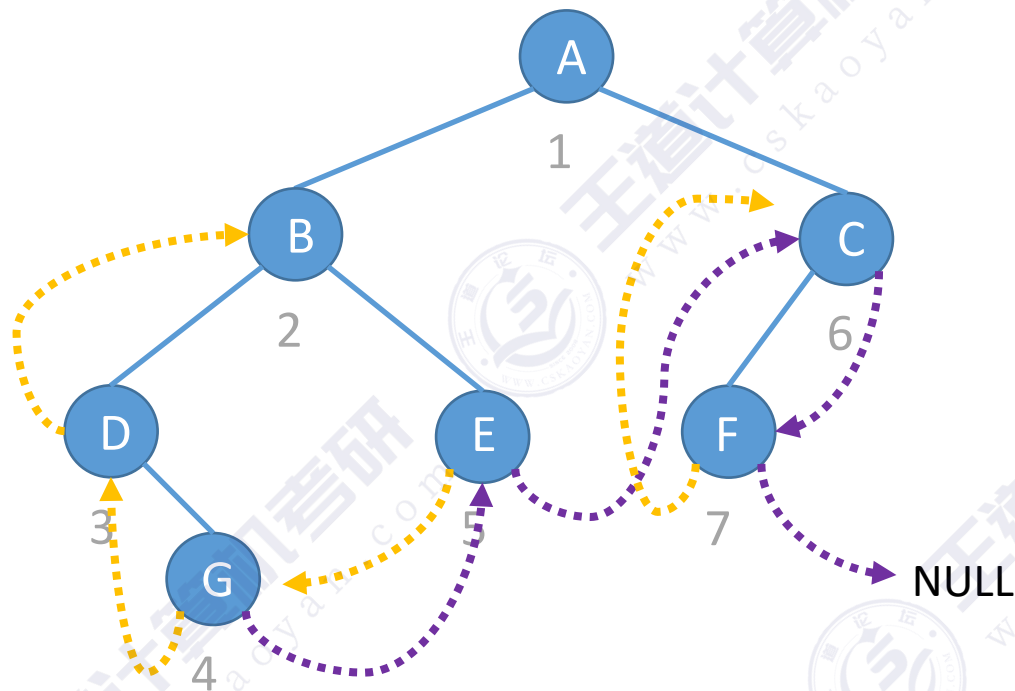
假设有左孩子

若p没有左孩
子, 则先序后
继为右孩子

先序遍历——根 右
根 (根 左 右)

假设没有左孩子

先序线索二叉树找先序前驱



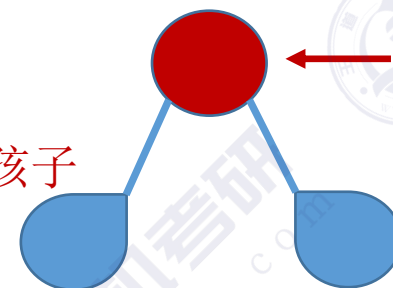
先序遍历序列: A B D G E C F

在先序线索二叉树中找到指定结点*p
的先序前驱 pre

①若 $p \rightarrow ltag == 1$, 则 $next = p \rightarrow lchild$

②若 $p \rightarrow ltag == 0$

p 必有左孩子



先序遍历——根 左 右



荒唐的答案

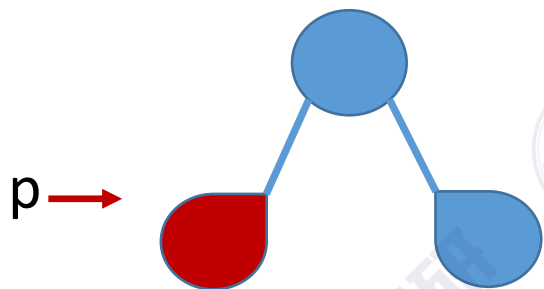
除非用土办法
从头开始先序
遍历

先序遍历中, 左右子树
中的结点只可能是根
的后继, 不可能是前驱

改用三叉链表可以找到父节点

先序线索二叉树找先序前驱

①如果能找到 p 的父节点，且 p 是左孩子

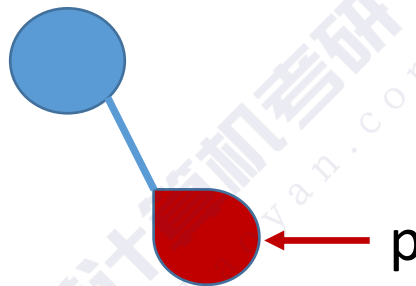


先序遍历——根 左 右

根 (根 左 右) 右

p 的父节点即为其前驱

②如果能找到 p 的父节点，且 p 是右孩子，其左兄弟为空

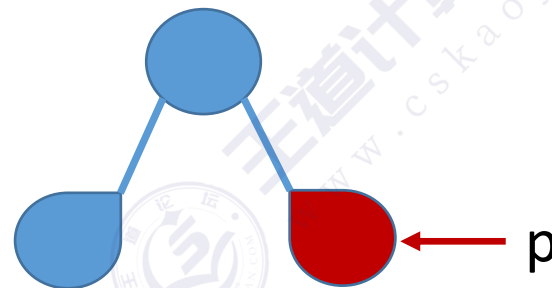


先序遍历——根 右

根 (根 左 右)

p 的父节点即为其前驱

③如果能找到 p 的父节点，且 p 是右孩子，其左兄弟非空

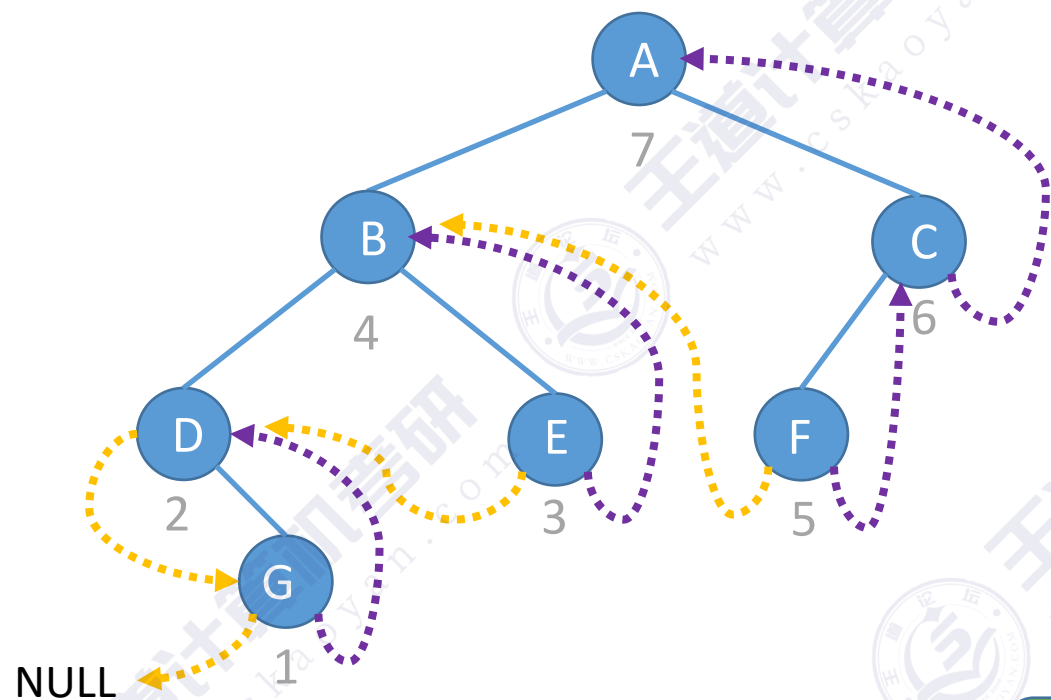


根 左 右

p 的前驱为左兄弟子树中最后一个被先序遍历的结点

④如果 p 是根节点，则 p 没有先序前驱

后序线索二叉树找后序前驱



后序遍历序列: G D E B F C A

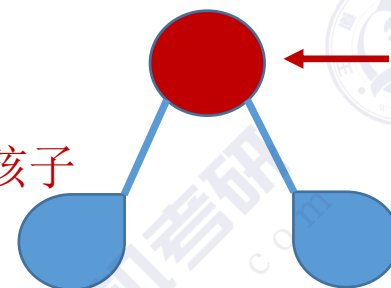


在后序线索二叉树中找到指定结点*p
的**后序前驱** pre

①若 $p \rightarrow ltag == 1$, 则 $pre = p \rightarrow lchild$

②若 $p \rightarrow ltag == 0$

p 必有左孩子



若p有右孩子,
则后序前驱为
右孩子

后序遍历——左 右 根

假设有右孩子

左 (左 右 根) 根

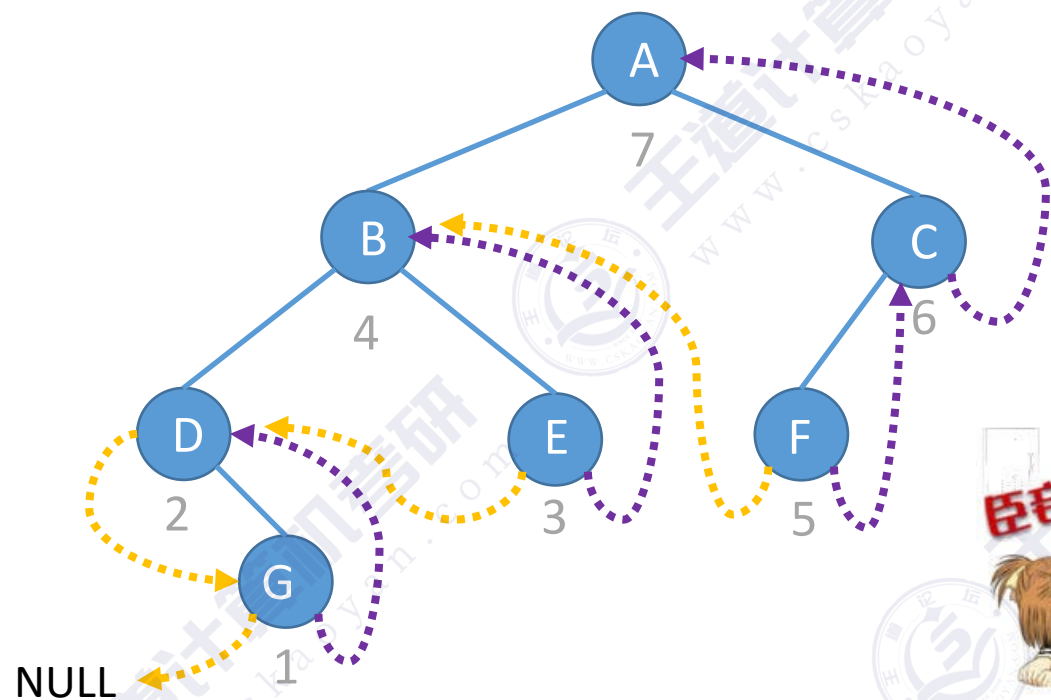
若p没有右孩
子, 则后序前
驱为左孩子

后序遍历——左 根

假设没有右孩子

(左 右 根) 根

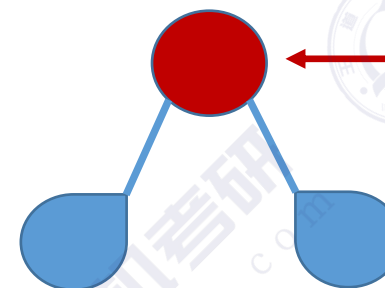
后序线索二叉树找后序后继



在后序线索二叉树中找到指定结点*p
的**后序后继** next

①若 $p \rightarrow rtag == 1$, 则 $next = p \rightarrow rchild$

②若 $p \rightarrow rtag == 0$



p 必有右孩子

后序遍历——左 右 根



荒唐的答案

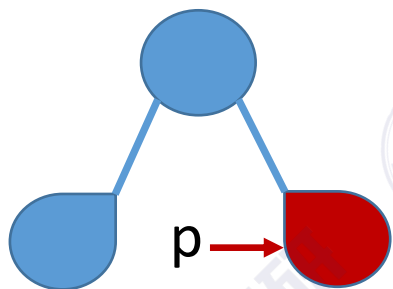
除非用土办法
从头开始先序
遍历

后序遍历中, 左右子树
中的结点只可能是根的
前驱, 不可能是后继

改用三叉链表可以
找到父节点

后序线索二叉树找后序后继

①如果能找到 p 的父节点，
且 p 是右孩子

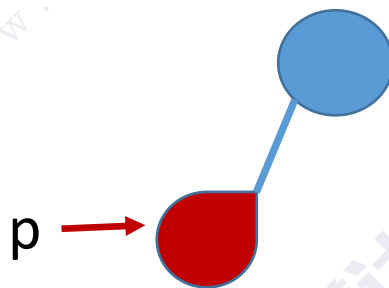


后序遍历——左 右 根

左 (左 右 根) 根

p 的父节点即为其后继

②如果能找到 p 的父节点，且
 p 是左孩子，其右兄弟为空

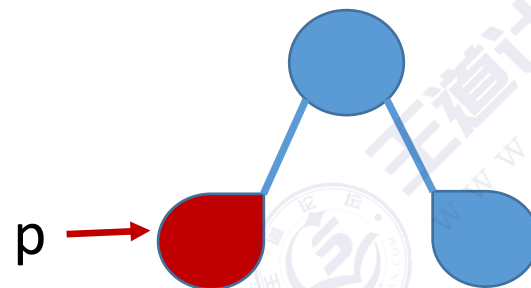


后序遍历——左 根

(左 右 根) 根

p 的父节点即为其后继

③如果能找到 p 的父节点，且
 p 是左孩子，其右兄弟非空



左 右 根

p 的后继为右兄弟子树中
第一个被后序遍历的结点

④如果 p 是根节点，则 p 没有后序后继

知识回顾与重要考点

若 $ltag/rtag == 0$

中序线索
二叉树

后继

p的右子树中最左下结点

前驱

p的左子树中最右下结点

先序线索
二叉树

后继

①若p有左孩子，则先序后继为左孩子

②若p没有左孩子，则先序后继为右孩子

前驱

①如果能找到p的父节点，且p是左孩子，则p的父节点为其前驱

②如果能找到p的父节点，且p是右孩子，其左兄弟为空，则p的父节点即为其前驱

③如果能找到p的父节点，且p是右孩子，其左兄弟非空，则p的前驱为其左兄弟子树最后一个被先序遍历的结点

④如果p是根节点，则p没有先序前驱

后序线索
二叉树

后继

①如果能找到p的父节点，且p是右孩子，p的父节点即为其后继

②如果能找到p的父节点，且p是左孩子，其右兄弟为空，p的父节点即为其后继

③如果能找到p的父节点，且p是左孩子，其右兄弟非空，则p的后继为其右兄弟子树中第一个被后序遍历的结点

④如果p是根节点，则p没有后序后继

前驱

①若p有右孩子，则后序前驱为右孩子

②若p没有右孩子，则后序前驱为左孩子

知识回顾与重要考点

	中序线索二叉树	先序线索二叉树	后序线索二叉树
找前驱	✓	✗	✓
找后继	✓	✓	✗

除非用三叉链表，
或者用土办法从根
开始遍历寻找

线索二叉树高频考点

线索化

! 手算

代码

! 找前驱、找后继